

VECTOR

APL, J and K ...

- | | |
|------------------------------------|-----|
| • Matrikx Reloaded – Garland on K | 49 |
| • Clark on FCRC 2003 at San Diego | 56 |
| • Kromberg on Libraries | 62 |
| • Scholes on the Dfns Newsgroup | 74 |
| • Taming Barrels with Cliff Reiter | 87 |
| • Robertson's New Foundations | 132 |



*The Journal of the
British APL Association*

ISSN 0955-1433

www.vector.org.uk

Vol.20 No.1 July 2003

A specialist Group of the British Computer Society

Contributions

All contributions to VECTOR may be sent to the Journal Editor at the address on the inside back cover. Letters and articles are welcome on any topic of interest to the APL community. These do not need to be limited to APL themes, nor must they be supportive of the language. Articles should be accompanied by as much visual material as possible (b/w or colour prints welcome). Unless otherwise specified, each item will be considered for publication as a personal statement by the author. The Editor accepts no responsibility for the contents of sustaining members' news, or advertising.

Please supply as much material as possible in machine-readable form, ideally as a simple ASCII text file on an IBM PC compatible diskette or via email. APL code can be accepted in workspaces from I-APL, APL+Win, IBM APL2/PC or Dyalog APL/W, or in documents from Windows Write (use the APL2741 TrueType font, available free from Vector Production), and MS Word (any version).

Except where indicated, items in VECTOR may be freely reprinted with appropriate acknowledgement. Please inform the Editor of your intention to re-use material from VECTOR.

Membership Rates 2003-2004

Category	Fee	Vectors	Passes
UK Private	£20	1	1
Overseas Private	£22	1	1
(Supplement for Airmail, not needed for Europe)	£4		
UK Corporate Membership	£100	5	5
Overseas Corporate	£110	5	
Sustaining	£500	10	5
Non-voting Member (Student, OAP, unemployed)	£10	1	1

The membership year normally runs from 1st May to 30th April. Applications for membership should be made to the Administrator using the form on the inside back page of VECTOR. Passes are required for entry to some association events, and for voting at the Annual General Meeting. Applications for student membership will be accepted on a recommendation from the course supervisor. Overseas membership rates cover VECTOR surface mail, and may be paid in sterling, or by Visa, Mastercard or JCB, at the prevailing exchange rate.

Corporate membership is offered to organisations where APL is in professional use. Corporate members receive 10 copies of VECTOR, and are offered group attendance at association meetings. A contact person must be identified for all communications.

Sustaining membership is offered to companies trading in APL products; this is seen as a method of promoting the growth of APL interest and activity. As well as receiving public acknowledgement for their sponsorship, sustaining members receive bulk copies of VECTOR, and are offered news listings in each issue.

Advertising

Advertisements in VECTOR should be submitted in typeset camera-ready format (A4 or A5) with a 20mm blank border after reduction. Illustrations should be photographs (b/w or colour prints) or line drawings. Rates (excl VAT) are £250 per full page, £125 for half-page or less (there is a £75 surcharge per page if spot colour is required).

Deadlines for bookings and copy are given under the Quick Reference Diary. Advertisements should be booked with, and sent to Gill Smith, Vector Production, Brook House, Gilling East, YORK YO62 4JJ. Tel: 01439-788385.

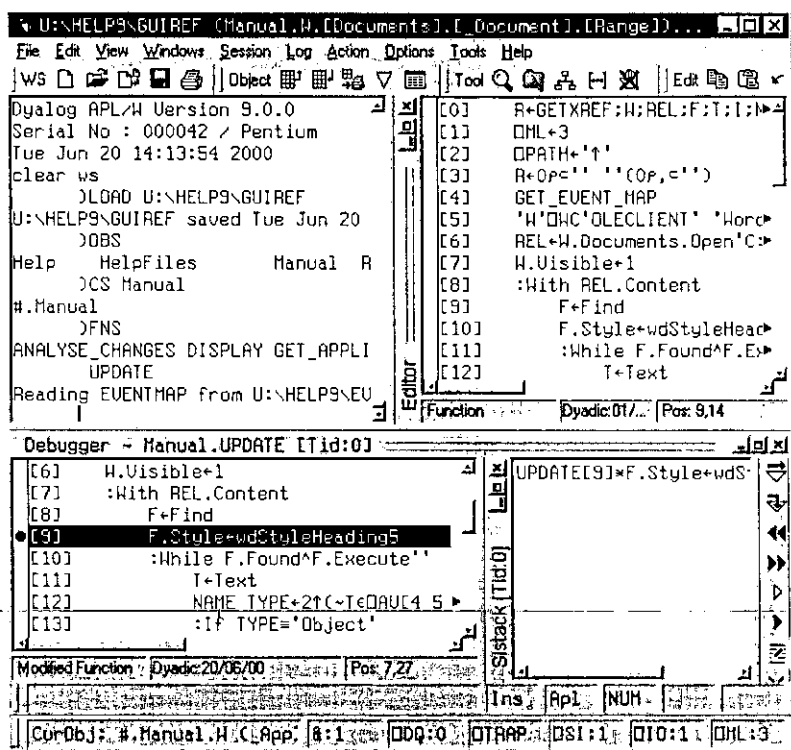
Email: apl385@compuserve.com.

Contents

		Page
Guest Editorial	Sylvia Camacho	3
APL NEWS		
Quick Reference Diary		5
British APL Association News Report and Accounts		7
APL Product Guide	Gill Smith	10
The Education Vector Zark Newsletter Extracts	edited by Jon Sandles	25
Crossword and Puzzle		35
J-ottings 37: How many obtuse-angled triangles are there?	Norman Thomson	38
RECENT MEETINGS		
BAA AGM and Meeting	Adrian Smith	44
APL Germany	Adrian Smith	45
Matrikx Reloaded	Simon Garland	49
Kx Systems Celebrates 10 Years	Steph Johnson	54
APL at FCRC2003 in San Diego	Ian Clark	56
GENERAL ARTICLES		
Why APL Programmers Don't Use Libraries dfns@dyalog.com	Morten Kromberg	62
Hungarian Method Cost Assignment	John Scholes	74
Taming Barrel and Tilt Distortion	Cliff Reiter	87
Graphical Simulation of Rearview Mirrors	Brian Schott	96
TECHNICAL SECTION		
Hackers' Corner: Square Me Up	Adrian Smith	114
At Play with J: Greed	Gene McDonnell	115
Tools, Part 2. Basics too.	Dan Baronet	122
New Foundations	Graeme Robertson	132
Index to Advertisers		143

dyalog APL

The Definitive APL for Windows™



Version 9 - an even better IDE(A)

<http://www.dyadic.com>

Dyalog Systems Limited, Riverside View, Basing Road, Old Basing, Basingstoke,
Hants. RG24 7AL, United Kingdom.

Tel:+44 1256 811125 Fax:+44 1256 811130 Email: sales@dyadic.com

Microsoft is a registered trademark and Windows and the Windows Logo are trademarks of Microsoft Corporation

Should *Vector* Teach Mathematical Grammar?

Guest editorial by Sylvia Camacho

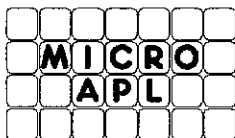
There comes a time in the life of an old married couple when the friends of their youth reach retiring age and throw parties. The most recent we attended followed a predictable course. We fell into conversation with a retired engineer enquiring about our working lives and this elicited the fatal question, "What is so special about APL?". Anthony, as always, fell into the trap and failed, as always, to convey the elegance of the notation for, and the implementation of, Inner Product. Our engineer thought of computers as fast adding machines with logic gates and had always been content to leave the intricacies of programming to his tradesmen in the field. He understood well enough that a specification might be wrong or incomplete but not that some of the fault might lie in the tool itself. We had no common language in which to discuss it further.

One of our daughters has an ambition to speak Spanish. Well, with a name like ours you would, wouldn't you? Added to which she is married to a 'Somerset Spaniard' – born in the UK of Spanish parents and speaking Spanish at home. After several years of evening classes she decided to enroll with The Open University and do the job properly. She has a problem in common with all her fellow students of age less than forty five. "What", she said, "is this thing called 'a participle'?" Her difficulty is that schools stopped teaching Grammar when they abandoned Latin. Grammar is the metalanguage of natural languages. It is unnecessary for the ones you learn at your mother's knee but to learn another in an academic environment grammar is an essential tool.

I believe our difficulty with APL is a consequence of similar ignorance of the metalanguage of mathematics. Our engineer friend is, no doubt, fluent in Calculus and several other of the mathematical dialects that are the tools of his trade but he had never been taught the grammar, the metalanguage of mathematics. Iverson is a mathematical philosopher and has spent a lifetime developing such a grammar – indeed eventually coming to conflate some of it with the grammar of a natural language, calling the result 'J'.

Very few schools teach English grammar, none that I know of teaches mathematical grammar. Until they do our students will have as much difficulty comparing Visual Basic with APL as they have comparing English with Spanish. After all why should they try? Everyone speaks English now, don't they?

APLX *for*



Mac OS 9

Windows 95/98/ME

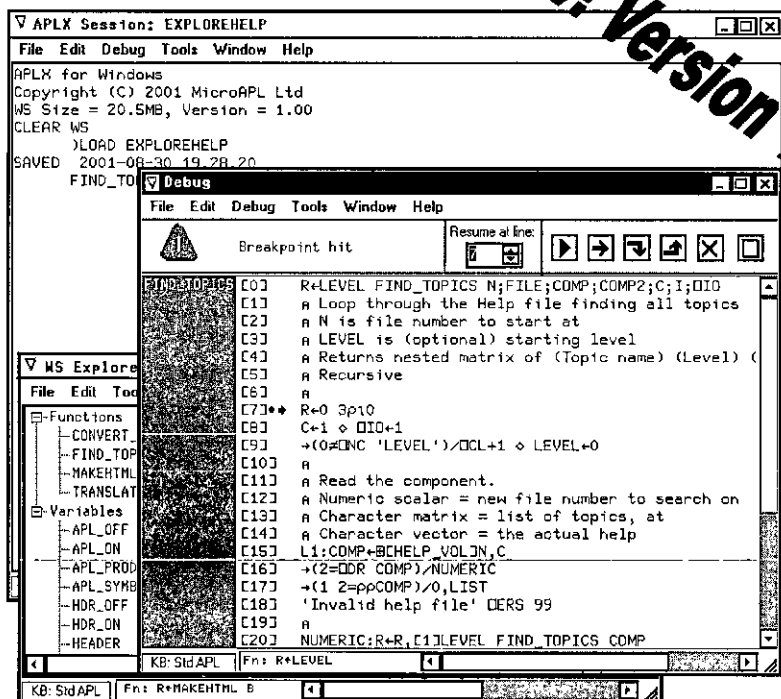
Linux

Mac OS X

Windows NT/2000/XP

AIX

New! Version 1.1



The cross-platform, full-featured, user-friendly,
GUI-enabled, cost-effective APL. *New! Version 1.1*
Check it out at <http://www.microapl.co.uk/apl>

APLX and MicroAPL are trademarks of MicroAPL Ltd. Other trademarks acknowledged.

Quick Reference Diary

Date	Venue	Event
2-5 Nov 2003	USA	APL2000 User's Meeting. Naples Beach Hotel, Florida.

Dates for Future Issues of VECTOR

	Vol.20 No.2	Vol.20 No.3	Vol.20 No.4
Copy date	3rd Sept	4th Dec	5th March
Ad booking	12th Sept	11th Dec	12th March
Ad Copy	19th Sept	18th Dec	19th March
Distribution	October 2003	January 2004	April 2004

Vector Back Numbers

Back numbers of Vector are available from:

**British APL Association,
c/o Gill Smith,
Brook House, Gilling East,
YORK YO62 4JJ**

Price in UK: £10 per complete volume (4 issues);
£12 (overseas); £16 (airmail) including postage.

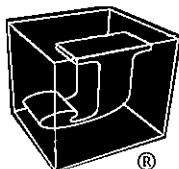
Please note that Vol.1 No.2 is now out of stock.

**Download
from the
web for
free!**

Jsoftware™ Version 5.01

- ♦ **Session Manager, window driver, IDE and GUI support in Unix and Linux**
- ♦ **GUI event support extended to more closely match VB and Java**
- ♦ **New Debug facilities**
- ♦ **New Documentation**
- ♦ **Performance Improvements**

Jsoftware Inc.
P.O. Box 330
Excelsior, MN 55331
Info@jsoftware.com
Tel (952) 470-7345
Fax (952) 470-9202



The programming language
ideal for complex analytics
and data manipulation.

www.jsoftware.com

Minutes of the AGM of the British APL Association held at the Royal Statistical Society on 23rd May 2003

by Anthony Camacho, Hon Sec

Agenda

- 1 Report from the Chairman (Adrian Smith)
- 2 Treasurer's report
- 3 Committee for 2003-2004
- 4 Questions and any other business

The Chairman opened the meeting at 2:05 when there were about 25 members present. Previous AGM minutes were taken as read and various apologies were noted.

Chairman

The Chairman reported that the Association has not done much except keep Vector going. The April Vector is at the printers and will shortly be posted to members. Eighty copies will be sent to San Diego to be given to delegates. The Association is considering setting up a wiki site on the web. This allows any member to edit it and could become an accumulating resource of APL material.

The Chairman felt we were not doing as much as we ought to promote APL or serve our members. We have enough reserves to finance some projects and he appealed to members for suggestions of projects to support.

Treasurer

The Treasurer, Nicholas Small, circulated the accounts (reproduced below) and offered to answer questions: there was none. John Sullivan, having indicated his willingness to continue, was re-appointed as auditor.

Committee

The Chairman announced the committee for 2003-2004:

Chairman	Adrian Smith (co-opted)
Secretary	Anthony Camacho (co-opted)
Treasurer	Nicholas Small
Editor	Stefano Lanzavecchia (co-opted)
Webmaster	Bob Hoekstra
APL in Schools	Stephen Taylor
Activities	Jon Sandles (co-opted)
Projects	Alan Mayer (co-opted)

Questions and AOB

In reply to a question, the Chairman said we distribute a little over 500 copies of each issue of Vector, plus additions given away at conferences. Of the individual members there are about 112 in the UK and about 222 overseas. There was no other business.

The meeting closed at 2:20 pm.

Membership at 30.4.03 (previous year's figures in parentheses)

	UK		FOREIGN		TOTAL	
	Number	Vectors	Number	Vectors	Number	Vectors
Sustaining	6 (6)	28 (28)	2 (3)	16 (41)	8 (9)	44 (69)
Corporate*	4 (4)	16 (29)	1 (1)	10 (10)	5 (5)	26 (39)
Corp. Ind*	9 (14)	9 (14)	16 (2)	17 (2)	25 (16)	26 (16)
Individual	112 (117)	111 (117)	222 (238)	221 (237)	334 (355)	332 (354)
Non-voting	19 (16)	19 (16)	0 (0)	0 (0)	19 (16)	19 (16)
Life	1 (1)	1 (1)	1 (1)	1 (1)	2 (2)	2 (2)
Library	1 (1)	1 (1)	6 (6)	6 (6)	7 (7)	7 (7)
Russians			11 (11)	11 (11)	11 (11)	11 (11)
APL Groups			13 (13)	39 (39)	13 (13)	39 (39)

506 (553)

*Add the Vector numbers in these rows to get the total subscribed for by corporate members.

Provisional Summary of Annual Accounts 2002/03

Summary of income and expenditure/receipts and payments:

	2002/03 (R&P) £	2002/03 (I&E) £	2001/02 (I&E) £
Income/Receipts			
Subscriptions	12685	12487	9884
BCS services	0	0	0
Bank interest	1455	1455	1703
Vector advertising (incl. VAT)	3182	3432	4056
Other	204	214	629
	-----	-----	-----
Total receipts	17527	17589	16272
	-----	-----	-----
Expenditure/Payments			
Meetings	190	190	240
Administration	1213	1213	1348
BCS services	76	76	0
Vector production and despatch	16700	16640	15352
Projects	743	743	2369
Other	417	384	308
	-----	-----	-----
Total payments	19339	19246	19617
	-----	-----	-----
Assets summary:			
Bank and other balances		35537	37349
Debtors		1361	1121
Creditors		(5602)	(5516)
		-----	-----
Net assets		31296	32954
		-----	-----

Notes.

Pence figures have been omitted, so columns may not add exactly. The value of stocks of Vector have not been assessed, nor has the value of the Association's computing hardware and software.

For 2002/03, figures are shown both as income and expenditure, i.e. revenues strictly relating to the activities of that year, and as receipts and payments, i.e. what goes in and out of our bank account. The comparative figures for 2001/02 relate to income and expenditure.

The Vector Product Guide

compiled by Gill Smith

VECTOR's exclusive Product Guide aims to provide readers with useful information about sources of APL hardware, software and services. We welcome any comments readers may have on its usefulness and any suggestions for improvements.

We reserve the right to edit material supplied for reasons of space or to ensure a fair market coverage. The listings are not restricted to UK companies and international suppliers are welcome to take advantage of these pages.

For convenience to readers, the product list has been divided into the following groups ('poa' indicates 'price on application'):

- Complete Systems (Hardware & Software)
- APL and J Interpreters
- APL-based Packages
- Consultancy
- Other Products
- Overseas Associations
- Vendor Addresses
- World Wide Web and FTP Sites

Every effort has been made to avoid errors in these listings but no responsibility can be taken by the working group for mistakes or omissions.

We also welcome information on APL clubs and groups throughout the world.

Your listing here is absolutely free, will be updated on request, and is also carried on the Vector web site, with a hotlink to your own site. It is the most complete and most used APL address book in the world.

Please help us keep it up to date!

All contributions and updates to the Vector Product Guide should be sent to: Gill Smith, Brook House, Gilling East, York, YO62 4JJ. Tel: 01439-788385, Email: apl385@compuserve.com

APL INTERPRETERS

COMPANY	PRODUCT	PRICES(£)	DETAILS
APL Borealis Inc.	Dyalog APL	poa	Distributor of Dyalog APL products from Dyadic
	APL2000	poa	Distributor of APL2000 products
APL Systems IDC SL	APL2000 APL interpreters and toolspoa		Distributor of APL2000 products for the UK Consulting and maintenance for APL applications
Beautiful Systems	Dyalog APL/W for Windows	poa	US Distributor of Dyalog APL products from Dyadic.
	Dyalog APL for Unix	poa	See Dyadic listing for product details.
Dinosoft Oy	Dyalog APL/W for Windows	poa	Finnish distributor of Dyalog APL products.
	Dyalog APL for Unix	poa	See Dyadic's listing for product details.
Dltrich & Partner	APL+Win	poa	Cognos/APL2000 Inc products
	Dyalog APL	poa	Dyadic Systems Ltd. products
	IBM APL products	poa	
Dyalog	Dyalog APL for DOS/386	995	Second generation APL for DOS. Runs in 32-bit mode, supports very large workspaces. Unique "window-based" APL Development Environment and Screen Manager. Requires 386/486 based PC or PS/2, at least 2Mb RAM, EGA or VGA, DOS 3.3 or later.
	Dyalog APL/W for Windows	995	As above, plus object-based GUI development tools. Requires Windows 3.0 or later.
	Dyalog APL for Unix	995-12,000	Second generation APL for Unix systems. Available for Altos, Apollo, Bull, Dec, HP, IBM 6150, IBM RS/6000, Masscomp, Pyramid, NCR, Sun and Unisys machines, and for PCs and PC/2s running Xenix or AIX. Oracle interface available for IBM, Sun and Xenix versions.
DynArray	DICE for Windows	poa	Software development kit which includes an APL interpreter as a DLL and the ability to run and link existing and new APL code to non APL code such as VB, C/C++, Java and integration with various Windows software applications and database packages such as MS Office.
I-APL Ltd	I-APL/PC or clones	8	ISO conforming Interpreter. Supplied only with manual (see 'Other Products' for accompanying books).
	I-APL/BBC Master	8	As above
	I-APL/Archimedes	8	As above
IBM APL Products	TryAPL2	free	APL2 for educational or demonstration use. Write, fax or Email to APL Products; specify disk size desired.
	Workstation APL2 V2 Version 2	\$1500	AIX, Linux, Solaris, Windows Product 5724-B74, Part Number 45P7514
	APL2 Version 2	poa	Product No. 5688-228. Full APL2 system for S/370 and S/390
	APL2 Application Envt Vn2	poa	Product No. 5688-229. Runtime environment for APL2 packages
Insight Systems	Cognos/APL2000 Inc	poa	Leading distributor of APL2000 products in Denmark
	Dyalog Ltd.	poa	Leading distributor of Dyalog APL products in Denmark
	IBM	poa	Leading distributor of IBM APL & GraphX products in Denmark
J Austria	J	poa	Distributor for Austria and Switzerland
	Dyalog APL	poa	Distributor
	Causeway Products	poa	Distributor
	Structural Analysis Software	poa	Complete package by IG Zenkner&Handel to perform structural analysis/engineering calculations. Also suitable for dynamic problems, e.g. earthquake simulation.
JSoftware Inc.	J on the Web online registration ...		
	J Professional (online reg.)	\$895	includes manual set and one year of updates
	J Standard (online reg.)	Free	Free for download only

Books and accessories			
	J Dictionary	\$50	
	J Phrases	\$50	
	J Primer	\$50	
	Fractals, Visualization and J	\$80	
	Concrete Math	\$40	
	Exploring Math	\$50	
Lescasse Consulting	APL+PC	poa	Lescasse Consulting is the exclusive APL2000 distributor in France and also distributes in Switzerland and Belgium. Call for price quotes.
	APL+Unix	poa	
	APL+DOS	poa	
	APL+Win	poa	
	Dyalog APL/W	poa	French distributor for Dyalog
MasterWork Software	Manugistics Products and ISI	poa	New Zealand distributor
MicroAPL	APLX for Windows/MacOS	499	Cross-platform APL development environment with GUI programming facilities. Interpreter modelled on APL2. Available for Windows 95/98/ME/NT/2000/XP, Mac OS 9 and Mac OS X.
	APLX Server Edition	poa	For running large multi-user APL applications on x86 Linux, RS/6000 AIX, and Windows NT/2000.
	APLX for Linux (commercial)	499	Linux desktop edition of APLX, compatible with Windows and MacOS versions, with full development environment and GUI programming facilities. Runs under RedHat, Debian, Mandrake and SuSe Linux distributions.
	APLX for Linux (personal)	Free	Full version of APLX, can be downloaded free for personal use.
Oasis	Dyalog APL	poa	Dyalog Ltd
	APL*PLUS	poa	Manugistics
	APL.68000	poa	MicroAPL Ltd
	APL2	poa	IBM
Omega	Zero	poa	A "small simple and fast" alternative to APL
Optima	Dyalog APL/W	995	Dyalog's best, leading edge APL interpreter
RE Time Tracker Oy	APL+PC (APL*PLUS/PC)	poa	Complete APL+ and Statgraphics product range and links to various 3rd party products.
	APL+DOS (APL*PLUS II)		
	APL+Win (APL*PLUS III), APL+Link		
	APL+UNIX		
	APL*PLUS Sharefile		
Soliton Associates	SHARP APL for OS/390	poa	for IBM OS/390 mainframes
	SHARP APL for UNIX	poa	for SunOS and IBM AIX
	SHARP APL for Linux	poa	for Intel Linux
Strand Software	Canada		
	All APL*PLUS products	poa	All APL*PLUS products including upgrades and educational.
	Dyalog and JSoftware products	poa	
	USA		
	Dyalog and JSoftware products	poa	

APL PACKAGES

COMPANY	PRODUCT	PRICES(£)	DETAILS
ADAPTA Software	MPS	poa	Master Production Scheduling
	FBS	poa	Forecasting and Budgeting System
	DRP	poa	Distribution Requirements Planning
Adaptable Systems	FLAIR	poa	Finite loader and interactive rescheduler. Customisable full-function scheduling system. (Available outside Australia by special arrangement only.)
Adaytum	Adaytum e.Planning	poa	Adaytum e.Planning offers a Web-based solution that combines planning, forecasting, budgeting, modelling and reporting in a single, integrated application.
APL Software Services	APL Utilities	poa	Software: mostly .AWS for DOS, utilities for most APL interpreters. Public domain APL*Plus v10 with on-screen documentation and interactive tutorials. APL Conference Software. Books: APL user manuals for STSC, IBM, and Sharp. Request email catalog from dick.holt@juno.com.
Beautiful Systems	ASF_FILE	\$399	Dyalog APL/W auxiliary processor for access to APL*PLUS/PC APL component files (*.ASF).
	NAT_FILE	\$299	Dyalog APL/W auxiliary processor which emulates the APL*PLUS/PC quad-N native file subsystem for access to the DOS file system.
	DBF_FILE	\$299	Dyalog APL/W auxiliary processor for efficient block mode access to dBASE format files. Designed to get large amounts of data in and out of dBASE. Not suited for random access to small amounts of data (it does not handle keys).
	SF_READ	poa	Dyalog APL/W functions to read APL*PLUS data objects of any type or structure from *.SF style component files created by APL*PLUS II or III.
Causeway	CausewayPro for Dyalog/W	400	Causeway application development platform for Dyalog APL/W.
	RainPro Business Graphics	250	The ultimate graphics toolkit for the APL developer. Adds 3D charting capability, Web publishing and clipboard support to the shareware product. Charts can be included in <i>NewLeaf</i> reports. Functionally compatible across Dyalog/W and APL+Win.
	NewLeaf for Dyalog and +Win	400	Frame-based reporting tool with comprehensive table-generation and text-flow support. Offers multiple master-page capability, bitmap wrap-around and on-screen preview with pan and zoom. Fully supported on Dyalog/W and APL+Win
Cinerea AB	ORCHART	250	Organization chart package for IBM APL2/PC. Full & heavily commented source code included - free integration into other applications. NB: ASCII output with line-drawing (semi-graphic) characters for boxes.
CODEWORK	3-way TANGRAM	poa	3-way TANGRAM is a DSS-OLAP product, basically a powerful and versatile handler of multi-way tables (also known as hypercubes). It entails 46 analysis modules, including computations, cube merging, sensitivity analysis, time intelligence, free format queries, HTML and LaTeX outputs. Current version is 7.0. At the time of writing, the product is available on Dyalog APL 8.3.1 for Windows 95/NT. Future plans include an APL+Win version and later a LINUX version.

CoSy	K.CoSy	\$30 % yr sub	K.CoSy is a general purpose computing and programming environment constructed, all in open code, in Arthur Whitney's very high level, yet structurally transparent Array Programming Language, K, and its tightly coupled User Interface. K.CoSy is an extremely productive environment in one of the most powerful and fastest of APL's progeny, and therefore, likewise, of all languages. K.CoSy provides a workspace-like interactive development environment previously impossible in K. Because of its unique open construction within the language itself, this environment is clearly competitive in a large domain with the APLs from the other vendors. K.CoSy notepad nature, interactivity, and open K code vocabulary make learning Arthur Whitney's K far less daunting and far more productive than its raw console, or any external scripting method. If you are a client of Kx Systems, or are investigating the possibilities, Contact us. See CoSy/K/CoSy for more information.
DynArray	DynaWeb Server	poa	A web server providing web based access to applications running on the DICE interpreter from DynArray, or on an IBM mainframe running APL2.
	DynaHarry	poa	A DSS system which offers the next generation capabilities for current APLDI, IC/E and IC/1 users. It comes with ROLAP capabilities, multisystem access to a wide variety of databases and data warehouses.
	DynaLink	poa	An ODBC client interface for DICE and IBM APL2 programs.
Entropy Software	Personal Information Management	poa	Entropy specialises in Personal Information Management Systems (PIMS) which run on a PC. Most systems are inadequate in some way. This is an attempt to fulfil all the needs of George MacLeod who has been an unhappy user of a variety of systems for many years. The product is being developed in modules (Calendar, Address List, To Do List, etc) which will be available at no charge. When all the modules are complete they will be integrated together. There will be a charge for the integrated product.
HMW	4XTRA	poa	Networked, Windows/Unix based Front End and Middle Office Foreign Exchange and Money Market Dealing System. Scalable from 1 user to 120+.
	Inca	poa	Software Change Management System. Enables the user to co-ordinate development work from several sources, resolve clashes, promote work items for testing and configure releases to a live environment.
	Maya	poa	APL code file manager. A comprehensive suite of tools giving a multi-window IDE style interface to file based APL code. Offers features such as: copying from file to file, object comparison, string search, style formatting, hot-spot editor for filed objects (including variables), etc.
	Aztec	poa	System shell for APL development. Manages real-time and batch applications across multiple platforms. Offers standardised error trapping, job scheduling, task communication and recovery/restart features.
I-APL Ltd	Olmec	poa	APL GUI environment, providing menu bar, tool bar, status area, navigation sidebar (with treeviews & listviews) and client area. All are configured by simple text files and require no programming. Client area has a "tab wizard" option to provide ordered transaction processing
	Nazca	poa	fast, flexible and reliable static database and editor.
	Educational workspaces	5	PC format disks with the examples from: Thomson, Espinasse (Kits 1-4), Kromberg, Jizba & FinnAPL. All the examples to save your fingers!
IBM APL Products	A Graphical Statistical System	\$250	for DOS, Product Number 6764-009
Insight Systems	Causeway	poa	Leading distributor of Causeway products in Denmark

All our old products are now either OEM'd, in the public domain, out of date, or all of the above. We'll be back!

JAD Software	JAD SMS	poa	JAD SMS is a multi-user software management system for Dyalog APL™ based on shared, hierarchical databases. JAD SMS databases let you keep historical versions of apl items as well as attributes such as timestamp, user name and documentation. The software includes a graphical user interface as well as specialized functions for inclusion in applications. No charge for single-user version; \$100/user for multiple users
Lescasse Consulting	APL+Win Monthly Training	\$600	Download 50+ page document about APL+ programming each month. You also get one or more workspaces full of re-usable APL code and sometimes additional files or products.
	Advanced Windows Programming	\$95	200-page book plus companion disk on interfacing APL and Delphi. Contains full coverage of Delphi-2, +Win and Dyalog.
	DLL parser for APL	\$250	Parse any Visual Basic DLL declaration file into a set of quadNA definitions. Turn constants and structures into APL variables. Available for APL+Win and Dyalog/W.
	Delphi Forms Translator	\$195	Design forms with Delphi and turn them automatically into APL programs which recreate the same form (+Win and Dyalog/W).
	APL+Link Pro	poa	ODBC interface for APL+Win
	SQAPL Pro	poa	ODBC interface for Dyalog APL/W
	RainPro	poa	Highly customisable 2D and 3D publication graphics for APL+Win and Dyalog APL/W
	NewLeaf	poa	Page layout and printing tools for APL+Win and Dyalog
	GraphX and ChartFX	poa	High-quality business graphics for APL+Win
	Formula One and Dyalog APL	\$95	100-page book + companion disk on how to use the Formula One VBX with Dyalog APL/W
Lingo Allegro	FACS	poa	EMMA-like interface to DB2 or ODBC databases
	QWIN	poa	Legacy DOS Windowing support for APL+Win
	ODBC/127	poa	IBM AP127-like ODBC Interface for APL+Win and Dyalog APL/W
Optima	ServiceLine	poa	A property management system which keeps a record of all outstanding tasks, produces an up to date list of work to-do and Scheduled reminders plus automated standard letters and basic financial control.
	TravelLine	poa	A system designed to control the workload allocation for a fleet of chauffeur vehicles plus a reminder system for fleet management and Local Authority requirements.
	BPA	poa	"Brand Performance Analysis" allows for the modelling of product/brand performance over time and comparison with competitor products.
	DBI	poa	"Database Interrogator" allows for the non technical user to ask sensible questions of a large database such as a questionnaire and obtain results tables and graphs quickly, easily and accurately.
Qualedi	Qualedi	\$850-\$5,500	Electronic Data Interchange (EDI) translation software for the PC, with strict compliance checking.
RE Time Tracker Oy	UIT/W	poa	Comprehensive high-level Windows User Interface library for APL+Win and +II v 5.1. Comprehensive spreadsheets, replicated fields, special field types, etc. 16 and 32 bit versions available.
	AJGRAPH	poa	Graphpak-compatible 2D graphics package for +Win and +DOS. Includes multi-window support, print and metafile support. No DLLs required.
	ECCO PRO with APL	poa	Leading group and personal information management system with comprehensive customising. Supplied with sample +Win workspace to interface to ECCO databases via DDE.
	NEWTT TCP/IP SDK with APL	poa	Lead TCP/IP SDK with interfaces to all protocols. Supplied on 3 CD ROMS together with a sample +Win workspace.

	DB+	poa	Database interface for APL+DOS under Windows. Allows combining character-based APL applications with ODBC-compliant databases such as Oracle and SQL-server.
Warwick University	BATS	250	Menu driven system for time series analysis and forecasting using Bayesian Dynamic modelling. Price is reduced to £35 for academic institutions.
	FAB	free	Training program for the above.
Weighahead Systems	Weighahead Windows Weighing System (3WS)	poa	Recipe Weighing System for Manufacturing Industries. Pharmaceutical, Cosmetics, Foods etc. Works without keyboard or mouse. Uses Electronic Balances, Laser scanners, bar codes and label printers.
Zark	APL Tutor (PC)	\$299	APL computer-based training. Available for APL*PLUS PC & APL*PLUS II. Demo disk \$10.
	APL Tutor (MF)	\$5000	Mainframe version.
	Zark ACE	\$99	APL continuing education. APL tutor news and hotline phone support.
	APL Advanced Techniques....	\$59.95	488pp. book, (ISBN 0-9619067-07) including 2-disk set of utility functions (APL*PLUS PC format).
	Communications	\$200 pc, \$500 mf	Move workspaces or files between APL environments.

APL CONSULTANCY AND DEVELOPMENT

COMPANY	PRODUCT	PRICES(£)	DETAILS
Adfee	Consultancy	poa	Development, maintenance, conversion, migration, documentation, of APL products in all APL environments
Andrews	Consultancy	poa	APL programming and analysis, algorithms, tree processing and design programs for craft work.
APL Borealis Inc.	Support and Development	poa	APL Software Support and Development. Specialists since 1979 in Sharp APL, APL*Plus, APL+Win, Dyalog APL
APL Solutions Inc	Consultancy	poa	APL systems design, development, maintenance, documentation, testing and training. Providing APL solutions since 1969.
APL Systems IDC SL	Consultancy	poa	Consulting and maintenance for APL applications.
AUSCAN Software	Consultancy and Training	poa	APL software development, training
Camacho	Consultancy	poa	Manuals; feasibility reports and estimates; analysis and programming; APL and MS Windows applications; Sharp, ISI APL, APL*PLUS, APL2/PC and other APLs spoken. Fixed price systems a speciality
Ian Clark	Consultancy	poa	Interfacing APL, VB, C/C++, ActiveX/COM. Screen design and documentation. National language porting.
Ray Cannon	Consultancy	poa	APL; C; Assembler; Windows; Graphics; PC and mainframe
Causeway	Consultancy and Training	poa	On-site training for Causeway, RainPro and NewLeaf. Customisation and enhancement to meet local needs. Code review and pre-implementation check of Causeway applications.
Paul Chapman	Consultancy	250-500	24-hour programmer: APL, Smalltalk, C; Windows front end design a speciality.
CODEWORK	Consultancy	poa	Development, maintenance, migration, documentation of APL applications. Speciality: info systems for top executives, internet applications.
CoSy	Consultancy	poa	CoSy.com, Coherent Systems, provides rapid development in the K language and associated data base products, with a particular interest in quantitative (financial) modelling.
David Crossley	Consultancy	poa	Experienced in large APL system developments since 1969 for PC or mainframe.
Dinosoft Oy	Consultancy	poa	Specialised in very large databases.

Dittrich & Partner	Consultancy	poa	APL programming and analysis; APL workshops and training on the job
Dyalog	Consultancy	poa	APL and Unix system design, consultancy, programming and training.
DynArray	Consultancy	poa	DynArray offers consulting in the areas of DSS, Y2K and APL programs upgrade/conversion to modern Web enabled platforms.
Evestic AB	Consultancy	poa	Excellent track record from 15+ years of APL applications in banking, insurance, and education services. All dialects, platforms and project phases. SQL expertise.
First Derivative Analytics Ltd.	Consultancy	poa	Analysis, design, prototyping, development & testing of APL (especially financial) applications: Sharp, Dyalog APLW.
General Software	Consultancy	from 200	Over 20 years experience with every version of APL, large mainframe systems and small PC based programmes.
Godin London Inc	Software Development	poa	We have applications in the food manufacturing field, travel agency and airline bookings field and in product lease management.
HMW	Consultancy	poa	System design consultancy, programming. HMW specialize in banking and prototyping work.
Hoekstra Systems	Consultancy	poa	APL consultancy, programming, etc. Also UNIX system administration
Michael Hughes	Consultancy	poa	APL consultant with 20 years experience with all versions of APL. I can create your dynamic Web sites using the full power of APL working with Microsoft IIS (Internet Information Service) on Windows NT or 2000. I also undertake System design, Programming and Maintenance on all platforms, particularly MS Windows.
INFOSTROY	Consultancy	poa, competitive	Broad experience in various APL platforms. Special skills and knowledge in developing complex applications for investment, financial and construction markets. Implementation of hybrid solutions based on APL, Delphi, C++, VBA, SQL servers.
Insight Systems	Consultancy	poa	We have experience with just about every APL system and platform in common use during the last 20 years, from SHARP APL under MVS or Linux to APL+Win and in particular Dyalog APL under Windows 9x, NT or 2000. If you have decisions to take about adapting your APL application to take advantage of emerging technologies, or would like your strategy reviewed, give us a call. We have extensive experience in all areas of APL development, from legacy systems, up, down and sideways migrations, to the development and support of shrink wrapped solutions based on APL. Even if we don't have time to do the work ourselves, we will know where to find someone who is an expert in your version of APL and your application area, on your continent.
JAD Software	Consultancy	poa	Systems design and development, project management, technical manuals, financial and actuarial expertise in APL.
KJK	Consultancy and software development	poa	APL-based data management: conversions, ad hoc-analyzing tools, well-interfaced methods for defining, processing and browsing of multi-dimensional reports. Rapid custom software development based on proven modular toolset approach.
Lambent Technology	Consultancy	poa	APL programming, consulting & training; web design and construction.
Phil Last	Consultancy	poa	APL consultancy, modelling and programming.
Lescasse Consulting	Consultancy	poa	A range of consultants, experts in Windows programming, with APL+Win and Dyalog APLW. More than 100 major APL applications already developed. We all have additional expertise in Formula One and Delphi.
Lingo Allegro	Consultancy	poa	General APL consulting, internet website development, migration and downsizing, performance tuning, education and training.
Lucas Solutions	Consultancy	poa	Rates depend on task and location.

Mackay Kinloch Ltd	Consultancy	poa	Design, analysis and programming for banking, insurance and pensions, financial planning and modelling, corporate performance and legal reporting
MasterWork Software	Consultancy	poa	Consulting and J programming for econometrics and statistics in public policy, health and food industries.
Milinta Inc	Consultancy	poa	Design, development, maintenance, conversion, documentation in all APLs, most APs and some specific Sharp products (LOGOS, ViewPoint, Retrieve). Experience in multi-user, multi-task systems, databases, Windows programming.
Ellis Morgan	Consultancy	250-500	Business Forecasting & APL Systems.
Oasis	Consultancy	poa	Expertise in APL system design, Project management, conversion, migration, tuning; for all APL versions (10+ years experience)
Omega Computing	Consultancy	poa	APL consultancy, programming, etc.
Optima	Consultancy, Training & Development	poa	We have been in business since 1990 and have a team of APL professionals with many years experience in pharmaceutical, industrial and financial systems on both PC and Mainframe platforms. In addition to pure APL we can also offer assistance with network integration, ACT1 customisation and issues concerning the internet and/or web design.
RadSys Technologies	Consultancy	poa	Areas of expertise: financial systems, risk analysis systems, healthcare systems.
Resources & Results	Consultancy	poa	Knowledge management company builds decision support, data warehouse, data mining and strategic planning systems for CFO's, CEO's, and senior management, using our Rapid Application Development (RAD) methods and tools. Extensive experience in large-scale system development and ad hoc executive support for Fortune 500 clients.
RE Time Tracker Oy	Consultancy	poa	APL application conversions, APL Windows interfaces, APL to API-level interfacing to any system under Windows, TCP/IP network and database connectivity.
Rex Swain	Consultancy	poa	Independent consultant, 25 years experience. Custom software development, PC and/or mainframe.
Graeme Robertson	Consultancy & Training	250-500	Mathematical, statistical and financial APL runtime systems designed, written from scratch and documented. Legacy systems upgraded. 20 years experience covering most APLs (esp. Dyalog) on most platforms (esp. Windows). Particularly interested in dynamic-web or Mathematica applications.
Rochester Group	Consultancy	poa	Specialise in MIS using Sharp APL
Shepp & Associates	Consultancy	poa	APL applications development and consulting, especially in the travel industry, especially on small computers. 25 years experience in APL programming.
Snake Island Research Inc	Consultancy	poa	APL interpreter and compiler enhancements, intrinsic functions, performance consulting. APL parallel compiler APEX is giving very good initial performance tests with convolution somewhat faster than FORTRAN.
SovAPL	Consultancy	poa	Offshore APL development service.
Strand Software	Consultancy	poa	Advice on migrating to and from all flavours of APL and hardware platforms. Full-screen Interface implementation, APL utilities, benchmarking, efficiency analysis, actuarial software, system development tools, valuation, pricing and modelling systems.
Sykes Systems Inc	Consultancy	poa	Complete APL services specialising in audit, optimisation and conversion of APL systems. Excellent design skills. All dialects and platforms. 17-23 years experience.
Welghead Systems	Consultancy	poa	Specialising in industrial systems. Links to PLCs, laser scanners, bar codes, weigh scales, label printers etc. Also programmable hand held scanners.
Stephen Wynn	Consultancy	poa	Most experience of financial planning, and mathematical areas: operational research, quality control, experimental design.

OTHER PRODUCTS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Adfee	Employment	poa	Contractors and permanent employees
APL-385	Typefaces	poa	Variants of the APL2741 typeface available to specification.
APL Borealis Inc.	APL Training	poa	Hands-on courses in Introductory, Intermediate, Advanced and Windows APL. Courses are customized and flexible, and may be delivered on-site, with strong emphasis on methods for efficient and maintainable APL systems development.
Atomic Vector	RELCELL	poa	Details: Report Writer, Data Mining, Database Visualization, and Export to Excel from relational databases. Offered in English, Spanish, Italian, Portuguese, French and German. With this product, you don't have to know English in order to use SQL. Point and click with easy to use Wizards.
	APL Games Volume 1	\$15	Details: Volume 1 - A collection of four APL games on cd, saved from past yearly issues of the APL Planet. Great way to demonstrate the capabilities of APL, and to entice the younger generations into APL. Written entirely in Dyalog APL.
ComLog	Comic-Logger	\$25.95+p&p	APL*PLUS II comic-book inventory system. Shareware version available on America OnLine.
HMW	Employment	poa	Contractors and permanent employees placed.
I-APL Ltd	Books	poa	I-APL stocks books written to go with the I-APL interpreter and some APL Press books. For a list write to 11 Auburn Road, Bristol BS6 6LS, ring 0117 973 0036 or email 100612.1057@compuserve.com.
Oasis	Training	poa	Introductory courses in APL Advanced courses for different APL versions
Optima	IBM Compatible	poa	Complete business solutions including hardware, software and support plus additional network installation and maintenance if required.
Renaissance Data Systems	Booksellers		The widest range of APL books available anywhere. See Vector advertisements.
Right Seat Software	Vox Proxy	\$199 (comm)	Vox Proxy is authorware for PowerPoint(r) 2000 or 2002 which allows the use of Microsoft Agent Technology (3D talking animated characters) within slide shows. VP appears on PowerPoint's main menu and provides editing side-by-side with slides. Automated script-writing provides control of PowerPoint, allowing the use of characters for live presentations or fully-automated tutorials, demos, or training programs. Optional CD Prep program allows the user to create auto-starting CD's that will play on any version of PowerPoint or without PowerPoint.
		\$69.95 (edu)	

OVERSEAS ASSOCIATIONS

GROUP	LOCATION	JOURNAL	OTHER SERVICES	Ann.Sub.
ACM SigAPL	International	APL QuoteQuad	Conferences; APL white pages; web site	\$30
APL Bay Area	USA N. California	APLBUG	Monthly Meetings (2nd Monday)	\$20
APL Club Austria	Austria	-	Quarterly Meetings	200AS(indiv), 1000AS(corp)
APL Germany e.V.	Germany	APL Journal	Semi-annual meetings	DM60
Ass. Francophone pour la promotion d'APL	France	Les Nouvelles d'APL		FF350 (private) FF2800 (Company)
BACUS	Belgium	APL-CAM	Conferences & Seminars	£18 (\$30)
Danish SIG	Denmark			
Dutch APL Assoc.	Holland	Vector provided	Mini-congress, APL ShareWare initiative	
FinnAPL	Helsinki, Finland	FinnAPL Newsletter	Seminars on APL	100FIM(private), 30(student), 1000 (Co)
Japan APL Assoc	Tokyo	APL Journal	Monthly meetings (4th Sat)	10,000yen to join

NY SigAPL	New York, USA	Big Apple APL	Monthly meetings	\$35/\$25(ACM)
Rome/Italy SIG	Roma, Italy			
SE APL Users Grp	Atlanta, Georgia	The APL Planet	Semi Annual meetings	\$13 (US), \$25 (non-US)
SovAPL	Moscow, Russia	-	Seminars and Annual Meeting	
SwedAPL	Sweden	SwedAPL Nytt	Semi-annual meetings, seminars	SEK 75
SWAPL	Texas, USA	SWAPL		\$18
Swiss APL (SAUG) Bern		Part of Qily St-Info		SF60 (SI) + SF20 (SAUG)
Toronto SIG	Toronto, Canada		Occasional meetings, APL Skills Database, Toronto Toolkit	

ADDRESSES

ORGANISATION	CONTACT	ADDRESS, TELEPHONE, FAX, EMAIL etc.
ACM SigAPL	David Siegel	ACM, 1515 Broadway, 17th Floor, New York, NY 10036, USA (Subs only)
ADAPTA Software GmbH	Michael Baas	Stellinger Weg 19, 20255 Hamburg, Germany. Tel: +49 40 40170951 Fax: +49 40 40170952. Email: info@adapta.de
Adaptable Systems	Lois & Richard Hill	49 First Street, Black Rock 3193, Australia. Tel: +61 3 9589 5578 Fax: +61 3 9589 3220 Email: hillrj@melbpc.org.au
Adfee	Bernard Smoor	Dorpsstraat 50, 4128 BZ Lexmond, Netherlands. Tel: +31 347 342 337 Fax: +31 347 342 342 Email: adfee@concepts.nl
Andrews	Dr Anne D Wilson	12 Thorny Hills, Kendal, Cumbria LA9 7AL, UK. Tel: 01539-731205 Email: ADWilson@kencomp.net
APL-385	Adrian Smith	Brook House, Gilling East, York YO62 4JJ, UK. Tel: 01439-788385 Email: apl385@compuserve.com
APL2000 (Europe)	Fred Honea	see APL Systems IDC SL
APL Bay Area APLBUG	Curtis Jones (Sec)	228 South 15th Street, San Jose, CA 95112-2150, USA Tel: +1 (408) 292-4060 Email: jonesca@vnet.ibm.com
APL Borealis Inc.	Richard Procter	381 Manor Road East, Toronto, Ontario M4S 1S7, Canada. Tel: (416) 457-7828. Fax: (416) 482-6582 Email: info@aplborealis.com
APL Club Austria	Harald F. Nelson	c/o N-TECH, Siebenbrunnfeldg. 4-6, A-1050 Wien, Austria. Tel: +43 1 5458063 Fax: +43 1 5458063-17
APL Germany e.V.	Dieter Lattermann	Rheinstraße 23, D-69190 Walldorf, Germany. Tel: +49 6227-63469 Compuserve: 100332,1461
APL Software\Services	Dick Holt	3802 N Richmond St, Suite 271, Arlington, VA 22207 USA Tel: +1 (703) 528-7624; Fax: +1 (703) 528-7617; Email: dick.holt@juno.org
APL Solutions Inc	Eric Landau	1107 Dale Drive, Silver Spring, MD 20910-1607 USA Tel: +1 (301) 589-4621 Fax: +1 (301) 589-4618 Email: aplsi@starpower.net
APL Systems IDC SL	Fred Honea	Alfredo Marquerie, 12 - 2 F, 28034 Madrid, Spain. Tel: +34 91 730 7006 (Office) +34 60 680 5949 (Mobile). Fax: +1 775 743 6131. Email: uksales@apl2000.net
Association Francophone pour la promotion d'APL	Ludmila Lemagnen	174 Boulevard de Charonne, F-75020 Paris, FRANCE Email: lemagnen@aol.com
AtomicVector Productions Inc.	John Manges	413 Comanche Trail, Lawrenceville, GA 30044, USA. Tel: +1 (770) 972-3755 Email: E-mail: mailto:seapldoc@aol.com
AUSCAN Software Ltd	Richard Procter	PO Box 39, Mansfield, Ontario L0N 1M0 Canada Tel: +1-705-434-1239 Email: rjp@ca.inter.net
BACUS	Joseph De Kerf	Rooienberg 72, B-2570 Duffel, Belgium. Tel: +32 15 31 47 24
Beautiful Systems, Inc.	Jim Goff	308 Old York Road, Suite 5, Jenkintown, PA 19046, USA Tel: +1 (215) 886-2636; Fax: +1 (215) 886-4888 Email: BeautifulSystems@gooffs.net
Camacho	Anthony Camacho	11 Auburn Road, Redland, Bristol BS6 6LS, UK. Tel: 0117-973 0036. email: acam@blueyonder.co.uk
Ray Cannon		21 Woodbridge Rd, Blackwater, Camberley, Surrey GU17 0BS, UK. Tel: 01252-874697 Email: ray_cannon@compuserve.com

Causeway Graphical Systems Ltd	Adrian Smith	The Maltings, Castlegate, MALTON, North Yorks YO17 7DP, UK. Tel: 01653-696760 Fax: 01653-697719 Email: adrian@causeway.co.uk
Paul Chapman		51B Lambs Conduit Street, London WC1N 3NB, UK. Tel: 020 7404 5401. Compuserve: 100343,3210
Cinerea AB	Rolf Kornemark	Box 61, S-193 00 Sigtuna, Sweden. Tel/Fax: +46 859 255 421 Email: rolf@cinerea.se
Ian Clark	Ian Clark	1205 Deer Creek Drive, Plainsboro, NJ 08536, USA. Tel: +1 609 716 8832 Email: earthspot2000@hotmail.com
CODEWORK	Mauro Guazzo	Corso Lanza 105, 10133 Torino, Italy. Tel: +39 11 6601560 Fax: +39 11 6601560 Email: codework@codework- it.com
ComLog Software	Jeff Pedneau	18728 Bloomfield Road, Olney, MD 20832 USA Tel: +1 (301) 260-1435 Email: jeff@softmed.com
CoSy.com	Bob Armstrong	42 Peck Slip #4B, New York, NY 10038-1725, USA. Tel: +1 212-285-1864 Fax: +1 212-285-1864. E-mail: bob@CoSy.com.
David Crossley	David Crossley	187 Le Tour du Pont, 84210 ST DIDIER, France. Tel: +33.4.90.66.08.87 Email: crossley@au-village.com
Danish User Group	Helene Boesen	c/o Insight Systems ApS, Nordre Strandvej 119G, Hellebæk, Denmark
Dinosoft Oy	Pertti Kalliojärvi	Lönnrotinkatu 21C, 00120 Helsinki, FINLAND. Tel: +358 9 70028820 Fax: +358 9 70028824 Email: dinosoft@dinosoft.fi
Dittrich & Partner Consulting GmbH	Axel Holz Müller	Kieler Strasse 17, D-42697 Solingen, Germany. Tel: +49 212-260 660 Fax: +49 212-260 6666; Email: info@dpc.de
Dutch APL Association	Bernard Smoor (Sec)	Postbus 1341, 3430BH Nieuwegein, Netherlands. Tel: +31 347 342 337 Fax: +31 347 342 342
Dyalog Ltd.	Peter Donnelly	Grove House, Lutyens Close, Chineham Court, Basingstoke, Hampshire RG24 8AG, UK. Tel: 01256-336461 Fax: 01256-316559 Email: sales@dyalog.com; support@dyalog.com
DynArray Corporation	Dr James Brown	16360 Monterey Rd. Suite 260, Morgan Hill, CA 95037, USA Tel: +1 (408)-782-6648 Fax: +1 (408)-782-6627 Email: info@DynArray.com
Entropy Software Limited	George MacLeod	Calle Viena 13, Residencia Larrosa, Guardamar del Segura, 03140 Alicante, Spain. Tel: +34 61 00 41 940 Email: george.macleod2003@yahoo.co.uk
Evestic AB	Olle Evero	Berteliusvagen 12A, S-146 38 Tullinge, Sweden Tel & Fax: +46 778 4410 Email: olle.evero@mail.com
FinnAPL	Olli Paavola	Suomen APL-Yhdistys RY, FinnAPL RF, PL 1005, 00101 Helsinki 10, Finland Email: olli.paavola@pyr.fi
First Derivative Analytics Ltd.	Ken Chakahwata	114 Lemsford Lane, Welwyn Garden City, Herts AL8 6YP, UK Tel/Fax: 01707-336620. Email: KenChakahwata@compuserve.com
General Software Ltd	M.E. Martin	Little Wester House, Westerhill Road, LINTON, Kent ME17 4BS Tel: 01622 749365 E-mail: martin@soft.freemove.co.uk
Godin London Incorporated	Gaëtan Godin	12 Gerrard St., London, Ontario, Canada N6C 4C5 Tel: +1 (519) 679-8290 Fax: +1 (519) 438-6381 Email: info@godin.on.ca
HMW Computing	Chris Hogan	Hamilton House, 1 Temple Avenue, London EC4Y 0HA, UK. Tel: 0870-1010-469; Email: HMW@4extra.com
Hoekstra Systems Ltd	Bob Hoekstra	Dominique, Salisbury Road, Woking, Surrey, GU22 7UR, UK. Tel: 01483-771028. Fax: 01483-837324 Email: Bob.Hoekstra@HoekstraSystems.ltd.uk
Michael Hughes		28 Rushton Road, Wilbarston, Market Harborough, Leics. LE16 8QL, UK. Tel: 01536-770998 Email: Michael@Hughes.uk.com
I-APL Ltd	Anthony Camacho	11 Auburn Road, Redland, Bristol BS6 6LS, UK. Tel: 0117-973 0036. Email: 100612.1057@compuserve.com
IBM APL Products	Nancy Wheeler	APL Products, IBM Silicon Valley Lab, Dept H36/F40, 555 Bailey Avenue, San Jose CA 95141, USA. Tel: +1 (408) 463-APL2 [+1 (408) 463-2752] Fax: +1 (408) 463-4488 Email: APL2@vnet.ibm.com
INFOTROY	Alexey Miroshnikov	3 S. TulenIn Lane, St. Petersburg 191186 Russia. Tel: +7 812 325-9797 Fax: +7 812 311-2184 Email: aim@infostroy.ru
Insight Systems ApS	Helene Boesen	Nordre Strandvej 119G, DK-3150 Hellebæk, Denmark Tel: +45 70 26 13 26 Fax: +45 70 26 13 25 Email: info@insight.dk
JAD Software	David Crossley	175 East 96th St., Apt. 17G, New York, NY 10128 Country: USA Tel: +1 (212) 389-6713 Fax: +1 (212) 781-0124 Email: jadsms@usa.net

Japan APL Assoc	Toshio Nishikawa	1-8-13 Masujima Buid.6F Higashi Gotanda Shinagawa-ku, Tokyo Japan 141-0022. Tel: +81 (03) 3280-0411 Fax: +81 (03) 3280-0418 Email: KYY00361@niftyserve.or.jp
J Austria	Joachim Hoffmann	Hochsteingasse 13/26, 8010 Graz, Austria. Tel:0043 (0)316 91 42 51 Mobile: 0043 (0)699 1 91 42 51 2. Email: joachim.hoffmann@gmx.at
JSoftware Inc.	Eric Iversen	33 Major Street, Toronto, Ontario, Canada M5S 2K9. Tel: +1 (952) 470-7345 Fax: +1 (952) 470-9202 Email: info@jsoftware.com
KJK-tieto Oy	Kimmo Kekäläinen	Merikasarminkatu 10 B 56,00160 Helsinki, Finland. Tel: +358 50 55 27 207; Email: Kimmo.Kekalainen@pp.htv.fi
Lambent Technology Ltd	Stephen Taylor	81 South Hill Park, London NW3 2SS, UK. Tel: +44(0)20 7813 3786. Email: stj@lambenttechnology.com
Phil Last Ltd	Phil Last	146 Crossbrook Street, Cheshunt, Herts, EN8 8JY, UK. Tel: 01992-633807 Fax: 0121-359 0375 Email: phil.last@net.nfl.com
Lescasse Consulting	Eric Lescasse	18 rue de la Belle Feuille, 92100 Boulogne, France. Tel: +33.1.46.05.10.76 Fax: +33.1.46.04.60.23 Email: eric@lescasse.com
Lingo Allegro USA, Inc	Walter G. Fil	203 N. LaSalle Street, Suite 2100, Chicago, Illinois 60601, USA. Tel: +1 (800) 546 4621 E-mail: lingo-allegro@visto.com
Lucas Solutions	Jim Lucas	Stubbedamsvej 9C, 3.tv., 3000 Halsinger, Denmark Tel: +45 49 26 52 42. Email: jel@danbbs.dk
Mackay Kinloch Ltd	Alastair Kinloch	519 Webster's Land, Edinburgh EH1 2RX, Scotland, UK. Tel: +44 (0)131 228 5235 Email: alastair.kinloch@btinternet.com Voicemail/Pager 07626 98 3858
MasterWork Software Ltd	Fraser Jackson	PO Box 56-036, Tawa, Wellington, New Zealand. Tel: +64 (4) 232-4440 Fax: +64 (4) 232-4452 Email: fraser.jackson@xtra.co.nz
Mercia Software Ltd.	Gareth Brentnall	Mercia House, Ashted Lock, Aston Science Park, Birmingham, B7 4AZ, UK. Tel: 0121-359 5096. Fax: 0121-359 0375
MicroAPL Ltd.	Richard Nabavi	The Roller Mill, Mill Lane, Uckfield, E.Sussex TN22 5AA Tel: 01825 768050. Fax: 01825 749472 Email: MicroAPL@microapl.demon.co.uk
Milinta Inc.	Dan Baronet	Contact Dan Baronet at danb@milinta.com
Ellis Morgan	Ellis Morgan	Myrtle Farm, Winchester Road, Stroud, Petersfield, Hants GU32 3PE, UK. Tel: 01730-263843 Email: Ellis@mrllfr.demon.co.uk
NY Sig	David Siegel	PO Box 2697, New York, NY10163-2697, USA. Email: NYSIGAPL@ACM.ORG
Oasis b.v.	Louis Rijkse	Lekstraat 4, 3433 ZB Nieuwegein, Holland. Tel: +31 30 60 66 336 Fax: +31 30 60 65 844 Email: rijkse@oasis.nl
Omega Computing Inc	Alan Graham, Andrew Chou	3 Columbus Avenue, Edison, NJ 08817, USA. Tel: +1 (732) 965 9519 Email: alangraham@mindspring.com
Optima Systems Ltd	Paul Grosvenor	Optima House, Mill Court, Spindle Way, Crawley, West Sussex, RH10 1TT, UK. Tel: 01293 562700 Fax: 01293 562699 Email: paul@optima-systems.co.uk
Qualedi Inc.	Nicole Schless Georges Brigham	121 West Main Street, Milford, CT 06460, USA. Tel: +1 (203) 874-4334 Fax: +1 (203) 878-9083. Email: sales@qualedi.com; info@qualedi.com
RadSys Technologies AB	Randolph Schrab	Lovsangerav. 18, S-756 52 Uppsala, Sweden. Tel: +46 18 32 41 53 Fax: +46 708 1996 11 Email: randolph.schrab@radsys.se
Renaissance Data Systems Ed Shaw		P.O. Box 511, Botsford, CT 06404, USA. Tel: +1 (203) 270-9729 Email: apibooks@earthlink.com
Resources & Results	Frank Rhodes	2438 W. Northgate Dr., Irving, TX 75062, USA. Tel: +1 972 523 5117 Email: frank@decision-support.net
RE Time Tracker Oy	Richard Eller	Mikonkatu 8 A, 2.krs, PL 363, 00101 Helsinki, Finland. Tel: +358 9-621 3300 Fax: +358 9-621 3378 Email: re@rett.fi
Right Seat Software, Inc.	Tom Atkins	1110 12th Street, Golden, CO 80401, USA. Tel: +1 303 278 2244. Fax: +1 303 278 6957. Email: info@voxproxy.com
Graeme Robertson	Graeme Robertson	15 Little Basing, Basingstoke, Hants RG24 8AX, UK. Tel: +44 0 1256-364071 Email: GraemeDR@aol.com
The Rochester Group Inc.	Robert Miller	600 Park Avenue, Rochester, NY 14607-2928, USA. Tel: +1 (716) 271-1110. Fax: +1 (716) 271-1230
Rome/Italy SiG	Mario Sacco	Casella Postale 14343, 00149-Roma Trullo, Italy Email: mario.sacco@tin.it
SE APL Users Group	John Manges	413 Comanche Trail, Lawrenceville, GA 30044, USA Tel: +1 (770) 972-3755 Email: seapldoc@aol.com

Shepp & Associates LLC	Andrew Shepp	1312 Washington Avenue, 8th Floor St. Louis MO 63103, USA Tel: +1 (314) 621-3272 Fax: +1 (314) 621-4267 Email: ashepp@compuserve.com
Snake Island Research Inc	Bob Bernecky	18 Fifth Street, Ward's Island, Toronto, Ontario M5J 2B9 Canada Tel: +1 (416) 203-0854 Fax: +1 (416) 203-6999 Email: bernecky@interlog.com
SOCAL (South California)	Roy Sykes Jr	Sykes Systems Inc, 4649 Willens Ave, Woodland Hills, CA 91364-3812 USA. Tel: +1 (818) 222-2759 Fax: +1 (818) 222-9250
Soliton Associates	Benoit Paquin	Soliton Denmark, Havsgårdsvej 4, 2900 Hellerup, Denmark Email: sales@soliton.com
SovAPL Russian Chapter of SIGAPL	Alexander Skomorokhov	PO Box 5061, Obninsk-5, Kaluga Region 249020, Russia Tel: +7(08439)47109 Fax: +1 (530) 6885510 Email: askom@obninsk.com
Strand Software Inc	Anne Faust	P.O. Box 330, Excelsior, MN 55331 USA Tel: +1 (952) 470-7345 Fax: +1 (952) 470-9202 Email: info@strandsoft.com
Rex Swain	Rex Swain	8 South Street, Washington, CT 06793 USA. Tel: +1 (860) 868-0131 Fax: +1 (860) 868-9970 Email: rex@rexswain.com
SwedAPL	Christer Ulfhjelim	Novator Consulting Group AB, Svårdvägen 11C, S-182 33 Danderyd Sweden. Tel: +46 8 622 63 50 Fax: +46 8 622 63 51 CService: 100341,404
Swiss APL User Group		Swiss APL User Group, CH-3001, Bern 1, Switzerland Email: si@ifi.unizh.ch
Sykes Systems Inc	Roy Sykes Jr	4649 Willens Ave., Woodland Hills, CA 91364, USA Tel: +1 (818) 222-2759 Fax: +1 (818) 222-9250
Toronto SIG	Richard Procter	PO Box 55, Adelaide St. Post Office, Toronto Ontario M5C 2H8, Canada Email: info@torontoapl.org
Weighahead Systems	Philip Bulmer	Camberley House, 1 Portesbery Road, Camberley, GU15 3RB, UK. Tel +44 1276 20769 Email: sales@weighahead.com
Stephen Wynn		8 Clarence Gardens, Brighton, Sussex BN1 2EG, UK. Tel: 01273-327238 Email: centre@cwcom.net
Zark Incorporated	Gary A. Bergquist	23 Kelchbrook Lane, Ellington CT 06029, USA. Tel: +1 (860) 872-7606

FTP SITES

IBM APL2	ftp.software.ibm.com/ps/products/apl2
Waterloo Archive	archive.uwaterloo.ca/ftparch/languages/apl
APL-to-ASCII	archive.uwaterloo.ca/languages/apl/workspaces/aplascii

WORLD WIDE WEB SITES

ACM SigAPL	www.acm.org/sigapl/
Adapta Software	www.adapta.de/
Adaptable Systems	www.assuredsystems.com.au/
AFAPL	www.afapl.asso.fr/ (Journal available on line)
APL2000	www.APL2000.com/
APL-385	www.demon.co.uk/apl385/
APL Journal, Germany	www.rhombos.de/apljourn.htm
APL Borealis Inc.	www.aplborealis.com
APL Systems IDC SL	www.apl2000.net
AUSCAN	http://home.ca.inter.net/rip/auscan/
Eke van Batenburg	www.bio.LeidenUniv.nl/~Batenburg/index.html
Causeway	www.causeway.co.uk/
CODEWORK	www.codework-it.com/tangram/eng/
CoSy (Bob Armstrong)	CoSy.com/
Dinosoft Oy	www.dinosoft.fi/
Dittrich & Partner	www.dpc.de ; www.apl-online.de
DMOZ - Open Directory	http://dmoz.org/Computers/Programming/Languages/APL/

Dyalog Ltd	www.dyalog.com/
DynArray	www.dynarray.com/
FinnAPL	www.pyr.fi/apl/
Godin London Inc	www.godin.com/
Houben (IQL)	www.apl.clap.club.tip.nl
Hoekstra Systems	www.HoekstraSystems.ltd.uk/
IBM APL2	www.ibm.com/software/ad/apl
Infostroy	www.infostroy.ru
Insight Systems ApS	www.insight.dk/
Japan APL Association	www.naska.co.jp/JAPLA/
JSoftware Inc	www.jsoftware.com/
KJK-tieto Oy	www.kjk-tieto.com
Lambent Technology	www.lambenttechnology.com
Lescasse Consulting	www.lescasse.com/
Lingo Allegro USA Inc	www.lingo.com/
Mackay Kinloch	http://mackaykinloch.ltd.uk/
MicroAPL Ltd	www.microapl.co.uk/apl
Milinta Inc	www.milinta.com
Oasis b.v.	www.oasis.nl/
Optima Systems Ltd	www.optima-systems.co.uk
Qualedi, Inc.	www.qualedi.com
Renaissance Data	www.aplbooks.com/
Resources & Results	www.decision-support.net
RE Time Tracker Oy	www.rett.fi/
Right Seat Software, Inc.	www.voxproxy.com
The Rochester Group Inc.	www.rochgrp.com/
Shepp & Associates	www.digitravel.com/
SigAPL	www.acm.org/sigapl/
Soliton	www.soliton.com/
Snake Island Research Inc.	www.snakeisland.com
Strand Software Inc.	www.strandsoft.com/
Rex Swain	www.rexswain.com/
Toronto SIG (for Toolkit)	www.torontoapl.org/
Jim Weigang	www.chilton.com/~jimw/
Weighahead Systems	www.weighahead.com

Zark Newsletter Extracts

Introduced by Jon Sandles

Here is another Zark newsletter extract. As always, don't hesitate to send in any solutions you may have that you may be inspired to investigate. There may be new more elegant ways of solving an old problem, which may also turn out to be faster. Please don't be afraid to try and shed "new" light on the "old" problems. Once again, thanks to Jonathan Barman for helping typeset this column.

Utility Corner:

Numbers to Words

(The purpose of this column is to make you more productive by introducing you to utility functions. Think of utility functions as APL functions that have names instead of symbols. By expanding your function vocabulary, you'll be able to write APL code that's more concise, more efficient, and more readable.)

In last issue's *Limbering Up* column, you were asked to define a utility function named `NUMBERWORDS`, which takes a single positive integer as its right argument and returns a character vector that describes the number. For example:

```
NUMBERWORDS 1234
one thousand two hundred thirty four
NUMBERWORDS 1007
one thousand seven
NUMBERWORDS 0
zero
NUMBERWORDS 1003011
one million three thousand eleven
```

Given such a function, the task of formatting a cheque amount into words is straightforward:

```

▽ R←CHECKAMOUNT N;C;D
[1] A Format a $ amount (e.g. 1234.56)
[2] A into words (e.g. 'One thousand tow
[3] A hundred thirty four and 56/100
[4] A dollars'). Needs NUMBERWORDS.
[5] A
[6] D←[N A Dollars
[7] C←100×N-D A Cents
[8] D←NUMBERWORDS D A Dollars to words
[9] A Capitalize first letter:
[10] D[⌊IO]←'EFNOSTZ'[ 'efnostz'⌊D[⌊IO]]
[11] A PUT it all together:
[12] R←D, ' and ', (0≠C), '/100 dollars'
▽

```

CHECKAMOUNT 1234.56

One thousand two hundred thirty four and 56/100 dollars

The most important trait we looked for in the NUMBERWORDS functions we received was that it produced the correct result. Speed, program size, and elegance were not considered.

Unfortunately, the concept of "correct" got a bit fuzzy when we started receiving the entries from outside the United States. The British "billion" (a million million) and the American "billion" (a thousand million) are different. Likewise for Trillion, Quadrillion, and so on.

For the purposes of this discussion, we will use American terminology (what did you expect?). All functions have been converted accordingly.

The solutions below do not attempt to spell numbers beyond 999 quadrillion. This may be due to the fact that such immense numbers are rare indeed, and that the need to spell them is even more rare. Or, it may be due to the fact that no one was quite sure what follows "quadrillion". The most practical explanation is that most APL systems maintain only 16 or 17 significant digits of precision. Since the number 999 quadrillion has 18 digits, the spelling of such a number will be faulty at best.

The first solution, from Jim Weigang, breaks the number into six groups of three digits. Each trio of numbers is then passed to a subfunction (SPELL3), which spells out those three digits (e.g. 'one hundred twenty three') and includes the appropriate suffix (e.g. 'billion').

```

▽ R←NUMBERWORDS N;D;IO
[1] A Returns lowercase character vector
[2] A spelling (in American English)
[3] A integer N (0 to 1E18-1).
[4] A Needs subfn: SPELL2 and globals
[5] A SPELLTAB1 and SPELLTAB2
[6] A
[7] IO←1 A Origin 1 is fine
[8] R←'zero'
[9] →(N=0)/0 A Special case for zero
[10] D←(18p10)÷N A Split digits apart
[11] R←D[1 2 3]SPELL3' quadrillion'
[12] R←R,D[4 5 6]SPELL3' trillion'
[13] R←R,D[7 8 9]SPELL3' billion'
[14] R←R,D[10 11 12]SPELL3' million'
[15] R←R,D[13 14 15]SPELL3' thousand'
[16] R←R,D[16 17 18]SPELL3''
[17] A Discard padding and leading space
[18] R←1;R~','

```

▽

```

▽ R←D SPELL3 S;A;B;IO
[1] A Spells the three digits D and adds
[2] A the 'xillions' suffix S.
[3] A
[4] IO←1
[5] R←'' A Empty result for zero
[6] →(D^=0)/0
[7] A Branch if less than 100
[8] →(D[1]=0)/L1
[9] A Spell the first digit
[10] R←SPELLTAB1[1+D[1];],' hundred'
[11] A Branch if 20 or more
[12] L1:→(D[2]≥2)/L2
[13] A Else pack 0 to 19 and spell it:
[14] R←R,SPELLTAB1[1+10L1+D;]
[15] →L3
[16] L2:A+SPELLTAB2[D[2];] A E.g. forty
[17] B←SPELLTAB1[1+D[3];] A E.g. seven
[18] R←R,A,B
[19] L3:R←R,S A E.g. million

```

▽

Here are the globals needed by SPELL3:

SPELLTAB1

```

.....
one.....
two.....
three....
four.....
five.....
six.....
seven....
eight....
nine.....
ten.....
eleven...
twelve...
thirteen.
fourteen.
fifteen..
sixteen..
seventeen
eighteen.
nineteen.

```

SPELLTAB2

```

.....
twenty.
thirty.
forty..
fifty..
sixty..
seventy
eighty.
ninety.

```

Notice the use of period (.). On line [18] of NUMBERWORDS, the expression $R \leftarrow 1 \downarrow R \sim ' . '$ generates the desired spacing (one space between each word) by excluding the periods but leaving the one space that precedes each word in SPELLTAB1 and SPELLTAB2 (except for the very first space) —

The second solution, from Ray Powell, breaks the number into only as many digits as it contains (using $10 \circ N$), and then works with the digits in reverse order of significance. The words are then reversed at the end. Ray's solution uses nested arrays (submitted in Sharp APL notation, but displayed here in APL2/APL*PLUS notation) to simplify working with words of different lengths.

```

▽ R←NUMBERWORDS N;A;C;I;S;SFX;T;TENS;UNITS;⌈IO
[1]  A Returns lowercase character vector
[2]  A spelling (in American English)
[3]  A integer N (0 to 1E18-1)
[4]  A

```

```

[5]      [IO+1 A Origin 1 is fine
[6]      R+zero'
[7]      →(N=0)/0 A Exit if zero
[8]      A Nested vectors of possible values:
[9]      UNITS+one' 'two' 'three' 'four' 'five' 'six' 'seven' 'ei
ght' 'nine' 'ten' 'eleven' 'twelve' 'thirteen' 'fourteen' 'fifte
en' 'sixteen' 'seventeen' 'eighteen' 'nineteen'
[10]     TENS+ten' 'twenty' 'thirty' 'forty' 'fifty' 'sixty' 'sev
enty' 'eighty' 'ninety'
[11]     SFX+thousand' 'million' 'billion' 'trillion' 'quadril
lion'
[12]     A Break number into digits reversed:
[13]     S→pA+φ((1+[10*N)p10)τN
[14]     A
[15]     A R: nested vec of words in result.
[16]     A C: nested vec of last 3 digit grp
[17]     R+C+10
[18]     I+0 A Loop by digit
[19]     LOOP:→(S<I+I+1)/END
[20]     A Units, tens, or hundreds?
[21]     →(1 2 0=3|I)/LU,LT,LH
[22]     A
[23]     A C contains the description for the
[24]     A previous group of three. Save it.
[25]     LU:R+R,C
[26]     C+10
[27]     A Branch unless this group is all 0:
[28]     →(0v,≠3†(I-1)†A)/L1
[29]     I+I+2 A Jump to the next group
[30]     →LOOP
[31]     A Branch if not SFX description:
[32]     L1:→(1=I+I÷3)/L2
[33]     C+SFX[I] A Else grab it
[34]     A Ignore if this digit is zero:
[35]     L2:→(A[I]=0)/LOOP
[36]     A Branch if next digit gives 20+
[37]     A (or if no more digits)
[38]     →(S=I)/L3
[39]     →(19<I+10+A[I+1 0])/L3
[40]     A Else use 1-19 table and increment:
[41]     C+C,UNITS[I]
[42]     I+I+1
[43]     →LOOP
[44]     A Use 1=9 table and don_t increment:
[45]     L3:C+C,UNITS[A[I]]
[46]     →LOOP
[47]     A
[48]     A Ignore if this digit is zero:
[49]     LT:→(A[I]=0)/LOOP
[50]     C+C,TENS[A[I]]
[51]     →LOOP

```

```

[52] A
[53] A Ignore if this digit is zero:
[54] LH:→(A[I]=0)/LOOP
[55] C←C,←(A[I]>UNITS), ' hundred'
[56] →LOOP
[57] A
[58] A Stick blanks between words:
[59] END:R←1↑,/, ' ', "ϕR,C

```

▽

The third solution, from Ram Biyani, recurses, i.e. calls itself. The function is written to handle 0, or 1 to 19, or 20 to 99. Beyond 99, the function breaks the number into seven groups of digits (quadrillions, trillions, billion, millions, thousands, hundreds, and 0 to 99), and calls itself once for each of these new numbers, applying the appropriate suffix to each, and retaining only those that are not zero. Again, nested arrays are used.

```

▽ R←NUMBERWORDS N;SFX;TENS;UNITS;V;IO
[1] A Returns lowercase character vector
[2] A spelling (in American English)
[3] A integer N (0 to 1E18-1). Recurses.
[4] A
[5] R←'zero'
[6] →(N=0)/0 A Exit if zero
[7] →(N≥100)/BIG A Branch if 100+
[8] A
[9] UNITS←' ' 'one' 'two' 'three' 'four' 'five' 'six' 'seven'
'eight' 'nine' 'ten' 'eleven' 'twelve' 'thirteen' 'fourteen' 'fi
fteen' 'sixteen' 'seventeen' 'eighteen' 'nineteen'
[10] A Branch unless 1 to 19:
[11] →(N≥20)/MEDIUM
[12] R←(N+IO)UNITS A Pick it out
[13] →0
[14] A
[15] A Tens (20 to 99)
[16] MEDIUM←TENS←'twen' 'thir' 'for' 'fif' 'six' 'seven' 'eigh'
'nine', 'c'ty'
[17] A Break apart tens and untis:
[18] V←10 10↑N
[19] R←1↑,/(V≠0)/' ', "IO←"(V-2 0)↑"TENS UNITS
[20] →0
[21] A Recurse to translate numbers ≥100:
[22] BIG←SFX←' quadrillion' ' trillion' ' billion' ' million' '
thousand' ' hundred' ' '
[23] A Break apart number:
[24] V←(0,((~3+pSFX)p1000),10 100)↑N
[25] R←1↑,/(V≠0)/' ', "(NUMBERWORDS"V),"SFX

```

▽

The fourth solution, from Brad Buck, also recurses. The function is written to handle 0 to 19 without recursing. For any number beyond that, it recurses. This function looks at the magnitude of the number (via $\lfloor 1000 \times N \rfloor$) to determine the largest suffix required, and recurses only so many times as it needs to. Nested arrays are not used.

```

      V R+NUMBERWORDS N;F;M;P;T;IO
[1]  A Returns lowercase character vector
[2]  A spelling (in American English)
[3]  A integer N (0 to 1E18-1). Recurses.
[4]  A
[5]  IO+1 A Origin 1 works best
[6]  A Zero, Units, Tens, Hundreds:
[7]  →(N≤0 19 99 999)/LZ,LU,LT,LH
[8]  A
[9]  A Else 1000 or more:
[10] P+[1000*N A 1=1E3+1E6, 2=1E6+1E9...
[11] T+5 11p'thousand million billion trillion quad
million'
[12] T+T[P;]~' '
[13] R+(NUMBERWORDS M+[N÷F+1000*P),' ',T
[14] →MORE
[15] A
[16] A Zero:
[17] LZ:R+'zero'
[18] →0
[19] A
[20] A Units (1 to 19):
[21] LU:T+19 9p'one two three four five si
x seven eight nine ten eleven twelve thi
rteen fourteen fifteen sixteen seventeeneighteen nineteen '
[22] R+T[N;]~' '
[23] →0
[24] A
[25] A Tens (20 to 99):
[26] LT:T+10 7p'ten twenty thirty forty fifty sixty seven
tyeighty ninety '
[27] R+T[M+[N÷F+10;]~' '
[28] →MORE
[29] A
[30] A Hundreds (100 to 999):
[31] LH:R+NUMBERWORDS M+[N÷F+100
[32] R+R,' hundred'
[33] A
[34] A Reduce by amount formatted so far:
[35] MORE:N+N-M×F
[36] →(N=0)/0 A Exit if nothing left
[37] A Else include the rest
[38] R+R,' ',NUMBERWORDS N
      V

```

From Rex Swain

23rd June 2003

Back in 1976, I was working for Scientific Time Sharing Corporation. I remember a broadcast request from Jack Becker, who wrote the company's internal accounting systems, for exactly such a function. I came up with what I thought was a very clever recursive solution:

```

      ▽ R←WORDS X;A;B;C;D;H;O;P;Q;S;T;□CT;□IO
[1]  A TRANSLATE AN INTEGER INTO WORDS
[2]  A REQUIRES: -
[3]  A REX SWAIN 11 MAY 1976; ENHANCED SLIGHTLY 30 DECEMBER 198
1
[4]  □CT←0 □ IO←1
[5]  S←+/□SI[;16]^.= 'WORDS[' A RECURSION DEPTH
[6]  +(0≠X)pa □ R←(1=S) / 'ZERO' □ →0
[7]  a←A+ 10 6 p 'ZERO ONE TWO THREE FOUR FIVE SIX SEVE
N EIGHT NINE
[8]  B←'TEN ELEVEN TWELVE THIRTEEN FOURTEEN FIFT
EEN
[9]  B← 10 10 pB, 'SIXTEEN SEVENTEEN EIGHTEEN NINETEEN
[10] C←' TWENTY THIRTY FORTY FIFTY SIXTY
'
[11] C← 10 8 pC, 'SEVENTY EIGHTY NINETY
[12] D← 5 9 p' THOUSAND MILLION BILLION TRILLION
[13] P←1+Q← 10 10 10 T X A LAST THREE DIGITS
[14] H←(XQ[1])/A[P[1];], 'HUNDRED ' A HUNDREDS
[15] T←(2≤Q[2])/C[P[2];] A TENS
[16] O←((1=Q[2])/B[P[3];]), (1 0 ^.=1+Q)/A[P[3];] A TEENS OR ON
ES
[17] R←(WORDS[X÷1000],H,T,O,(0≠1000[X])/D[S;] A RECURSE AND ASS
EMBLE
[18] →(1≠S)pO □ B←R←' ' □ R←(Bv1φB><\B)/R A DELETE EXTRANEOUS
BLANKS
      ▽

```

But a couple of days later another STSC'er named Bob DeCloss submitted a different solution:

```

      ▽ R←WORDS2 N;A;B;C;D;a;b;c;d
[1]  A CONVERTS INTEGER ARRAY INTO WORDS
[2]  A REQUIRES: -
[3]  A BOB DECLOSS, CIRCA 1976
[4]  a← 10 6 p'/////////ONE //TWO THREE /FOUR /FIVE //SIX SEVEN
EIGHT /NINE
[5]  b←'//////////TWENTY ///THIRTY ////FORTY ////
FIFTY ////'
[6]  b←b, 'SIXTY //SEVENTY ///EIGHTY ///NINETY ////TEN ///ELE
VEN ///'
[7]  b←b, 'TWELVE /THIRTEEN /FOURTEEN //FIFTEEN //SIXTEEN SEVEN

```

```

TEEN /'
[8] b+ 20 10 pb,'EIGHTEEN /NINETEEN '
[9] c+ '/////////////////ONE HUNDRED //TWO HUNDRED THREE HUNDRE
D /'
[10] c+c,'FOUR HUNDRED /FIVE HUNDRED //SIX HUNDRED SEVEN HUNDR
ED '
[11] c+ 10 14 pc,'EIGHT HUNDRED /NINE HUNDRED '
[12] d+ 4 11 p'/////////////////THOUSAND ///MILLION ///BILLION '
[13] C←3×D÷[(p⌈N÷N)÷3 ⋄ B←'ppA←((D×pN),3)p&(Cp10)τN
[14] C←(A[;2]=1)/ιB ⋄ A[C;2]+A[C;3]+10×A[C;2]
[15] A[C;3]+0 ⋄ A←A+1
[16] R←,c[A[;1];],b[A[;2];],a[A[;3];],d[1{(v/1<A)×φBpιD;}]
[17] R[(R=' ')/ιpR]+□TCDEL ⋄ R←((pN),D×41)pR
v

```

No recursion. No looping. No branching. And it does more than one number at once. Wow.

Rex Swain ; Independent Consultant
 Tel: 860-868-0131 | Fax: 860-868-9970
 Mail: 8 South Street; Washington, CT 06793; USA
 Email: rex@rexswain.com | Web: <http://www.rexswain.com>

Limbering Up: Comparing Functions

(The purpose of this column is to work some flab off your APL midsection. Like muscles, your APL skills can atrophy if not exercised with adequate frequency and variety. This column presents a task for you to perform. Set aside a few minutes from your busy schedule and work the task. Mail in your solution and stay tuned for the results.)

When maintaining a large APL application, you will occasionally find yourself face-to-face with two copies of the same function. Yet the copies are not the same. Perhaps two different programmers enhanced the same function in different ways. You would like to know how the functions are different.

The first objective is to bring the two functions together so they can be compared. Yet, if you copy both functions into the workspace, only one will survive (since they have the same name!) To compare the functions you must convert them to character array representations via □CR (canonical representation) or □VR (visual representation). For example:

```

)COPY WS1 MODEL
SAVED...
CR1←⊠CR 'MODEL'
)COPY WS2 MODEL
SAVED...
CR2←⊠CR 'MODEL'

```

Now you have two character matrices that “look like” the function. Your task is to write a function CRCOMPARE (or VRCOMPARE if you are using ⊠VR) that returns/displays the differences between the functions, like so:

```

CR1 CRCOMPARE CR2

[1] A+B+C
[2] D÷3
---- replaced by ----
[1] A+B+C ÷ D÷3

[5] A+B÷0 0 p0
---- added ----
[6] A÷0 0 p0
[7] B÷0 0 p0
---- deleted ----

[9] Z÷0.0×D
---- replaced by ----
[7] Z÷D÷100

```

If using ⊠VR, make sure you ignore the line numbers, which are part of the visual representation, when comparing the old and new functions. Otherwise, if a line is inserted between [5] and [6], each line in the new function from [7] up will appear different because of its new line number. If using ⊠CR, make sure you display the line numbers with the modified lines of code.

Send your solution to:

Vector Production
 Brook House, Gilling East
 York YO62 4JJ
 UK email apl385@compuserve.com

The notable functions and their authors' names will be printed in the next issue of *Vector*. Good luck and happy limbering.

19.4 Solution, Puzzle, and New Crossword

Solution to Crossword in 19:4

	L	/	(	V	>	0	)	/	V		(
			(		/		P		×	\	V
	→	J	V		M		C		J		≠
	T	-	ι	5		S	O	1	-	ι	1
	F		V	2	G		P	↓	R		↓
(	ρ	M	)	ρ	1		Y	ρ	1	ρ	V
	L	B	=	K	×	4		M		ρ	,
1	0	×	ι	1	2	3		T	2	M	-
		P	L			2	4		φ	V	1
(	ι	9	)	~	5			(	ι	0	)
		2	/	1		T				1	/

Puzzle (from Howard Peelle)

NOW I KNOW

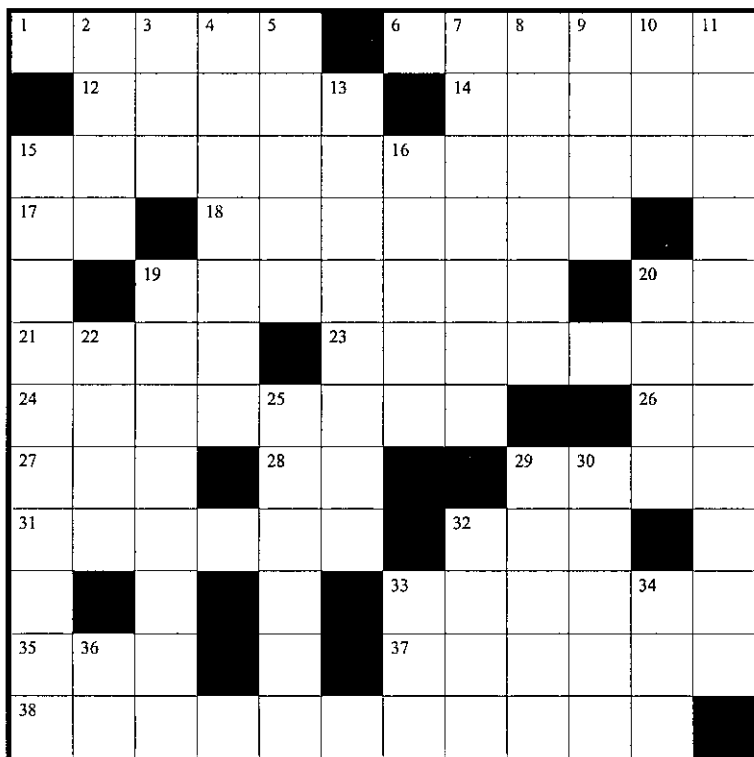
There are two positive integers, each greater than 1 and less than 100.

Pete knows their product. Serena knows their sum. First, Pete says: "I don't know what they are." Then Serena says: "I know you don't know." Then Pete says: "So, now I know." And Serena says: "So do I." What are the two integers?

This problem has been around for a while in various guises and is usually presented in terms of Mr. P (product) and Mr. S (sum).

I have changed the names to *Pete* and *Serena*, in reference to two great tennis players. Solution in the next Vector!

New Crossword



23. $RB[5+1TN]$
24. Current size of the symbol table
26. $\ast/Ip\ast 1$
27. $C[;1]\div C[;2]$ for two-column matrix C
28. $P=0$ for Boolean P
29. $J2L[]$ to get J2L in sorted order
31. $\sim 15+1$
32. $M[];$ to get M in alphabetic order according to the alphabet A
33. Name of the $\ast$ function
35. Rank (number of dimensions) of Y
37. One random number from the set $(U\times L+1), (U\times L+2), (U\times L+3), \dots (U\times L+Y)$
38. How good programmers spend the majority of their time

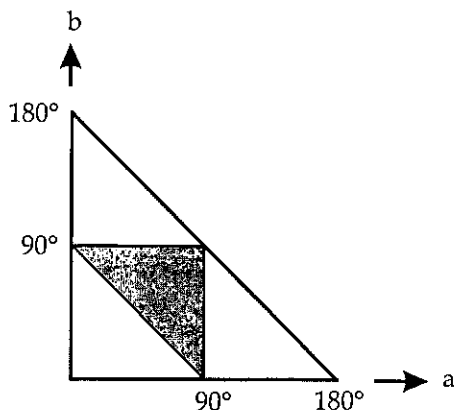
Down

2. Display VC to W decimal places
3. $A-S\times[A\div S]$
4. Index in vector IM of its largest number
5. Elements in vector VRV corresponding to 1s in bit vector D
7. Flag the first M elements of S that equal -AM
8. Index in vector SA of $1\div P5$
9. Array of 2s, with shape TT
10. Vector V with scalar E stuck to its front
11. Speed, size and _____: the key measures of APL program quality
13. Flag elements of TOP5 that match scalar XAD
15. Average across each row of matrix D
16. Local variables in suspension
19. Flag leading elements of vector YC that are integers
20. Area of circle of radius R
22. $(+/KS)p5$ for integer vector KS
25. $(\sim B\in M[12+1(pM)-12])/B$
29. Ranking vector for golf scores RLI (1 = lowest score)
30. Add N to each element of JMM
32. Product of AO and T
33. APL is _____ to use
34. From the set $1G$, return A distinct random numbers
36. Length of vector O

J-ottings 37: How many Obtuse-Angled Triangles are there?

asks Norman Thomson

In the world, that is. Lots, I know, but let me be more precise and ask what **proportion** of all triangles in the world are obtuse-angled? You might suggest that since only two of the angles, say a and b , of a triangle can be determined independently, the set of all possible triangles is denumerable with points contained within the large right-angled triangle in the diagram below



and further, since the acute-angled ones are denumerable with the points within the shaded triangle, it follows that 75% of all triangles are obtuse-angled. Well, let's ask J and see whether he/she/it (as far as I am aware J has not yet been honoured-with-an-official-gender) agrees.

Assume first that a point is represented by a 2-list of numbers such as 4 7. The verb `py` calculates the square of the distance between two such points :

```
py=. (+/@: *: @: - /) "2 NB. square of dist btwn 2 points
py 4 7, : 1 3 NB. py applied to a 3,4,5 triangle
```

25

A triangle is a 3-list of 2-lists (points). To calculate the squares of the three sides first select the points in pairs, and then apply `py` :


```

      sides =.(0 1;1 2;0 2)&({&>)}@< NB. select point pairs
      ]t=.3 2$_3 _2 _1 6 0 2          NB. specimen triangle
      _3 _2
      _1 6
      0 2
      sides t
      _3 _2
      _1 6

      _1 6
      0 2

      _3 _2
      0 2

      py sides t          NB. squares of three sides of triangle
      68 17 25

```

A triangle is obtuse if the square of the largest side exceeds the sum of the squares of the other two, so define `ifbig1` to test whether the largest of a set of numbers is greater than the sum of all the others:

```

      ifbig1=.({.>+/@).)@\::~ NB. 1 if biggest > +/rest
      ifbig1 49 43 100
      1

```

This leads naturally to

```

      obtest=.ifbig1@:py@sides NB. 1 if obtuse, 0 if not
      obtest t
      1

```

To generate random triangles centred on the origin, construct lists like `t` using an integer which determines the coordinate resolution. This should be big enough to minimise the inaccuracy arising from the fact that `?` returns integers:

```

      rt1=.-- ?@>:@+:@(3 2&$)NB. rand trngl,coords in[-y.,y.]
      rt1 100
      _94 16
      _36 _68
      _6 _14

```

Everything is now in place to test a single random triangle with given resolution:

```

      trial1=.obtest@rt1 NB. 1 if rand trngl is obtuse
      trial1 100
      0

```

so this particular triangle was acute-angled. Here are a thousand trials at a higher resolution :

```

+/trial1&>1000$10000      NB. test 1,000 random triangles
747

and a few more :

+/trial1&>1000$10000
721
+/trial1&>1000$10000
738
+/trial1&>10000$10000      NB. increase the number of trials
7251

```

By now you should be getting a feel for where the true value lies, namely somewhere between 72 and 73 per cent. Here is a summary of the J verbs used in this simulation :

```

py=. (+/@: *: @: - /) "2      NB. sq of dist btwn 2 points
sides=. (0 1; 1 2; 0 2) & ( { & > ) @ <      NB. 3 sides each a pt.pair
ifbig1=. ( { . > + / @ ) . ) @ \ : ~      NB. 1 if biggest > +/rest
obtest=. ifbig1 @: py @ sides      NB. 1 if obtuse, 0 if not
rt1=. ~ ~ ? @ > : @ + : @ ( 3 2 & $ )      NB. rand trngle in [-y., y.]
trial1=. obtest @ rt1      NB. obtuseness test

```

Another way to conduct the simulation is to generate random angles rather than random sides. Start by writing a verb which generates a number of random values (angles) between 0 and 2π :

```

rnd=. (% ~ ?) @ ( 1e9 & (# ~ ) )      NB. y.random uniform (0,1) s
pi2=. o. 2      NB. pi/2 = 6.28319
ra=. * & pi2 @ rnd      NB. y. random angles in (0, pi2)
ra 6
4.97256 1.18961 0.688103 3.88122 5.97772 5.69835

```

The sines and cosines of each angle generate a point within the unit circle, and so random angles can be transformed to random points within the unit circle by :

```

cs=. o. ~ / & 2 1 @ ra      NB. cos/sine s of y. random angles
cs 3
_0.0880986 0.996112
_0.23431 _0.972162
_0.772602 _0.63489

```

and then to random triangles in a circle with radius y. by

```

rt2=.3 :'y.*cs 3' NB. rand trgle in circle radius y.
rt2 100
37.8875 92.5448
_48.1939 87.6205
71.0356 _70.3843

```

The apparatus is already in place for determining whether these triangles are obtuse or not, and for doing some more experiments :

```

trial2=.obtest@:rt2
+ /trial2&>10000$10000
7555

+ /trial2&>10000$10000
7518

```

A little bit higher than our previous estimate, but close to the theoretical 75%. Maybe sampling from a square frame includes disproportionately many 'skinny' acute angled triangles which have one vertex tucked into one of the far corners. To test this conjecture, sample the co-ordinates within a square but accept only those triangles all of whose points lie inside the largest inscribed circle. The final line of rt3 embodies recursively the "accept-reject" technique which is common in simulation practice :

```

ssq=.+/@:*: NB. sum of squares
rt3=.3 :0 NB. reject if one pt. between circle and square
r=.rt1 y.
while. +./(*:y.)<ssq"1 r do.r=.rt3 y.end.
)

```

As before, everything else is already in place for the next experiment :

```

trial3=.obtest@rt3
+ /trial3&>10000$10000
7306

+ /trial3&>10000$10000
7255

```

Not much different from trial1 – ah well, that's one conjecture gone for a bang!

Another accept-reject technique for random triangles consists of generating the values of three random sides in order, and rejecting the result if the sides fail to form a triangle because one of them is bigger than the sum of the other two :

```

rt4=.3 :0      NB. reject if one side > sum of other 2
r=.:?3$y. NB. select three random sides in [1,y.]
while.ifbig1 r do. r=.rt4 y. end.
)

```

Since `rt4` returns lengths of sides, the obtuseness test is simply `ifbig1` applied to the squares of the sides :

```

trial4=.ifbig1@:.*:rt4
+ /trial4&>10000$10000
5675

+ /trial4&>10000$10000
5694

```

Seems I've mislaid nearly 20% of the world's obtuse angled triangles (careless of me!). Another possibility based on sides is to choose two sides at random, say these were 6 and 10. The range of acceptable values for a third side is then from 10-6 to 10+6, or more generally from $m-n$ to $m+n$ where m and n are the lengths of the sides in decreasing order. The third side can then be taken as $(m-n)$ plus a number drawn at random from 0 to $2n$:

```

choose2=. \:~@{?,?} NB. two sides chosen at random
third=., -/+?@>:@+:@{ NB. join feasible random 3rd side
rt5=.third@:choose2 NB. random triangle
trial5=.ifbig1@:.*:rt5 NB. test for obtuse
+ /trial5&>10000$10000
7263

+ /trial5&>10000$10000
7324

```

That's better – but let's get back to the question. How many obtuse-angled triangles did I say there were.....?

RECENT MEETINGS

In this issue, we cover two AGMs – the BAA in London and the German APL group at Stuttgart. We also have some supporting material from Simon Garland (for which many thanks), a note from Steph Johnson on the Kx 10-year celebration and a promise of more K articles to come for future Vectors.

Finally, thanks to Ian Clark for making the trip to San Diego for us, and to Ray Cannon for the pictures.

BAA AGM and Vendor Presentations

notes by Adrian Smith

John Scholes of Dyalog Ltd

John gave his usual laconic amble through the latest release of the Dyalog APL interpreter.

"We've been short-changing you for years – we never used the right-hand column of the session! With Version 10 you get 2% more columns!!"

Dyalog have also followed +Win in adding the option of automatic `⎕PW` and have taken a small walk on the wild side by allowing deletion of unwanted material from the APL session. Personally, I still rate the total anarchy of APL*PLUS/PC where you just typed over anything!

John showed how easy it now is to export your application as a bound executable (you also get to choose the icon) which is quite a step forward for serious developers. It makes it very much simpler to set up the 'associations' for your filetypes, allowing Windows to start you automatically on (for example) the user double-clicking one of your datafiles. Many of us have written 'trivial' launcher applications in C++ or Delphi which have little purpose other than providing Windows with a genuine .EXE to call. Hopefully, these can now head straight for the long grass where they belong.

In answer to a question from the audience, John gave us a brief outline of how the tokeniser treated idioms. Basically a 'spare' token 1B acts as a marker "Here comes an idiom" and is followed by the token for the idiom. However the original code is left in place so that version-9 can still run the V10 workspace (this was required as PocketAPL is effectively V10 but came out before the real V10 for general users). There is effectively no cost attached to the addition of more idioms, except that the interpreter gets a little bigger each time. All the recognition happens when the function is fixed, which is also why loading a V8 or V9 workspace into V10 takes noticeably longer. John recommended taking the time to resave applications destined for shipping in V10.

Two more things to mention – memory mapped files are now implemented, and everything is around 10% faster simply as a result of switching to the Microsoft compiler.

APL Germany Meeting IBM Stuttgart, 9th May 2003

notes by Adrian Smith

Background

The meeting followed the usual format of a Guide-Share day (joint with IBM), followed by an APL Germany day with the formal AGM and an assortment of speakers. I flew over with Richard Nabavi, which gave us enough time to kill at Heathrow that we very nearly had RainPro running on his APL.68000 box by the time the flight was called. Finding power outlets was a bit like finding carcasses in the desert – watch for circling vultures (otherwise known as KPMG consultants) and nip in the moment one unplugged and ran for the gate!

Then I had the adventure of finding the Novotel around the back of the Mercedes plant (Richard had reserved earlier and had a more central hotel), and just failed to make it through the door before a most splendid Rhine valley thunderstorm kicked off the evening's entertainment. Simon Garland met me in the bar, and fed me peanuts until I recovered enough to dry out.

The Meeting



We started with an excellent presentation from Simon Garland on K (the language) and Kdb (the hottest new database product in town).

Simon has sent us some excellent notes of his own, which follow this report, so I will just report some of the key points in his talk, and one or two of the more quotable quotes.

K was first seen at APL93 in Toronto, and Simon first tried it at Credit Suisse. Then it was locked away in UBS for a few years (including a 64-bit version – 10 years on we *may* finally see it). K is written in ANSI C so it can run anywhere. The

philosophy was simple – let the compiler-writers do the tricks. The K source is just 26 pages of C code (with lots of macro explosions) in the style of Roger Hui's J (see Vector 9.4 page 93 for some examples of this approach).

“Give it a PC and it will say ‘thank you, I’ll grab it ...’ – K has sharp elbows”

And the outcome (as Stefano commented recently) is that K is *insanely* fast when faced with data-heavy computations. So how does it do it?

What is different from APL?

Simon listed many of the key differences, and followers of A+ can see an interesting progression here as Arthur's thinking has moved forward:

- K has single glyphs (like APL) but uses plain ASCII (like J).
 - it runs from scripts, and has no saved workspace.
 - data is always mapped, rather than ‘loaded’ into memory. This is rather like having lots of private pagefiles.
 - functions can be called with an unlimited number of named arguments. *[This probably addresses the biggest single deficiency of APL and J as commercial programming tools – and improves on A+ which has a system limit of 9 named arguments. AS]*
 - iteration can be controlled with {eachleft} and {eachright} as well as the conventional pairwise {each} which requires both sides to match up.
 - there is an explicit *null* data type, which is very useful when working with databases.
 - K makes little use of booleans. There are some utilities for getting indices from temporary ‘monster’ bools.
-
- K has *lists*, not *matrices*. Essentially everything is one-dimensional (with no explicit *enclose*) but if your list is regular, then functions like {transpose} do work as you would expect.
 - K is very lightweight. The executable is about 5K (of which 4K is the icon) and the DLL is 150K. This allows applications to multi-thread very simply by launching new K tasks which can run synchronously or asynchronously (like Sharp S-tasks and N-tasks)
 - K has intrinsic support for Dictionaries (arrays of key-value pairs) which allow indexing by name, and can be organised into trees.

So where did Kdb come in?

Kdb was 'invented' sometime in 1998, more or less by accident. Everybody had written something like it via dictionaries, so it was getting silly. The message went out "Let's standardise and add some SQL stuff to search it".

Most customers simply use Kdb as a database, with K as the scripting language! This works superbly, as Kdb is written entirely in K (with no hidden stuff) so extending and customising the supplied tools is totally seamless. Kdb is (of course) column-oriented and is perfect for timeseries where you almost always want to process each column, rather than each record. And, of course, it is all built on mapped files, so even a huge table 'loads' really fast.

Some other features that an APL audience might relate to

- Kdb is aware of the sort order, so can (for example) do a binary search for names. It also holds timeseries dates as integers to simplify data-related selection.
- Joins are very elegant and fast, as we already have hard indices (pointers to the next table). You can do a 4-way join in a simple one-shot Select statement to chain down a series of reference tables (employee->dept->division->business unit for example).
- Kdb is very scalable. By default tables are folders and columns are files, but big columns can be chunked (by date or A-M, N-Z) and can be spread across machines (which may even be running different operating systems). The sort order can vary within a column, for example old data could be sorted and indexed where today's transactions are just held 'cast in order of appearance' to speed the logging process.

The K development experience

"Gui – either you write your own or you decide early on to forget it. K just decided to do without, so here we are on the DOS box ...".

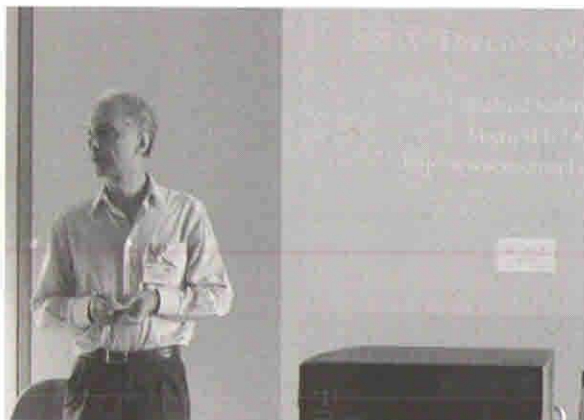
... OR, of course you write your own IDE, and plenty of people have done just that. Actually any editor which supports syntax colouring will do the job, and you can use a range of change-control tools as all your scripts are just simple text files. Charlie Skelton (c.skelton@skelton.de) has a very nice IDE which he can show us running a complex query on some 20M records (in around 0.2 sec).

For more information

Talk to *First Derivative* who are selling and supporting Kdb in Europe, watch the newsgroups at kx.com or mail me at simon@kx.com.

The Rest of the Meeting

Richard Nabavi followed with a demonstration of the latest incarnation of APL.68000 (now APLX) and showed one or two new capabilities from the beta of the next release.



Again, he has chosen the route of launching lightweight tasks to handle threading, and he showed how you can have multiple sessions open on the same workspace, which can synchronise under program control.

In fact, this seems a really nice usability feature for the developer. It allows you (for example) to keep

a few test lines sitting in session-a ready for repeated execution while you fool around in session-b getting the code to work. I have lost count of the number of times I have had to scroll miles back up my Dyalog or +Win session to grab my 'test piece' so I can re-execute it.

Adrian Smith did a pretty straight rerun of the talk on .Net and APLScript from Finland. As yet, no-one really cares much about this stuff, but they will get the message just as soon as the IT bosses issue the "if its not managed code, get it off my machines" decree. Ironical really, as APL applications have been managed code from APL\360 onwards.

Nancy Wheeler told us a lot of interesting stuff about 2 □TF and □ATR for sharing data between APLs or with external applications. Unfortunately, my notes run out about this point, and it is much too long since I used APL2 for me to make sense of what I have. I guess that any self-respecting APL2 shop already knows it all anyway, so I will take the safe option and leave it there. Sorry.

Summary

A well-organised meeting in an excellent venue. Thanks to APL Germany for the invitation and to IBM Ehningen for hosting the meeting.

Matrikx Reloaded

(all work and no k makes jack a dull boy)

by Simon Garland (simon@kx.com)

Although the array language K is immediately accessible to programmers familiar with APL or J, comparatively few have spent any time looking into it. For anyone who wants to add a new language to their toolkit this short article describes how to download and install a trial version – and points to further information for those who would like to learn more.

Installation:

Download and run a copy of `kdbsetup.exe` from the download page [2] of the Kx website [1], this 200K file will install the array language K and the database Kdb. It will install on any dialect of Windows, although NT or XP are more comfortable to use as the 95/98/ME command prompt is so clunky.

The trial version has restrictions on the amount of data it will work with, and the number of messages that can be sent between K sessions – but otherwise is a full version.

Try K:

Open a command window, and type

```
k
```

and you'll be in immediate execution mode in a K session, try a few familiar things:

```
C:\k>k
K 2.95t 2003-04-03 Copyright (C) 1993-2003 Kx Systems
Evaluation. Not for commercial use.
\ for help. \\ to exit.

2+2
4
2+2 3 4
4 5 6
+/2 3 4
9
```

to get out just type `\` in immediate execution mode.

K includes a built in reference, access it by typing a single `\` followed by a return in immediate execution, that will give you a list of pages available. Enter the particular one you want to see (`\+` to list the verbs for example).

If you type something complicated and get suspended you can clear the execution stack with a single `\` at the suspension prompt. Here a nonsense function `foo` is defined to add its argument to a string and cause an error:

```
foo:{"string"+x}
foo 17
type error
{"string"+x}
^
> \
```

If you just want to disable suspensions while looking around enter the command `\e 0` on a new line – the error will still be displayed but the stack will be cut back:

```
\e 0
foo 17
type error
Try Kdb:
```

Get out of your previous K session if you still have it open by entering `\` and then start a Kdb session by entering:

```
k db
```

this time you'll be in a kdb session as shown by the `ksql>` prompt:

```
C:\k>k db
K-2.95t-2003-04-03-Copyright-(C)-1993-2003-Kx-Systems
Evaluation. Not for commercial use.
\ for help. \ to exit.
```

```
KDB 2.5t 2003-06-25
ksql>
```

as Kdb is a database we'll load in a sample database to make things more interesting, at the `ksql>` prompt type in:

```
ksql>load 'trade.t'
ksql>
```

Note the single quotes, strings are delimited by double quotes in K, single quotes in Kdb – that makes it easier to embed KSQL strings in K code.

Now try some simple KSQL:

```
ksql>tables
`stock `trade
ksql>vars stock
`stock `industry
ksql>count stock
6
ksql>select from stock where industry='auto'
.((`stock
  `xx `yyyy
  .,(`K;1;))
  (`industry
   `auto `auto
  ))
ksql>count trade
100000
ksql>vars trade
`stock `price `amount `date
ksql>select avg price by stock from trade
.((`stock
  `aaa `bbb `ccc `dd `xx `yyyy
  .((`T
    ,`stock
    )
    (`K;1;)))
  (`price
   74.93937 74.93437 74.89528 74.95792 74.73562 74.71044
  ))
ksql>
```

Notice that the table columns are just vectors: At last – a database that you can work with not against/despite!

You can open a simple viewer on the database by typing in:

```
ksql>show ''
```

or you can view the results of a query by prefixing a show to the constraint:

```
ksql>show select avg price by stock from trade
```

you can escape from Kdb to K by entering a single \ at the ksql> prompt

```
ksql>\
  2+2 3 4
4 5 6
```

and you return from K to Kdb by entering `.d.1``

```
ksql>\

count stock
value error
count
^
parse error
.d.1`
ksql>count stock
6
ksql>
```

exit from a Kdb session with `\\` as you would from a K session.

Summaries of KSQL, and the Kdb SQL92 implementation can be found in the two files `s.txt` (SQL92) and `t.txt` (KSQL) in the directory `c:\k\db` created when you installed kdb earlier.

Documentation:

If the 5-10 minutes it took to try this out has caught your interest the best way to learn the basics of K and Kdb is to read through the Kdb manual 'Getting started with Kdb' available from the Documentation page [3] on the Kx website. Although its focus is on Kdb it takes you through the basics of K as well.

Apart from the other manuals on the Documentation page [3] there is a K reference card available from Mike Powell's directory on the User Contributions page [4].

When learning a new language it's often helpful to have other people's code to read, there is a large library of K and Kdb code on the Kx site in Arthur Whitney's directories [7].

More general articles about K and Kdb are indexed on Eberhard Lutz's Kaylang page [8]

More Information and Help:

If you have questions about K or Kdb the Kxforums [5] are open to anyone, or the K and Kdb listboxes can be searched from the archive page [6].

Links:

- [1] <http://kx.com> – home of k and kdb
- [2] <http://www.kx.com/download/download.htm> – download k and kdb
- [3] <http://www.kx.com/download/documentation.htm> – manuals
- [4] <http://www.kx.com/download/contributors.htm> – user contributions
- [5] <http://kxforums.com/kxf/> – user forums
- [6] <http://www.kx.com/listbox/index.html> – k and kdb listbox archives
- [7] <http://kx.com/a/> – sample code
- [8] <http://www.e713.com/kaylang.html> – extensive list of k articles

Kx Systems Celebrates 10 Years and Introduces 64-bit Kdb+ at SIA Conference

meeting notes from Steph Johnson

The single largest annual event for IT executives in the securities industry, the SIA Technology Management Conference & Exhibit takes place in New York City each June and draws an international audience. Kx is traditionally a sponsor of and exhibitor at the event. This year the conference coincided with the announcement of the company's new 64-bit kdb+ database, which Kx demonstrated on two networked Itanium 2 servers running Linux on the show floor.

Kdb+ has been in beta test at several leading firms including Lehman Brothers. The new database is more than simply a version of kdb for 64-bit architectures, although the need for trading, risk management and other real-time business applications to capture and analyze more transactions and events in memory was the impetus behind its development.

Arthur Whitney, developer of A+, K and all Kx Systems software, designed kdb+ to handle more data types than kdb and to provide data types that map to Java and .net. In addition, kdb+ is several orders of magnitude faster than kdb on operations such as inserts and updates, and it offers greatly reduced latency to speed up transaction processing. Kdb+ also merges the K language with ksql to provide a single language for programming and analytics.

Kdb+ is not a replacement for kdb, which continues to receive growing acceptance in financial and other industries. Kx will continue to develop and sell both platforms. According to Whitney, "After 10 years of experience with customers using K and kdb, we learned a lot, and it was time to incorporate that learning into a next-generation product."

Initially, kdb+ is expected to appeal to firms whose intraday data requirements exceed the capacity of 32-bit architectures. "Many of our customers want to deal with 50 million or more transactions per day," says Whitney. "64-bit makes this easy."

Kx customers are seeing trade data volumes that nearly double each year, and Whitney expects that the financial industry will be one of the first movers to 64-bit technology.

10-year celebration (July 8, 2003)

Founded in 1993, Kx celebrated its first 10 years with a party at mid-town Manhattan's stylish restaurant Noche ("night" in Spanish). The restaurant's Latin theme was complemented by the live music of a 4-piece Brazilian band, and nearly 150 customers, partners, prospects and friends of Kx attended.

Toward the end of the evening, the party took a surprise twist, as employees of Kx and its strategic partner First Derivatives presented Whitney and CEO Janet Lustgarten with a commemorative Steuben glass sculpture. The plaque read: Kx Systems – 10 years of success – 2003. Several long-time customers and friends, including Joel Kaplan, who first hired Whitney at Morgan Stanley, were on hand to express their congratulations.



Michael O'Neill of First Derivatives (centre) acknowledges Arthur Whitney and Janet Lustgarten at the Kx Systems party in New York

For Whitney, the party and SIA show helped him officially mark the completion of kdb+, which has been in development for several years. "Usually I am too busy programming or thinking about how to program something to pay much attention to product announcements. But this time, being surrounded by so many people who were celebrating the announcement and the company, I had to stop and appreciate what Kx has become over the years."

FCRC Conference San Diego 2003: Stretching the Mind

report by Ian Clark

FCRC 2003 was a combined conference of 30 sub-conferences or workshops under the umbrella of the Association for Computing Machinery (ACM). Of these SIGAPL was just one, with some 50 delegates out of a total of 2,400. Different SIGs had their conferences on different spans of dates between June 7-14, which ensured that if you turned up for lunch outside your appointed span you got turned away.



The event took place in a hotel large enough to accommodate all the delegates, the Town and Country Hotel and Conference Center in Mission Valley, San Diego, California. The centre consists of a compound with a number of buildings both short and tall, set among rose gardens, swimming pools and gazebos with acres of chairs and tables both outside and under cover. The ubiquitous laptop users lined the corridors and the walls of the Grand Foyer, wherever they could find a wall socket to plug into. These weren't in ample supply, but you could usually find one if you wanted one, and maybe even a chair and a table handy, though it helped to

be able to sit cross-legged on the floor. Buddha contemplating his laptop is all set to become an icon of the age.

Delegates were offered the chance to book rooms at the hotel itself, if they took it early enough. Your intrepid correspondent booked a berth in a backpackers' hostel he discovered in downtown San Diego, the Gaslight District (which wasn't quite so Jack-the-Ripper as it sounded). This ensured a vigorous nightlife and youthful company, very appealing to the young-at-heart and earplug-equipped. He then had the problem of commuting into the conference location, which turned out to be easier than it had originally looked from the non-existent pre-conference joining instructions – a twenty minute trip on the San Diego Trolley, the light railway which serves the town. Interestingly the end of the line is in Tijuana, Mexico, which lets people commute to work across the border – you don't have to swim the Rio any more. The FCRC organisers had thoughtfully provided a free trolley pass in the delegates' joining pack, which they then tried to retrieve for a refund from delegates not using them.



The combined conference permitted an overall programme, so that delegates got a timeline of all the activities in progress. In principle this allowed one to attend any presentation in the 12 or so concurrent streams on any one day. The greatest benefit of this was the three plenary sessions of talks of general interest by invited speakers, all leaders in their fields, which were well worthwhile.

Highlights of the conference for this particular delegate were:

Barbara Liskov, MIT: Issues in peer-to-peer computing

This plenary talk introduced us to the vocabulary and methods of peer-to-peer (P2P) communications. A P2P network differs from conventional client-server in that it consists of a group of nodes starting off undistinguished from each other, which nodes can join and leave at will. P2P has grown in importance with informal networks for sharing music and suchlike. Prof Liskov's group is refining the initial empirical solutions of the industry to make them more suitable for corporate networks. This is speculative work insofar as there is no guarantee whether in the end the result will be better than the traditional client-server architecture.

We learned about rings of ids, successor functions and slice leaders, Byzantine failure (where a node essentially lies) – the more important source of malfunction thanks to the growth of malicious attacks – and fault-tolerant strategies to overcome it. We also learned about the Sybil attack, which 'stacks' a network by creating sufficient new nodes to create a quorum. This demands a configuration service, a subset of the network forming a Byzantine quorum, to validate the network periodically. Existing applications of P2P, e.g. music sharing, assume uncontrolled entry and exit of nodes from the network, whereas an admissions policy is clearly more appropriate for a corporate intranet, where the turnover of nodes is hopefully lower.

This was a clear and economical presentation, which got into the subject with very little flim-flam.

Dan Baronet: NavPlanR – flight planning software for pilots

An entertaining talk that started off explaining the basics of aircraft navigation from "Piloting for Dummies." Dan came to this problem from his own experience as a light aircraft pilot. The program prints out a flight plan, reading weather from a particular website. It was offered as a practical example for getting started with the J Project Manager, including packaging up an app for distribution and installation, right down to drawing the explorer icon, printing the label and locking the script. A feature was a test script packaged with the project. Dan gave away the code on diskette, which can also be downloaded from: www.milinta.com

**Tama Traberman, Linda Alvord:
Simulating ordered RGB color triples using J**

A literally colourful talk split between the two presenters with a little movie at the end. There were coloured visuals and handouts to explain RGB graphics, as well as artwork demonstrating the practical results of the speakers' methods of sorting the colour palette by both hue and intensity. The effect of this was to achieve more convincing colour gradients in artwork. The technique produced most impressive results when a colour image was converted to greyscale, the results of doing so rarely being appealing in general.

Georg Reichard: INSTRUCTOR – a new technique of guiding the user in simulation software

This talk described a courseware building package used primarily with pre-existing energy-use simulations. Such simulations typically have a number of parameters whose effective usage is far from clear, leaving the student with the dual task of learning how to drive the simulation, as well as absorbing the lesson it is supposed to convey. The speaker showed how to eliminate the first task. A significant application of Causeway, INSTRUCTOR offers the tutor an orderly way of building instruction sessions using a given simulation, acting at runtime as a non-programmer top-end to the simulation packages.

Bernecky & Wooster: BITSlice: Representation without taxation

Roy Sykes presented this paper on behalf of the authors, who were faced with the task of trying to speed up a mainframe application which was too well-written to offer easy gains by straightforward recoding. However the interpreter wastes a lot of machine cycles slicing and dicing Boolean strings. Addressing this issue achieved performance gains of 40%. By way of a general answer to this sort of problem the authors wrote a Sharp APL intrinsic function, comparable with APL2000's `⊞OR`, which effectively coerces a datatype without rewriting the actual data representation in memory.

Roy Sykes: Faster than a speeding bullet, performance enhancements for APL using Assembler

Roy described his MACfns, replacements for primitives and idioms written in Intel Assembler. The machine code is coerced to integer datatype and serves as the right argument to `⊞CALL`. Roy described the special advantages this gives him over the interpreter's own strategy. One of these less obvious advantages is to get consistent error messages, e.g. in cases where there are potentially two or three

reasons to fail. Roy regaled the audience with the side-effects of very small numbers in 64bit versus 80bit IEEE floating point standards and the speed improvement of Log2 based on bit-shifting rather than calculating the log.

Vendor Forums

Nearly all the major interpreter vendors were there displaying their wares, *viz.* IBM with APL2, James Brown with SmartArrays, John Daintree of Dyalog Limited (formerly Dyadic Systems) with Dyalog APL and PocketAPL, Richard Nabavi of MicroAPL with APLX 1.1 and Fred Waid of APL2000 with Windows Server 2003.

Conclusion

For the APL delegates, the conference climaxed with the banquet on Friday night, a Mexican buffet at which the wine flowed free but the beer was controlled by ticket. Roy Sykes filled the gap left by the pre-arranged after-dinner speaker who was prevented from attending for health reasons. This degenerated into pandemonium as two teams of volunteers holding numbers representing scalar elements tried to ravel and transpose in parade-ground fashion, convincing your correspondent that APL is not yet ready for war. The assembly then adjourned to a guest room somewhere in the hotel for an evening seminar (not quite the same without the Russians).

Clearly this had been an easy, low-cost conference to organise, the bulk of the work being handled competently by the hotel and the FCRC organisers on behalf of all the participating groups. Stephen Mansour succinctly performed the role of Conference Chair, the program chair seeing fit not to attend.

The content had a somewhat lightweight feel about it, there being 14 papers in the APL proceedings, actually available at the conference, of which 11 were actually presented. But these were eked out by 5 vendor presentations, 3 tutorials and 2 invited presentations, plus the opportunity to attend 3 entertaining plenary sessions. Apart from these, the much-vaunted opportunity to attend other groups' sessions did not appeal overmuch, because they were indeed 'special interest groups'. However your scribe did get to show APL in the concourse poster session to a young student who had never heard of it, plus ample opportunity to remind himself how awful root beer really was.

In conclusion, a good time was had by all, with plenty of opportunity for networking.

GENERAL ARTICLES

Why APL Programmers Don't Use Libraries

by Morten Kromberg (mkromberg@insight.dk)

*The next best thing
to knowing something is
knowing where to find it.*

Samuel Johnson

As Stefano Lanzavecchia announced in Vol.19 No.4 (aka "April 2003"), the BAA is starting a Library Project, and I have volunteered to be the Librarian. The goal is to create a repository for freely available code, decently documented – supporting as many APL (and A+, J, K) interpreters as possible.

Here We Go Again

"Here we go again", I hear the old APL dogs groan to themselves. I was tempted to join in the chorus myself. We've been there, done that and – as everyone knows – you can't get APL programmers to use libraries (or, according to popular myth, adhere to standards of any kind). There is no doubt that – if you compare APL developers to most other programming communities – one of the ways in which we are different is in our use of libraries. For the most part, we don't – and in many situations, there are good reasons for this. My task as the librarian – as I see it – is to identify the most important reasons why we make less use of libraries, and – to the extent that it is profitable to us – help develop an APL library culture. I have accepted the challenge because I am convinced that unless we do, APL cannot find its place in the programming community, and it can only continue its current slide into anonymity and finally extinction.

-----The two main reasons why I am convinced of this are:-----

1. There are currently almost no APL training programmes in existence. This may not be quite as bad as it sounds, because – as fast as things are developing in this day and age – self-study accounts for an increasing fraction of IT training. But without well-documented libraries, you can't even teach yourself APL. Occasionally, APL, J and K get some sort of mention in relevant media, for example when Dyadic joined the DotNet programme or launched APL for the PocketPC – or when APL2000 was acquired by Cognos. Any interested young programmers who tried to dig a bit deeper and use the web to find out whether APL was fun almost certainly gave up within about 15 minutes.

2. The proportion of any application dealing with interfaces to other applications, using various standards, is growing continuously. Conforming to these interfaces without using libraries is ever more expensive. Knowledge about what interfaces are available is critical. This is not just a question of providing interface components in APL, but educating APL developers about how to tap in to the wealth of libraries which are available to other programmers. Modern APL environments actually allow you to do this quite easily, but we tend to be too ignorant to benefit. Paraphrasing Samuel Johnson, most of us don't even know "where to find it".

Now is the Time

I know that there are active APL developers with sufficient exposure to other environments to understand the importance of these issues and to help do something about it, but somehow the APL establishment has been unable to recruit these "youngsters" (currently mostly in their 30's, 40's – and I guess we'd better include a handful of youngsters older than that) into active service. Many of these people have been too busy making a living writing code to take much notice of the – as they perceived it – rather academic APL community. The community has had less and less to offer the professional APL developer since the dispersal of the large, professional APL development groups at IPSA, STSC, IBM and others when timesharing came to an end together with the 1980's – at the same time as the language split into two or three significantly different dialects.

It is my hope that a handful of APL developers who are still young in mind will be able to step forward at this time and make a contribution towards the continued profitability of the use of the APL (family of) language(s). APL, J and K have a lot to offer end users in terms of enabling usability and flexibility which they might otherwise never experience. Not only that: I totally agree with Stephen Taylor when he says in his article "A Place for Us" – also in Vol.19 No.4 – that my productivity in APL commands a premium for which I cannot charge in other languages. We need to make an extra effort at this time – while we still have the energy – to ensure that this favourable state of affairs is maintained.

So Why Don't We...

... use libraries? There are probably a large number of factors which have conspired to cause this. Many of the strengths of APL turn out to have a dark side with respect to libraries (and we must remember that in many situations, this remains a strength: having to learn to use a huge library is not necessarily a quick way to get things done).

Once Upon a Time, APL was WAY Ahead

When the art of APL application building was developed, in the late 70's and early 80's, the interactive APL environment was unique in its productivity and connectivity tools. While BASIC and ALGOL programmers needed libraries in order to do "trivial" things like sort a table or invert a matrix, we had these facilities built in to the language. This was not only true at the algorithmic level – many commercial APL systems had significantly better interfaces to operating system resources than competing environments such as FORTRAN and COBOL.

Using the Auxiliary Processors which were provided, any APL programmer could throw together a set of utility functions to interface to Sequential Files, TSO Commands, VSAM and DB2 in ways which fitted his or her personal taste or application requirements. People who had to do the same things from other environments had to get very well organised, help each other, and learn to accept other people's tastes. When C came along, it was primarily used for interfacing and writing operating system components, so the art of building and using libraries was developed very early on. You could not really be productive at all in the other languages without libraries.

APL's Auxiliary Processors *were* the equivalent of libraries. At I.P.Sharp Associates, there were also APL-coded services for printing, graphing and so on, accessed in the same way as the APs, using shared variables. However, these "libraries" ran in a different address space and never made classical library builders of us.

APL's primitives for getting user input included Quad, which would allow the user to execute APL statements to generate input! This quickly fell out of favour with serious application developers, but provided functionality which the FORTRAN guys would have given a limb for.

Managing user interfaces for large applications finally introduced complexity which prompted some of the first serious library-building in APL. Even before AP124 and GDDM appeared, some of us were working on sophisticated "libraries" to collect user input to multiple prompted lines of input, reacting to keywords like BACK, QUIT, HELP which allowed the user to navigate around TTY mode applications.

Compilers Encourage Modularity

Because of compile times on early machines (which often ran into several minutes even for small applications), if for no other reason, the user of a compiled language quickly became accustomed to compiling small pieces of his application and linking them together, only working on a small part of his application at a

time. While working on one piece, the rest of the application was static and unchangeable. The development environment was geared towards this mode of development, which encouraged clear division of the application into parts with interfaces which changed as little as possible.

A culture developed, with libraries, linking mechanisms, catalogues, publications and later web pages discussing and documenting what was available.

APL users saw no reason to adapt this cumbersome and restrictive mode of operation. The ability to perform rapid prototyping of all components simultaneously gave (and still gives) the APL developer a huge performance advantage.

APLers Focus on "The Problem"

One of the most important reasons why APL is so productive is that it allows people who have great ideas but are not IT professionals to get deeply involved in the software development process, sometimes even writing entire applications themselves. Not only do these people have even less understanding of professional software development techniques and tools, including libraries – they care even less than many APL developers about standards and interfaces. The important thing is calculating the value of that portfolio or price of that insurance premium – not whether we can produce a nice data entry form for the parameters, adhere to good (or at least common) GUI principles, or whether we can save the data in an SQL database or as XML.

The following story illustrates the kind of thing that probably still happens in APL teams:

I had the privilege of working with George Kunzle, who invented a internal budgeting and planning application called FREGI at IBM, which became the Kunzle Planning System, renamed Adaytum Planning and remains the cornerstone of Cognos Management Series Planning Analyst™. George hated (and probably still hates, although I believe he has learned to live with) Windows. He had developed a highly usable user interface in DOS, which made extensive use of colour to indicate whether numbers were input, calculated, changed (etc.) in his data entry screen. The white Windows background reduced the visual clues dramatically, and made data entry much slower. The "non-standard" DOS interface was superior in most respects, and George was quite right (from a functional point of view) to resist Windows, as it added no value for his users in the context of his application.

Finally, with Michael Gould and Ellis Morgan, George developed a Windows Interface in Dyalog APL version 6, the very first Windows APL, which had extremely rudimentary GUI objects available. With George's attitude to

usability, the only acceptable approach was to build the thing from the ground up using PolyLine and Text to implement everything. The Windows Reporter component, with its decidedly non-standard user interface, was a superhuman feat of engineering, and the last thing I heard, it was alive and kicking at Cognos, still resisting all efforts to replace it using modern technology.

At the end of the story, I am pleased to report, George was a wealthy man, but still had (most of) his principles intact.

APL developers are sometimes called uncompromising mavericks, who would rather rewrite something than try to understand what others have done. The truth is that much of what is done in APL is (or started out as) work on the leading edge. APL was successful because it was the only tool which could tread fast enough to avoid sinking where there were no libraries.

To take a positive view on this, our focus on the "real" issues contributes enormously to the value of software developed in APL, but has led us away from becoming good library users.

"Throw Away Code"

Some people claim that APL attracts anti-social individuals who take pleasure in going their own way and have a superiority complex about APL. If this once was true, I doubt that many such individuals are still making a living with APL, so I think we can safely ignore the problem at this point in time ☺.

However, it is true that the phrase "Throw Away Code" was coined, to describe the situation which would often occur during prototyping, where one or more lines of code was hacked together, often in a very experimental fashion, until the result had the desired shape and content. Since the entire process may only have taken minutes, APL developers would be content to leave it in the application, prepared to throw it away and rewrite it completely if input data or other constraints changed, rather than take the time to restructure the final code into something comprehensible, document it and perhaps isolate it in a utility function and save it in a library.

Because of the speed of development, the effort of maintaining documentation and remembering the content of libraries was not considered worthwhile – and this is clearly another reason why a library culture failed to evolve in the APL community.

However, I hasten to add that this is NOT a good reason not to use this approach – it may well be the best solution, depending on what you are trying to achieve –

but some programmers are undoubtedly a bit too quick to reject the idea that a clean version of the code might be useful soon.

Source Code Management Systems

The early APL workspace, much like a spreadsheet, was a very convenient sandbox in which to play. Unlike compiled environments, which had linkers and loaders, it provided absolutely no mechanisms for differentiating between the new application work you were doing, and any libraries you might be using. You would)COPY tools and utilities into the workspace, and they would become part of the big mess. Naming conventions were invented to solve the problem: one type of objects had underscored names, or were prefixed by delta, delta-underscore, the other not, thr chr nms for utilities, – and later uppercase *vs.* lowercase – but this was only OK until you met someone at a conference who gave you a diskette with a utility library, and you discovered that his company had the reverse convention (let's leave index origin out of the discussion at this point, in order to maintain a positive frame of mind).

Workspace size limits, multi-developer projects, and eventually corporate info centres based on a core of common code, forced some teams to develop source code management systems outside the workspace. These were either used to build workspaces which were then saved, or – especially in SHARP APL which usually ran with smaller workspaces – dynamically loaded packages of functions and data at runtime. Some of the more sophisticated dynamic loaders went as far as creating a shell function which localised all the dynamically loaded code, so that it was isolated from the environment and disappeared when the selected applet terminated.

Unfortunately, while these source code management systems often prompted the development of a good local library culture at each individual corporate site, and in many application development shops, they were home grown and all quite different – and could not help establish a global library culture. The APL vendors never really tried to lead the way. (OK, they did give us)PCOPY, so we could avoid accidentally copying over objects in the workspace ☺). IBM did release an enhanced version of the FSDESIGN application for 3270 panel design which had some source code management facilities, I believe it was named APE – but it never achieved widespread use. I.P.Sharp marketed a source code management system called LOGOS, and developed but never marketed an application environment called SHAPE. Before the vendors managed to get into the development environment business, they ran out of money, downsized, and much of the steam went out of the community. The introduction of the PC also contributed to making it harder to keep a community together.

How Close Did We Come?

As mentioned above, many large APL shops do have – or once had – a library culture. The source code management system contained utility libraries – often versioned – and a developer could usually be located who was responsible for a library and could explain how it worked, even if the documentation was often scanty.

At I.P.Sharp Associates, back in the early 1980's, I remember learning most of what I know about writing APL code by poking around the "public libraries" with numbers below 1000, reading the code and talking to the developers. Many of the libraries had discussion groups in the mailbox system, and it was always possible to get good advice from people who had written or used the code in question.

Back then, I actually felt that there was a strong community which provided useful libraries and helped me learn.

What Has Changed?

Unfortunately, I believe that one of the most important problems may be hard to deal with, namely, that the APL business (like many others) is nowhere near as lucrative as it was in the timesharing days. Back then, IPSA, STSC and IBM were making a bundle selling timesharing or hardware. Once the APL applications were built, they generated a steady monthly revenue stream and APL developers had plenty of time (compared to today), to sit back and think about how to do things faster. Most of the money was made by selling storage, CPU upgrades and new channel controllers, CPU cycles, connect time and KILOCHARS on the packet-switched network. Consulting was often given away or heavily subsidised, and poor programmer productivity clearly limited profitability. Today, a lot of APL developers make a living working in one-man companies selling consulting – and keeping very busy educating themselves (or driving a bus) in-between jobs. There is still a good deal of money being made by APL-based businesses, but due to fragmentation it is much less clear how "community work" can enhance productivity.

However, for the reasons I mentioned at the start of this article, I believe we are reaching a point where the use of libraries for interfacing is becoming critical, and the lack of new APL blood is seriously limiting the ability (or willingness) of many organisations to use APL to solve problems. We need to act now!

Target Audience for the Library

I believe that the library project has three quite different target audiences:

- **People who are learning (about) APL.** If it contains well-documented examples of good APL code, the library will act as a natural extension to training materials.
- **Developers starting new projects.** Our library should be of a quality which makes it a natural choice to replace the CLEAR WS as the starting point for new APL projects.
- **Developers with existing applications.** We must provide interfaces to new protocols, storage mechanisms and data formats as they are taken into use.

Benefits

If you are learning APL, the library will help you learn to write good APL. The libraries will make you productive in a shorter time, and you will be more likely to succeed in learning APL at all. As we all know, this will help you deliver more flexible solutions fast.

If you are starting a new APL-based project, the library will help you get started more quickly. As the library routines become known to many APL developers, it will be easier to move people to new projects, since there is more common knowledge. If we succeed in increasing the number of APL developers, the use of APL becomes safer, more politically correct and more cost-effective.

However, I would maintain the most important reason why it is now critical that the APL community develop some sort of library culture, is the interface requirements which are forced upon modern commercial applications. Although some large APL organizations can afford the cost at this point in time, it is increasing. I believe that, in order to ensure our long term survival, the APL community must learn to share the cost of development of code to support industry standard interfaces and data formats – or simply the cost of acquiring knowledge about the interfaces developed by users of other languages, and how to use them effectively from APL.

Content

To succeed, we must be pragmatic and set realistic goals. APL is most profitably employed to do rapid prototyping or to develop systems (or components) which must mutate rapidly to adapt to new market conditions. We will never use libraries in the same way as a C++ developer. I propose that we should set the

following short-term goals for our APL¹ library project, more or less corresponding to the three target groups:

- **Education:** Provide an introduction to newcomers to APL. Help teach good APL coding practices, in connection with literature (on the web). The production of training materials as such is beyond the scope of the Library project, but we must ensure that our efforts to meet other goals are co-ordinated with the production of new materials.
- **Utilities/Idioms:** Provide standard and high quality (correct, robust, fast) versions of commonly used algorithms, from idioms to slightly more complex code (row to row iota, etc.). These can be written in APL or implemented using external calls to C or other tools.
- **Interfaces:** Significantly reduce the cost of writing APL applications by providing interfaces to (external libraries which provide interfaces to) the most common facilities such as databases, messaging systems, XML.

If we achieve the above, we can consider extending our goals to include the following:

- **Community:** Help form the backbone of a global community, and hopefully re-ignite the exchange of knowledge between users of array-oriented languages.
- **Foster Component Building:** In the future, I believe that the natural place for APL is as a component building tool. We need to educate the APL community about the use of libraries in other environments, in order that we can become purveyors of quality components to the rest of the development community.
- **Code Management:** Efficient use of library routines requires some standards for "linking and loading". It would be nice if we could replace or extend the myriad of homegrown source code management systems with a universal standard. But I did say we should be pragmatic and realistic, and this may be more than we can manage.

Education

The BAA already has plans for a project to produce a modernised set of APL education materials to be made available via the internet. I propose that the Library project support this work by organising any code examples referenced in the training materials, and providing a collection of Utilities, Idioms and interface

¹ I would like to include A+, J, K and other similar languages in the project, but we may need to limit our level of ambition initially, and focus on what I would call the APL2 group of languages.

code which are documented in such a way that they provide a natural extension to the actual training materials.

Utilities and Idioms

Although we questioned whether APL has a library culture, I actually believe that every APL programmer understands what we are talking about. Every application workspace contains a base of small functions which are used dozens or hundreds of times to perform very common operations for which APL contains no primitives. Over time, many of these functions have been promoted to primitives, or at least interpreter-optimised idioms. Our goal is to select the best versions of these utilities, clean them up, document them and provide complete groups of code, identical across different APL interpreters where possible.

Where external tools exist which render APL-code obsolete, we will build interfaces to such external computational libraries.

The following list contains a few examples of utility functions which immediately come to mind, and is obviously not exhaustive:

Simple Data Manipulation

- String manipulation (justification, case conversion)
- Searching and lookup
- Transformation between lists, matrices etc.
- Formatting data

More Advanced Computational Tools

- Dates and Time
- Matrices; Inversion, Eigen Values, FFT, Statistics
- Complex numbers

Coding tools

- Timing/profiling
- Workspace searching/comparison
- Calling tree analysis, name cross references
- Array display, and editing
- Tools for moving code and data between APL systems

Interfaces

This part of the library will contain collections of code which are potentially more complex, and provide access to standard data formats or external resources which any modern application is going to be expected to support. While the Utility and Idiom part of the library is likely to remain relatively static, the interface library will be continuously evolving as new external resources become available. As such, it provides the greatest potential for payback to large development teams. Such teams are unlikely to replace existing utilities by a new set – the testing implications alone are enough to make this difficult. If our library can provide interfaces to new useful resources in a timely fashion, then even the largest and most professional teams can profit from shared development – at least as a source of research into uses of new technology as seen with an APL mindset.

Some interfaces will provide access to external computation libraries, to do things like statistical analysis, optimisation, linear programming – in cases where external libraries provide results faster, more accurately (or simply more “officially”), or (perish the thought!) with more flexibility than we care to write ourselves.

Others will provide access to operating system resources, such as a library to list, create, delete files and directories. We would hope to provide interfaces to database drivers, and to data formats like RTF, XML, bitmap files, Postscript, spreadsheets and other document formats.

We will provide a collection of GUI utilities to perform frequent operations like prompting for input in various ways (single/multiple selection), providing progress bars, file boxes, etc.

The library will contain code which implements client and server support for protocols like HTTP, FTP and SMTP.

Contribute!

Before we ask anyone to write new code for the library, we would like to collect, select, “clean” (if necessary) and document the best code which is already in use.

Whether you are a one-man (or woman) consulting team, a member of a large development team, or work for a vendor of APL systems, please discuss the proposed library project with your colleagues and management. If you believe that you have code which might be relevant to any section of the library, and your company is willing to donate it, please submit it to us. If you have input which

will help us understand what would make the library more useful to your organisation, please let us know.

Although clean, readable, well-documented code is preferred, all contributions are welcome, even if you believe that your code is likely to be used to document just how dire the situation is out there ☺. We are also very interested in descriptions of how source code is managed in contemporary APL development teams – in particular how utilities and interfaces are handled.

Once we have produced the first few sections in order to document how the library might work (and experiment a bit with delivery mechanisms), we will be looking for volunteers to take responsibility for sections of the library. Of course, you are welcome to volunteer now, if there is a particular section that you know you are the right person to manage!

I look forward to the day where APL library contributions outnumber the spam in my in-tray.

Morten Kromberg
Librarian

dfns@dyalog.com

John Scholes, Dyalog Ltd (john@dyalog.com)

"They all come from good families, but over the years they kind of individually developed some ways about 'em that just ain't right." – Tom Waits.

The Dfn-ers

The **dfn-ers** are a group of people who like to discuss programming in D, a functional subset of Dyalog APL [1, 2]. The group is hosted by Conrad Hoesle-Kienzlen's *Alisia* list server at dfns@dyalog.com. If you register as a subscriber and send mail to this address, it is reflected to all of the other subscribers. Anyone can join the group, which currently has around 90 members. It helps if you have Dyalog V9 or better, to run the code fragments that people post, but the discussions are often entertaining in their own right. Subscription is free.

Here is a selection from the last 100 headers:

- 893 Calling into CHMs
- 895 Spanned cells (idiom challenge!)
- 909 Spring Cleaning
- 916 apl and oop
- 923 dsamples updates
- 925 The Beyond
- 928 Please Ignore ...
- 930 77 keystrokes just for fun
- 931 Cows and Bulls
- 933 [JEA lookalike for APL2 compilation
- 934 itching eaches in easter version now removed
- 935 Yet more left to right
- 938 Array oriented roman numbers
- 939 Size of dfns
- 940 Roman beauty
- 955 nifty d-op
- 960 Legitimate children
- 961 OOP in D
- 967 I am puzzled - static scope rules
- 973 Communicating through the code
- 977 undocumented feature?
- 978 More on Shy dnfs
- 979 Rounding operator
- 988 Losing the result

The group seems to have developed a culture of its own, including an informal (and imprecise) ASCII transliteration scheme:

```
⌈⌊<-1 2{a w}"<iota>3      A ⌈+1 2{α w}"⌋3
```

We have experimented with embedded HTML to display APL characters directly in the body of the email text but on the whole, we didn't like it; I forget why.

Contributions are sporadic. Sometimes a week will pass with nothing and then there will be a deluge.

In the background, people's offerings are collated and collected into utility workspace: `dfns.dws` [3] This is a large and growing repository of good ideas contributed by folk from all over the planet. Each function and operator in the workspace has a corresponding description in the `notes` namespace. Between us we have so far produced around 2,000 lines of code and 10,000 lines of notes.

As an example, a year or so ago, the group became obsessed with writing variations of the seemingly trivial function:

```
      3 to 7
3 4 5 6 7
```

As well as the function itself, these musings finally produced the following `notes.to`:

```
nvec ← num #.to num                                A Sequence α .. ω
```

Returns a vector of numbers α to ω inclusive. α is the *first* value in the sequence, and ω the *last*. More generally, α may be a 2-item vector that determines the first two items of the sequence and hence the *step* size.

Notice that any of *first*, *step* and *last* may be positive, negative and/or fractional. If *last* is not a whole number of *steps* from *first*, the sequence will stop short of *last*.

Technical notes:

The following simple function suffices for the ascending sequence $\alpha, \alpha+1, \dots, \omega$. Notice how $(\cdot - \square IO)$ makes it independent of index origin:

```
to ← {(α - ⌊IO) + 1 + ω - α}
```

Useful refinements to the function allow *descending* sequences and optional *step* sizes other than 1:

```
to+{[IO+0                                a Sequence a .. w
    from step+1 ^1x~\2+a,a~xw-a          a step default is +/- 1.
    from+step~1+0[(w-from)÷step+step=0 a a thru w inclusive.
}
```

Attempts to re-cast the function as a one-liner, include:

```
to+{From+0,+Step~0[(Inc+Step+--/2pa,From~xInc+w-From+1pa)
to+{+/w{w~{w-[IO]~1+0[(a~a)÷w+w=0}\1 ^1x~\2+a,a~xw-a}
to+{(0pa)+(|-/02+a,a+1){a~(x~w)~(-[IO]+1+1|w÷a}w-0pa}
```

VMJ suggests the following extension, which accepts an optional explicit *step* size as a second item of its *right* argument. *step* defers to *next* if both are supplied.

```
to+{
  [IO+0 ~ a+1 ~ z+0.0000000001
  (a n)+2+a,a ~ (b s)+2+w,1
  d+|s{w~0:w ~ a~0:a ~ 1}n-a
  {w~z<|0-w}a+d{(a~x~w)~1+1|w÷a}b-a
}
```

```
1 3 to 5      a using: <next>
1 3 5
1 to 5 2      a using: <step>
1 3 5
```

Examples:

```
3 to 10      a Inclusive ascending sequence.
3 4 5 6 7 8 9 10
10 to 3      a Descending sequence.
10 9 8 7 6 5 4 3
7 to 7      a Single-item sequence.
7
5 7 to 13    a 2-item start determines step.
5 7 9 11 13
~10 ~15 to ~25 a Negative start and step.
~10 ~15 ~20 ~25
1.5 1.7 to 2.5 a Fractional start and step.
1.5 1.7 1.9 2.1 2.3 2.5
0 3 to 10    a Sequence stops short of <last>.
0 3 6 9
```

See also: sieve

At one point during this episode, I found myself trying to explain to my wife why an international team of rather clever people were collaborating to write a computer program to count from one to ten.

I didn't get very far.

Another time, we collectively attempted to write a function *isdfn* which, given the name of a function, tries to determine if it is a D-function as opposed to a traditional *trad-fn*. This went on for quite some time, during which we lost at least one subscriber because they couldn't stand one more *isdfn*, D-fn.

Apart from these trivial examples, many of the functions and operators in the *dfns* workspace are both substantial and useful. Here is the current top level index:

Structures.

- Arrays.
- Trees
- Graphs (wGraphs)
- Words
- Strings.
- Line_vectors
- Blank_removal
- Data_compression
- Power.operators
- Function_arrays
- Syntax.operators
- Expressions.

Numbers.

- Whole.numbers
- Real.numbers
- Dates

Workspace.administration

- Workspaces
- Namespaces
- Functions.and.operators

Files.

- Native.files
- Component.files

Diversions.

Performance.

A more useful example:

```
rvec+(excl)#.refs ref      A Vector of subspace references.
```

refs produces a vector of namespace references to *ref* and each of its subspaces. The optional left argument is a list of subspaces to be excluded from the traversal.

Some technical points:

The coding is complicated by having to cater for *cyclic* namespace references. As a space may contain a reference to one of its ancestors, the code must not blindly follow all subspace references, otherwise unbounded recursion will result in a *WS FULL*. The trick is to avoid re-visiting any space by maintaining a list of those already visited.

```

refs←(
  α←θ ∘ (ρ,α)+α{           a Vector of subspace refs for w.
    1∈w=α:α                a default exclusion list.
    w.(↑∇∘↑~/φ(=α,w),+[]NL 9) a already been here: quit.
  )w                        a recursively traverse subspaces.
                           a for given starting ref.
)

```

The outer code defaults the exclusion list to null (no exclusions) and removes the list from the final result:

```

α←θ ∘ (ρ,α)+α{           a default exclusion list.
  ...
}w                        a for given starting ref.

```

The first line of the inner function stops if the space is in the list:

```

1∈w=α:α                  a already been here: quit.

```

This code could be simplified to $w \in \alpha : \alpha$ with an enhancement of the interpreter to include refs within the domain of primitive set functions and even omitted altogether by set-subtracting the list of visited refs from the list of subspace refs. In the latter case however, the code would have to accommodate the execute of each of a null vector of sub-space names: $(1 \uparrow \uparrow + 0 \uparrow, +[]NL - 9) = \alpha$.

Perhaps a little daunting at first sight, the second line is interpreted thus:

```

w.(.....) a Within the subject space,
.....+[]NL 9 a take a vector of sub-space names,
.....(=α,w),..... a prefixed with extended visited-list.
.....↑~/φ..... a left-to-right reduction with operand:
.....∇..... a recursive call between visited-list
..... a and
.....↑..... a ref from sub-space name.

```


Notably absent from the code is any explicit test to quit when there are no more subspaces. In this case, $+ \square ML$ 9 is a null vector, so $\phi(\alpha, w)$, $+ \square ML$ 9 is a 1-item vector, which is just passed along as the result of the reduction without an application of its recursive operand function. The technique is similar to that used in the *pow* operator.

Examples:

<code>refs □se</code>	A Vector of space refs for □SE.
<code>(refs #).(□wx+1)</code>	A Set value of □WX in all spaces in ws.
<code>[/(refs #).□nl</code>	A Max □ML in workspace.
<code>[/(+/. '=w}"refs #</code>	A Max namespace depth.
<code>□+ftp"(refs #).□nl 2</code>	A Total number of global variables.
<code>t;/{w find'pp'}"refs #</code>	A Search for rank.
<code>^/0=(refs #).□nc='foo'</code>	A Is this a foo-free workspace?
<code>(refs #).□ex='foo'</code>	A Defoobarise the workspace.
<code>(refs #)fnrepl'"c'foo' 'bar'</code>	A In all the fns in all the spaces ...
<code>□se.(cbtop cbbot) refs □se</code>	A □SE refs, excluding CoolBands.

See also: tree up Namespaces

What next?

This is the first of a regular series of columns in Vector describing the activities of the dfns email group. To subscribe to the group, send email to dfns@dyalog.com with subject: subscribe

References:

- [1] Dynamic Functions in Dyalog APL, Vector 13.2, 88, available from: www.dyalog.com/download/dfns.zip.
- [2] D: A functional subset of Dyalog APL, Vector 17.4, 93
- [3] Workspace *dfns.dws* from www.dyalog.com/download/dsamples.zip.
- [4] Jantunen, Veli-Matti, Dynamic Functions in Practice, Vector 18.4, 50

Hungarian Method Cost Assignment

an extended example of dfns from John Scholes

`bmat ← #.assign costs` A Hungarian method cost assignment.

`assign` is a classical algorithm implemented in the d:style. H.W.Kuhn published a pencil and paper version in 1955, which was followed by J.R.Munkres' executable version in 1957. The algorithm is sometimes referred to as the "Hungarian method".

```

assign←{OML←0                                A Hungarian method cost assignment.

  step0←{step1(φ[\φpω)†ω}                    A 0: at least as many rows as cols.

  step1←{step2†(†ω)-[†ω}                      A 1: reduce rows by minimum value.

  step2←{                                       A 2: mark independent zeros.
    stars←{                                   A independent zeros.
      ~1εω:a                                  A no more zeros: done.
      next←<[<†ω                             A more independent zeros.
      mask←(rows next)∨cols next             A mask of dependent rows & cols.
      (α∨next)∇ω←mask                        A α=accumulated star matrix.
    }                                         A
    zeros←{ω+0 stars ω}ω=0                  A 1=>zero, 2=>independent zero.
    step3 ω zeros                            A next step: 3.
  }

  step3←{costs zeros←ω                        A 3: cover cols with starred zeros.
    stars←zeros=2                            A starred zeros.
    covers←2×cols stars                     A covered cols.
    ~0εcovers:stars                         A all cols covered: solution.
    step4 costs zeros covers                A next step: 4.
  }

  step4←{costs zeros covers←ω                A 4: adjust covering lines.
    mask←covers=0                           A mask of uncovered elements.
    open←1=mask×zeros                       A uncovered zeros.
    ~1εopen:([†(,mask)/,costs)step6 ω       A no uncovered zeros, next step :6.
    prime←first open                        A choose first uncovered zero.
    prow←rows prime                         A a row containing prime.
    star←2=zeros×prow                       A star in row containing prime.
    ~1εstar:prime step5{                     A no star in row, next step :5,
      costs ω prime                          A adjusted zeros matrix,
    }zeros←2×prime                           A new primed zero (3).
    cnext←covers+prow-2×cols star           A adjusted covers.
    znext←zeros[3×prime                     A primed zero.
    ∇ costs znext cnext                     A adjusted zeros and covers
  }

```

```

step5+{costs zeros prime+ω
  star+{cols prime}^zeros=2
  ~1εstar:step3 α{
    {costs ω}{ω-2×ω=3}ω-α^ω>1
  }zeros
  pnext+{rows star}^zeros=3
  (αvpnextvstar)∇ costs zeros pnext
}

step6+{costs zeros covers+ω
  cnext+costs+α×-1 1+.×0 3×.=covers
  znext+zeros+(×costs)-×cnext
  step4 cnext znext covers
}

rows+{(pω)p(≧φpω)/√ω}
cols+{(pω)p√ω}
first+{(pω)p< \, ω}

(pω)1step0 ω
}

```

A 5: exchange starred zeros.
 A next star.
 A no stars: next step :3.
 A unstarred stars; starred primes.
 A adjusted zero markers.
 A next prime.
 A α-accumulated prime-star- path.

A 6: adjust cost matrix.
 A add and subtract minimum value.
 A amended zeros marker.
 A next step: 4.

A row propagation.
 A column propagation.
 A first 1 in bool matrix.

A start with step 0.

The dfns workspace, which contains assign, is available as a free download from www.dyalog.com/download/dsamples.zip (you need Dyalog V9.0 or better to load it)

The method indicates an optimal assignment of a set of resources to a set of requirements, given a "cost" of each potential match. Examples might be the allocation of workers to tasks; the supply of goods by factories to warehouses; or the matching of brides with grooms. The function takes a cost matrix as argument and returns a boolean assignment matrix result.

The following table shows an optimal assignment of factories F, G, H to warehouses W, X, Y, given that the cost of transportation from F to W is 72 units, F to X is 99 units, ..., G to W is 23 units, ... and so on.

	W	X	Y
F	[72]	99	88
G	23	30	[35]
H	51	[59]	84

Minimum-cost assignment marked [.]:

Factory F supplies warehouse W,
 .. G Y,
 .. H X.

Notice that if the problem requires maximizing a benefit, rather than minimizing a cost, then a negative cost matrix is used. See below for an example.

Technical Notes:

Munkres' algorithm may be described in words as follows:

Step 0:

Ensure the costs matrix has at least as many rows as columns, by appending extra 0-item rows if necessary. Go to Step 1.

Step 1:

Subtract the smallest item in each row from the row. Go to Step 2.

Step 2:

Select a set of "independent" zeros in the matrix and mark them with a star (*). To do this, star leading zeros in each row and column, ignoring rows and columns already containing stars; repeat this process until apart from ignored rows and columns, no more zeros remain. Go to Step 3.

Step 3:

Draw a line through (cover) each column containing a starred zero. If all columns are covered, the starred zeros represent an optimal assignment. In this case, return a boolean matrix with the positions of the stars, as result. Otherwise, go to Step 4.

Step 4:

Find an uncovered zero. If there is none, go to Step 6 passing the smallest uncovered value as a parameter. Otherwise, mark the zero with a prime (') and call it P0. If there is a starred zero (S1) in the row containing P0, cover this row and uncover the column containing S1, then repeat Step 4. Otherwise, (if there is no starred zero in P0's row) go to Step 5.

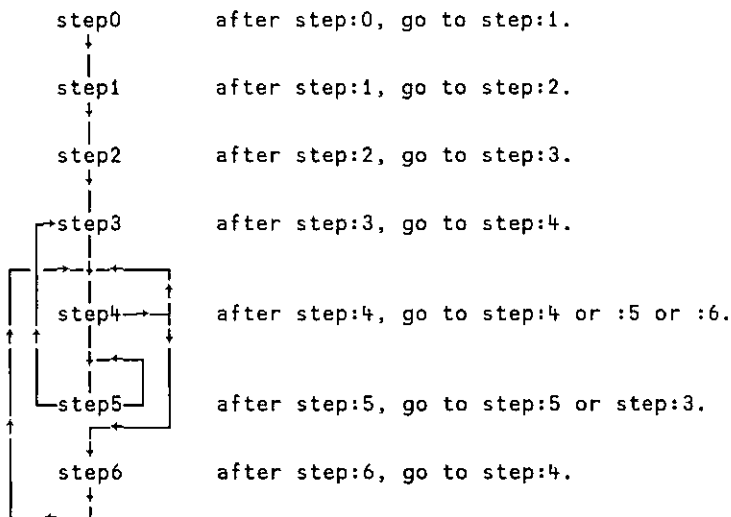
Step 5:

Find a path through alternating primes and stars. Starting with the uncovered prime (P0) found in Step 4, find a star S1 (if any) in its column. Then find a prime P2 (there must be one) in S1's row, followed by a star S3 (if any) in P2's column, ... and so on until a prime (Pn) is found that has no star in its column. In the series P0, S1, P2, S3, ... Pn, unstar each starred zero Si and star each primed zero Pj. Finally, unprime all primed zeros in the matrix, uncover all rows and columns. Go to Step 3.

Step 6:

Add the minimum cost value passed from Step 4 to each twice-covered (row and column covered) item, and subtract it from each uncovered item. Preserving all stars, primes and covering lines, go to Step 4.

The approach uses a rather convoluted "stepping algorithm", which can be represented as a flowchart:



The algorithm is implemented by coding each step as a separate sub-function, and tail-calling from one step to the next. This is the closest we come to branching in d:fns!

Notice that nearly all of the primitive and defined functions in `assign` deal in matrices - a testimonial to APL's native array-handling.

The state is represented by 3 matrices at each step:

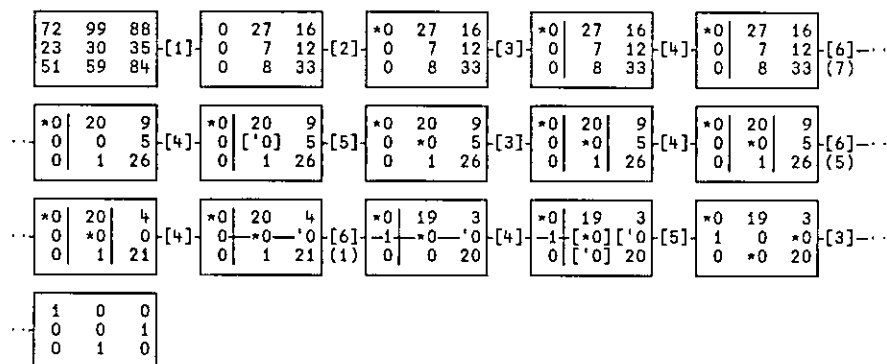
costs: the current cost matrix.

zeros: marked zeros: 0-not a zero, 1-unmarked zero, 2-starred, 3-primed.

covers: 0-uncovered item, 1-item covered by horizontal (row covering) line, 2-item covered by vertical (column covering) line, 3-item covered by both horizontal and vertical lines.

In addition, step 5 takes a boolean matrix left argument, indicating the position of the first primed zero.

Using the example cost matrix from above, the following trace shows the state of play between each step $-[n]-$.



References:

H.W.Kuhn, *The Hungarian method for the assignment problem*, Naval Research Logistics Quarterly, 2 (1955), pp. 83-97.

J.R.Munkres, *Algorithms for the assignment and transportation problems*, J. SIAM 5 (1957) 32-38.

Examples:

costs

72 99 88
23 30 35
51 59 84

A example cost matrix.

assign costs

1 0 0
0 0 1
0 1 0

A minimum cost assignment.

assign -costs

0 1 0
1 0 0
0 0 1

A maximum benefit assignment.

166 {+ / + / ω × assign ω } costs A total minimum cost.

206 {+ / + / - ω × assign ω } -costs A total maximum benefit.

costs + ? 6 10 p 50 A new 6 × 10 cost matrix.

costs
 7 38 23 27 11 3 34 34 47 20
 26 42 2 3 27 34 1 20 4 21
 35 30 47 43 27 5 33 21 36 46
 39 14 3 37 17 32 38 50 19 13
 50 37 38 33 4 32 45 14 22 39
 24 12 14 18 9 25 45 46 4 46

assign costs A minimum cost assignment.
 1 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 1 0 0 0
 0 0 0 0 0 1 0 0 0 0
 0 0 1 0 0 0 0 0 0 0
 0 0 0 0 1 0 0 0 0 0
 0 0 0 0 0 0 0 0 1 0

{ ω × assign ω } costs A cost per assignment.
 7 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 1 0 0 0
 0 0 0 0 0 5 0 0 0 0
 0 0 3 0 0 0 0 0 0 0
 0 0 0 0 4 0 0 0 0 0
 0 0 0 0 0 0 0 0 4 0

24 {+ / + / ω × assign ω } costs A total minimum cost.

assign -costs A maximum benefit assignment
 0 0 0 0 0 0 0 0 1 0
 0 1 0 0 0 0 0 0 0 0
 0 0 1 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 1 0 0
 1 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 1

{- ω × assign ω } -costs A cost per assignment.
 0 0 0 0 0 0 0 0 47 0
 0 42 0 0 0 0 0 0 0 0
 0 0 47 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 50 0 0
 50 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 46

282 {+/-ω×assign ω} -costs A total maximum benefit.

costs→?10 6p50 A new 10×6 cost matrix

costs
 14 1 21 2 36 47
 12 10 16 45 33 8
 35 20 20 25 8 30
 43 30 48 28 8 50
 21 8 29 13 25 24
 49 7 10 16 32 7
 33 32 41 13 24 20
 11 2 46 22 8 48
 21 7 45 5 9 4
 19 13 7 40 23 18

assign costs A minimum cost assignment.
 0 1 0 0 0 0
 1 0 0 0 0 0
 0 0 0 0 1 0
 0 0 0 0 0 0
 0 0 0 0 0 0
 0 0 0 0 0 1
 0 0 0 1 0 0
 0 0 0 0 0 0
 0 0 0 0 0 0
 0 0 1 0 0 0

Taming Barrel and Tilt Distortion

by Cliff Reiter (reiterc@lafayette.edu)

In the spring of 2002 Fraser Jackson suggested that I ponder the process of removing barrel distortion and he directed me to a commercial product that does a nice job [3]. I had been using a Nikon 990 digital camera and a wide angle lens that had relatively little barrel distortion so I was only casually interested. However, his e-mail happened to come at a time when I had just killed the Nikon camera during a wet hike, so I was in the market for a new digital camera. After getting advice and studying Steve's amazing Digital camera site [8], I got a Fujifilm S602 camera. I also got a wide angle lens (a Raynox DCR 720), but the lens gives substantial barrel distortion at full wide angle. Suddenly I am much more interested in algorithms for removing barrel distortion.

Barrel Distortion

Study of computer graphics books and searching the web readily leads to a standard model for barrel distortion. It is based upon the assumption that the distorted radius r_d is related to the true radius r by a spherical distortion of the form $r_d = r(1 + cr^2)$. Warren-Smith and Berry's page [9] is an easily available source that discusses useful facts regarding this model for barrel distortion.

There are a couple of issues to deal with in using the model. First, we typically do not know the constant for a given lens. While one can experiment with positive and negative coefficients to cause pin cushioning and barrel distortion using the model, we aim to automatically straighten a curve from control points. This requires having the distortion map and its inverse. The software [9] implements the inverse via an approximation based upon two steps of Newton's method. We follow that approach and asked *Mathematica* [4], via the following commands, to obtain the rational map that approximates the inverse.

```
f[r_] = r(1 + c r^2) - rd
```

```
Newt[r_] = r - f[r]/f'[r]
```

```
Simplify[Newt[Newt[rd]]]
```

Rewritten in a form useful for our J implementation; the approximate inverse is the following.

$$r = \frac{r_d (1 + 11cr_d^2 + 39c^2r_d^4 + 51c^3r_d^6 + 16c^4r_d^8)}{1 + 12cr_d^2 + 48c^2r_d^4 + 75c^3r_d^6 + 36c^4r_d^8}$$

The spherical distortion and its inverse can be readily implemented and applied in J. A script with these barrel distortion functions and a cover form is available from [6].

```
NB. radius to barrel distorted radius
R_to_RD=: ] + [ * ^&3@

NB. distorted radius to radius
p1_temp=.1 11 39 51 16&p.
p2_temp=.1 12 48 75 36&p.
RD_to_R=: ] * (p1_temp % p2_temp)@: ([**:@]) f.
```

For $c = 0.1$ we can convert between true and distorted radii and back with the following.

```
0.1 R_to_RD _1 0.8 1
_1.1 0.8512 1.1

0.1 RD_to_R 0.1 R_to_RD _1 0.8 1
_1 0.8 1
```

The function `barrel_undistort` defined below is designed to remove barrel distortion. The coefficient is given as the left argument and the image data as the right argument. We like to see as much of the un-distorted image as possible, which means some padding needs to be added. The padding can be subsequently clipped if desired. The function pads the image with one edge of white pixels on the bottom and right. Then, for each pixel in the result image, we compute what the corresponding pixel would have been in the input image. Some of those values are out of range, so by clamping the values between `_1` and the image width/height, white is used for those out of range values.

Notice the heart of the function, which is in the `for_k`. loop, converts pixel coordinates into polar coordinates and adjust the radius with `R_to_RD`.

```
NB. c barrel_undistort raw_image
barrel_undistort=: 4 : 0
wh=.|.hw=.2{.$y.
und=.wh sn_to_pix@(x. sn_undist)@(wh pix_to_sn)
y.=.(y.,{:a.),"_1 {:a.
if. x.<0 do.
'dw dh'=.|und 0 0
else.
'dw dh'=.~<./und -:wh*=i.2
end.
z=.((hw+2*dh,dw),3){:a.
dsn=. -{.-/hw pix_to_sn 0 0,:1 1
w=.j./"1 hw pix_to_sn (-dh),.(-dw)+i.(2*dw)+{:hw
for_k. (-dh)+i.({.hw)+2*dh do.
t=.12 o. w
r=.|10 o. w
r=.x. R_to_RD r
in=.;/_1 >. hw <."1 hw sn_to_pix +.r r. t
z=.{in{y.}(dh+k)}z
w=.w+dsn
end.
NB. 'one'+1
z
)
```

The function also uses various utility functions such as `pix_to_sn` and `sn_to_pix` which convert between pixel coordinates (bounds specified on the left) and semi-normal coordinates. Semi-normal coordinates range from `_1` to `1` while respecting the aspect ratio. See the script for the particular implementation details of those and related utilities.

We will see in our example that we determine the coefficient of distortion by selecting points that should be on a line. Various coefficients are tried and how close the resulting points are to a line is determined. A modified bisection strategy is used to determine the coefficient to three decimal places. That coefficient is then used to remove the distortion from the image. Again, the script contains the implementation details.

An Example

Figure 1 is an image of Angela Coxe discussing her work [1,2] with Professor Chawne Kimber that was taken under adverse conditions. The room was very crowded and not very bright. Taking the photo required a lucky break in the crowd and use of the wide-angle lens. The intensity has been slightly increased in the style of [5] and the image has been loaded into the `transform_image` function defined by `transform_m.ijc` [6]. Notice the wall section edge near the top is curved. It should be a straight line. Three points along the curved section near the top have been selected and marked with blue crosses. These are used as control points that determine the distortion coefficient. For those points, the distortion coefficient was `_0.078`.

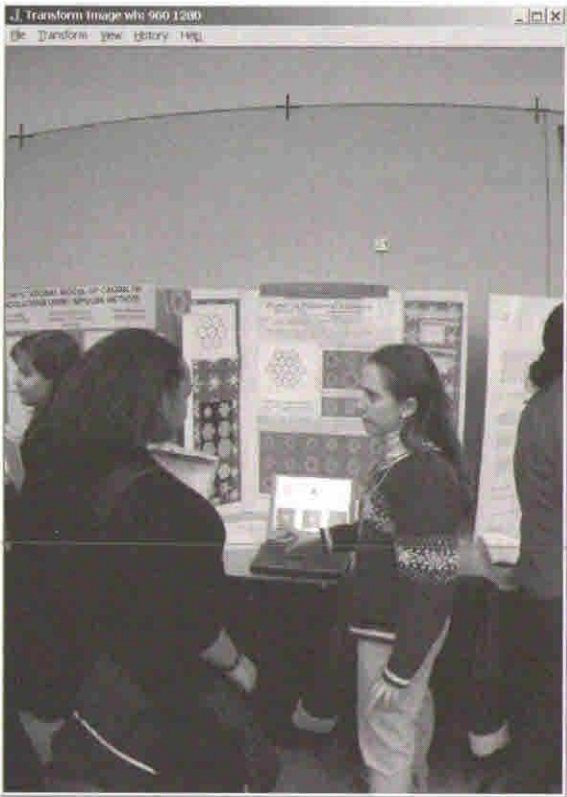


Figure 1. Barrel Distortion and Points to be on a Line

Running the "transform"-"remove barrel distortion" menus on the Transform Image form results in the underlying image that appears in Figure 2. Notice that the section edge on the wall and even the poster edges are now lines. The barrel distortion has been removed.



Figure 2. Barrel Distortion Removed and Points to Form a Rectangle

However, the angle between the horizontal and vertical section edges is not a right angle. In the next section we see how to make the angle into a right angle.

Tilt Removal

A problem commonly arising in images is that the image was taken at some angle other than centred and perpendicular to the object of interest. It is common [7] to

adjust such a tilted image by using a function that maps a point (x, y) to a new point via

$$(x_{new}, y_{new}) = (1, x, y, xy)A$$

where A is a 4 by 2 matrix of parameters. This is a slightly nonlinear map since it contains the xy term. The eight parameters in A may be determined by knowing the behavior of four control points. We have found it convenient to select four points and to map those to a rectangle that is an average of the points. Thus, the four selected points in Figure 2 correspond to points that we want to map to a rectangle. Using the “transform” – “tilt removal” menus, we obtain the image in Figure 3.

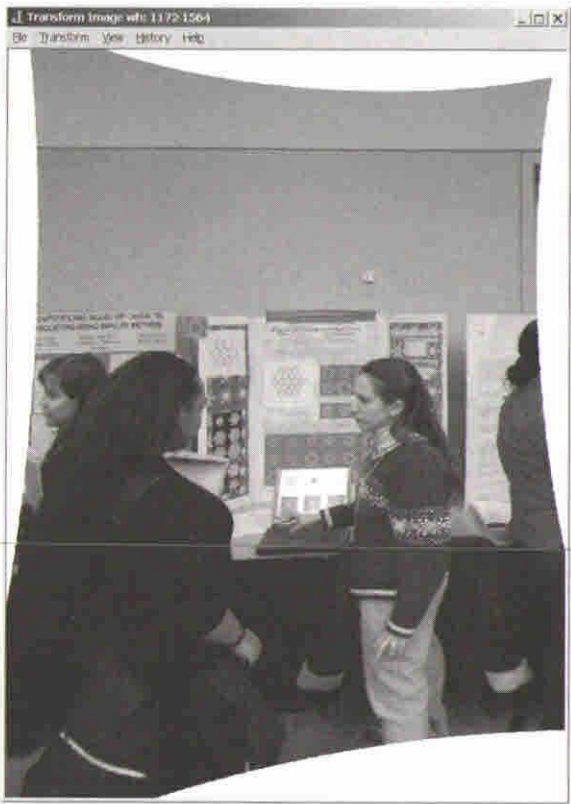


Figure 3. Quadrilateral Tilted to a Rectangle

Notice that the wall section edges now appear perpendicular and the top edges of the posters are horizontal. Of course, other features are somewhat distorted. We can trim the image by selecting a rectangle with the left mouse down and using the "transform" – "trim" menus.

Of course we are not limited to using this distortion to make images "correct". For example, one can select points more or less eccentric than those required by the barrel distortion. One can also use the menus to directly specify the distortion constant. Figure 4 shows the result of applying the distortion removal with a coefficient of 0.3. This adds to the distortion.



Figure 4. Barrel Distortion Added $c=0.3$

While the software at [3] is faster than the functions described here, we have developed simple and effective tools for removing barrel distortion in J. One of the advantages of the J tools is that the source script is available, ready for modifications and

improvements. For example, once the distortion coefficient for a particular lens is known, we are able to modify images taken with that lens under program control simply by using `barrel_undistort` with the suitable coefficient.

References

- [1] A. M. Cox and C. A. Reiter, Boolean Hexagonal Automata, *Vector*, 19 3 (2003) 113-121.
- [2] A. M. Cox and C. A. Reiter, Fuzzy Hexagonal Automata and Snowflakes, *Computers & Graphics*, to appear.
- [3] Grasshopper's Image Align, www.imagealign.com.
- [4] *Mathematica 4*, Wolfram Research, <http://www.wolfram.com>.
- [5] Cliff Reiter, With J: Image Processing 2: Color Spaces, *APL Quote Quad*, submitted to APL 2003 Proceedings.
- [6] Cliff Reiter, Transform Image script, <http://www.lafayette.edu/~reiterc/j/vector/index.html>
- [7] J. C. Russ, *The Image Processing Handbook*, 4th edition, Boca Raton, CRC Press LCC, 2002.
- [8] Steve's Digicam Site, <http://www.steves-digicams.com>
- [9] R. F. Warren-Smith and D. S. Berry, AST: A Library for Handling World Coordinate Systems in Astronomy, <http://star-www.rl.ac.uk/star/docs/sun211.htm/node235.html>

The Milan Cathedral Challenge from Adrian Smith

I had already made several attempts to 'straighten out' some shots of the wonderful stained glass in Milan Cathedral. As always, these can only be taken at a sharp upwards angle, and the various filters in PaintShop Pro tended to trash the quality of the transformed image, I suspect because you have to go through several independent stages.

I sent an original to Cliff to see how much better his software handled the task, and I have to say that the results are considerably superior. Here is his reply, along with a section of the final image.

I updated the transform_m materials on my web page to include nonlinear rescaling and adding menu items "balance vertical" and "rescale". Also attached as a tiny zip.

It seems that a cubic transformation does a good job of balancing the size of windows—so I included that as a separate feature rather than imposing it upon the barely nonlinear "tilt to rectangle".

Taking your original image, I selected four points which should form a rectangle, I applied "tilt to rectangle" (sent previously) Then, I selected four more points, now giving the top/bottom of two windows, one near the top, the other the bottom. I used "balance vertical". Then I rescaled the vertical direction by a factor of 1.2 to adjust the aspect ratio.

The result is attached.



Graphical Simulation of Rearview Mirrors

by Brian Schott

*"Mirror, Mirror, on the wall
Who's the fairest one of all?" from Snow White and the Seven Dwarfs*

This article describes a J wireframe graphics simulation of a driver's mirror views of passing vehicles. A recent Car Talk column in our local newspaper answered a question about positioning car sideview mirrors to mostly eliminate the "blind spot" for the driver by turning the left mirror out dramatically (in the USA where we drive in the right lane). You can find a similar suggestion on the website of the National Motorists Association

http://www.motorists.com/issues/safety/adjust_mirrors.html
as well as others.

I have been attempting to use the suggested mirror adjustments with some success but with some visual disorientation, also. I have been trying to figure out what is disorienting. I assume the disorientation is related to some discontinuity between the images in the sideview and rearview mirrors. I once set up a "simulator" in my dining room with a pair of mirrors and my wife "passing" me, and have gone to a crowded parking lot to attempt to see the phenomenon. But these two experiments have not clarified the situation to me.

Graphical Simulation

Another possibility for understanding my disorientation was to simulate the image on four separate dynamic gl2 (gl2 is a 2 dimensional rendering feature available with J) screens. The four screens are constructed using 3D to 2D projections.

NB.	+	-----	+	-----	+	-----	+	-----	+	-----	+
NB.		SCREEN		TITLE		VIEW		VIEWER		PROJECTION	
NB.	+	-----	+	-----	+	-----	+	-----	+	-----	+
NB.		1		Left		Mirror in car		Right lane driver		Perspective	
NB.	+	-----	+	-----	+	-----	+	-----	+	-----	+
NB.		2		Centre		Mirror in car		Right lane driver		Perspective	
NB.	+	-----	+	-----	+	-----	+	-----	+	-----	+
NB.		3		Above		Camera		Bird's eye		Parallel	
NB.	+	-----	+	-----	+	-----	+	-----	+	-----	+
NB.		4		Side		Camera		Sidewalk		Parallel	
NB.	+	-----	+	-----	+	-----	+	-----	+	-----	+

Screens 1 and 2 show what a cyclops driver-in-the-right-lane's eye sees in her mirrors of a passing car either behind or to the left. These two are called Left and Centre screens corresponding to the sideview and the rearview mirrors. These screens are created using perspective projection. The wire-frame drawing of the passing car in perspective shows only 7 edges of the car, the 4 front edges are rendered in red, and the 3 side edges are rendered in blue. The amount viewed in each mirror depends on the mirror adjustment and the cars' positions.

```
NB. Mirror view of the front and right sides of a wireframe car
NB.          /|
NB.         /e|
NB.        /u/
NB. +-----+ /L/
NB. |  red  |b/
NB. +-----+ /
```

Screens 3 and 4 provide a bird's-eye view of the cars on the street below and a sidewalk-pedestrian's view, both using parallel projection. These two screens show two vehicles, a car being passed in the right lane and a car passing it, and lines marking the viewing range of the two mirrors in the being-passed car.

The "world" view of this system is centred on the driver's eye; the "world" origin corresponds to $(X\ Y\ Z)=(0\ 0\ 0)$. The cars travel along the Y axis in the positive Y direction; naturally we are most interested in the cars while they are at negative values of Y, behind the mirror-using driver. The passing car alongside the mirror-using driver is in the negative direction along the X axis relative to the origin.

The human's, cars' and the mirrors' dimensions are based upon me sitting in my 2000 Toyota Camry. The simulation assumes 10 foot wide driving lanes. Measurements are in inches or pixels. Several parameters are required to define the simulation.

In the list below, the first four are measurements related to the left and centre mirror, respectively.

```
m =: _21 28 _8.5 ,: 14 18 1.5  NB. mirror from eye (X Y Z)
S =: 3.5 2,:4 1.125           NB. mirror width, height
Vsize =: 500 500,:500 500     NB. viewport size in pixels
Vcentre =: 500 500,:500 500   NB. viewport centre in pixels
```

The next list of four gives the World and Viewport horizontal and vertical ranges using the symbols *V*, *W*, *a*, and *s* as described in NB.s².

NB. *V* means Viewport coordinates
 NB. *W* means World coordinates
 NB. *a* means view from Above
 NB. *s* means view from the Side

```
W_a_cars_ =: _200 100,._650 50
V_a_cars_ =: 0 300,._0 700
W_s_cars_ =: _650 50,._150 150
V_s_cars_ =: 0 700,._0 300
```

The last four in the list define car dimensions, car path, and the spatial relation between the eye and the car's outer walls.

```
ify =: *%36          NB. inches from yards
cardim_cars_ =: ify 2 5 1.5  NB. car dimensions in yards
pcardim_cars_ =: ify 2 5 1.5  NB. passing car dimensions in yards
eyeincar =: 18 94 46  NB. measured from left, back, ground
passpath =: 0 _132 _360 _240  NB. defines path of passing car
```

The passing car's path is defined by the noun *passpath* with the verb *xandy*:
xandy passpath . The four components of *passpath* correspond to *xs xe ys ye*, the starting and ending values in the X and Y dimensions of the passing car's path.

² The object oriented features of J refer to class names ('locales' in J) such as 'mirrors' and are symbolized by a postfix naming convention of the form *mirrors* . Names using such postfixes are referred to as 'locatives'. Instantiated objects of a class are also permitted and are easily named in J using 'indirect locatives' based on a slightly different postfix naming convention, such as '*__left*' for the left hand mirror. The locale 'cars' is referred to in this example.

```

NB. +-----+
NB. |      (xe,ye) is the upper kink in the X-Y plane
NB. | \
NB. |  \
NB. |   | (xs,ys) is the lower kink in the X-Y plane
NB. +-----+
xandy =: dyad define
  xs xe ys ye =. y.
  if. ys=ye do. xdelta =. 0
  else. xdelta =. (xe-xs)%ye-ys
  end.
  constantAgenda =. <&ye
  slopeAgenda =. (>:&ye)+.(<:&ys)
  constant =. (xe _) (xs _)@.constantAgenda 0 x.
  slope =. (xdelta _) 0:@.slopeAgenda 0 x.
  ,constant + slope * x.- ys
)

```

In the simulation the driver can adjust each mirror only by rotating it around the X or Z axes (left-right and up-down). While this is not strictly true for all mirrors for all cars, it simplifies matters considerably. In my Camry the left mirror electronic control applies only to left-right and up-down. (There are parameters in the program which permit Y rotation, as well.)

Quickstart

Save the scripts in the j502\user directory. If you wish to save them in another directory, you may change the 'require' instructions in the beginning of the script 'mirrorsrun.ijs'. Run | File the script entitled 'mirrorsrun.ijs' into a JFEwd version of J502. The run produces 4 blank gl2 windows which you should position on your monitor so they do not overlap. Next type the following command.

```
_6 _5 2 24 run 5 360
```

This will execute 5 instances of the simulation, the first of which places the passing car 360 inches (30 feet) behind the being-passed car. If the simulation is too fast, you may repeat each individual instance of the simulation after all are completed, by moving the cursor in the .ijs window to any of the (five) instances listed, and pressing return twice. The four atoms of the left hand argument of run are mirror angles in degrees: the first two are for the left mirror, the last two are for the centre mirror. For example the centre mirror is rotated 2 degrees around the X axis (downward barely) and 24 degrees around the Z axis (toward the driver).

The default mirror images may be too realistically small for you to understand the simulation. You may wish to artificially enlarge the mirrors with the following commands.

```
re_startleft 10 10 NB. default size is Sleft
re_startcentre 10 10 NB. default size is Scentre
```

Some Basic Parameters

cube contains the coordinates for 8 corners of a cube. `_1 1{~#: i. 8` creates the coordinates and `/: &'DHAECGBF'` sorts them so that ABCD are the corners of the bottom cube face and EFGH are the corners of the top cube face, each in clockwise order starting at the far left. Only the 6 corners ABCEFG are used in this simulation to avoid the need to manage hidden wireframe lines.

```
cube =: -: /: &'DHAECGBF' _1 1{~#: i. 8
_0.5 0.5 _0.5
0.5 0.5 _0.5
0.5 _0.5 _0.5
_0.5 _0.5 _0.5
_0.5 0.5 0.5
0.5 0.5 0.5
0.5 _0.5 0.5
_0.5 _0.5 0.5
```

```
NB.      +-----+F
NB.      /E      /I
NB.      / |      / |
NB.      /H |      / |
NB.      +-----+G |
NB.      |      |      |
NB.      |      +-----+B
NB.      | /A      | /
NB.      | /      | /
NB.      |D      | /
NB.      +-----+C
ify =: * &36 NB. inches from yards
mincar =: 1 1 1 0 1 1 0 NB. Only the 6 corners ABCEFG are used
cardim =: ify 2 5 1.5 NB. car dimensions in yards
carcorners =: cardim* 1 mincar #cube NB. centred around the origin
```

To graph lines in `gl2` requires the (starting point, ending point) pair for each line. Next we define the points which comprise each line. The number of lines drawn includes the outline of two sides of the car, and lines connecting the two diagonal corners of each side.

```

      A B C E F G  =: i. 6
      |: _2] \ lines =: F,B,B,C,C,G,G,F,G,B,C,F,F,E,E,A,A,B,A,F,B,E
4 1 2 5 5 2 4 3 0 0 1
1 2 5 4 1 4 3 0 1 4 3

```

Some obvious parameters—nouns in J—are set. I3 and I4 are identity matrices used as a basis to be amended for several transformations.

```

I3 =: =@i. 3
I4 =: =@i. 4
X Y Z =: i. 3
XX YY ZZ =: _2] \ 2#i. 3
XYZ =: i. 3

```

Matrix transformations of 3D points and axes

The main transformations needed in this project are listed below and except for the last, are performed using square matrices which post-multiply row vectors representing points. When multiple transformations are required, first composite transformation matrices are constructed using matrix multiplication, and finally the composite matrix post-multiplies the row vector. The most complicated transformations are listed as 4 and 5. These two require the others as compositions or as input data preparation. The last transformation in the list is so simple that only scalars and vectors are used.

0. translation
1. rotation
2. scaling
3. reflection
4. rotation around arbitrary coordinate axes
5. perspective projection
6. parallel projection

Constructing mirror images

Two major matrix-based graphics processes are required for constructing mirror images of objects or of the viewpoint in software:

- a) eye rotation and reflection around an arbitrary orthogonal axis system (positioned at the mirror) producing a virtual eye, and
- b) car image perspective projection to a 2-dimensional screen or mirror.

To place the eye of the driver being passed behind the mirror is accomplished quite easily by a 180 degree rotation of the eye around the axis defined by the

mirror. But eye rotation alone results in an unrealistic orientation. Axes reflection is required to correct the orientation. Similarly the perspective projection process requires an extra reflection, also; a z-axis matrix reflection transforms the left-handed coordinate axes "world" view into a right-handed "mirror" or "viewport" view. Axes reflections are accomplished by matrix multiplying image points by an altered identity matrix. The alteration involves toggling diagonal matrix elements from +1 to -1. "Principles of Interactive Graphics, 2nd edition," by William M. Newman and Robert F. Sproull was an indispensable resource in crafting this system, especially the arbitrary rotation and perspective projections.

In all, four kinds of transformations via 4x4-matrix multiplication are implemented in this system: translation, rotation, reflection, and scaling. Although the matrices are 4x4, the transformations are applied to points in 3D, 3-element row vectors. When the integer 1 is appended to the end of such a row vector, the result is a projection vector. The utility verbs `pfv` and `vfp` construct and deconstruct such projection vectors.

```
pfv =: ,&1 NB. projection from vector
vfp =: pfv^:_1 NB. vector from projection
```

Several transformation matrices are created by amending a 4x4 or a 3x3 array using `}`, as is seen in several of the following verbs and nouns. Matrices of order 3 by 3 and of order 4 by 4 are distinguished symbolically by names beginning with lowercase or uppercase alphabetic characters.

```
Translation transformation is accomplished by the following verbs.
Translate0 =: (] , 1:) 3} [
Translate =: I4&Translate0
```

```
(Translate_mirrors_]) Left m
+-----+-----+
| 1 0 0 0|_21 28 _8.5|
| 0 1 0 0|           |
| 0 0 1 0|           |
|_21 28 _8.5 1|       |
+-----+-----+
```

Rotation transformation is accomplished by the following verbs. The main rotation verbs' (`yrotaxes` and `xrotaxes`) arguments are right triangle side lengths (not angles).


```

but =: [:#&XYZ-.@(XYZ&=)      NB. all indices BUT ...
butbx =: [:<@:~but
hyp =: {.@*.@j./              NB.hypotenuse from other sides

axes0y=: ({:,-@{:.),:({:,{:})
axesy =: axes0y % hyp
yrotaxes =: axesy@] (butbx Y)}I4 _

axes0x =: ({:,-@{:.),:({:,{:})
axesx =: axes0x % hyp
xrotaxes =: axesx@] (butbx X)}I4 _

(yrotaxes_mirrors_;xrotaxes_mirrors_) 3 4
+-----+
|0.8 0 _0.6 0|1 0 0 0|
| 0 1 0 0|0 0.6 _0.8 0|
|0.6 0 0.8 0|0 0.8 0.6 0|
| 0 0 0 1|0 0 0 1|
+-----+

```

Reflection transformation is accomplished by the following nouns. Notice that R corresponds to 4 by 4; r to 3 by 3.

```

reverseX =: _1 (<XX)}I3      NB. reverse x axis
ReverseZ =: _1 (<ZZ)}I4      NB. Reverse z axis
ReverseXYZ =: _1 (;/XX,YY,:ZZ)}I4 NB. Reverse xyz axes

```

Other rotation transformation matrices, while also amending an identity matrix, require angle arguments (not triangle side lengths).

```

pi=: 1p1
dfr=: *&(180%pi)             NB. degrees from radians
rfd=: *&(pi%180)              NB. vice versa
sin=: 1&o.                    NB. in radians
cos=: 2&o.                    NB. in radians

Sin =: sin@(*&1r180p1)        NB. Sine in degrees
Cos =: cos@(*&1r180p1)        NB. Cosine in degrees

```

The rotations here are all for right-hand systems; for left-hand systems the rotations are slightly different from below. All rotations in this system are performed in the right-hand axes systems.

```
rx0 =: (Cos,-@Sin),:(Sin,Cos) NB. 2-dim rotation matrix in degrees
ry0 =: (Cos,Sin),:(-@Sin,Cos) NB. 2-dim rotation matrix in degrees
rRz0 =: rx0
```

```
(rx0_mirrors_;ry0_mirrors_;rz0_mirrors_) 30
-----+-----+-----+
|0.866025    _0.5|0.866025    0.5|0.866025    _0.5|
|    0.5 0.866025|    _0.5 0.866025|    0.5 0.866025|
-----+-----+-----+
```

```
bip =: < 1@(-, 0/-) NB. Table of boxed index pairs: eg, bip 0 2
```

The argument of `bip` specifies the direction of rotation. For example `bip 0 2` would be used for a rotation around the Y axis (axis 1) in the direction from axis 0 to axis 2, a clockwise rotation in a right-hand coordinate system.

```
rx =: (rx0@])`'(bip@])`'(=@i.@3:)}
ry =: (ry0@])`'(bip@])`'(=@i.@3:)}
rz =: (rz0@])`'(bip@])`'(=@i.@3:)}
```

```
(2 0&rx_mirrors_;1 2&ry_mirrors_;0 1&rz_mirrors_) 30
-----+-----+-----+
|0.866025 0    0.5|1    0    0|0.866025    _0.5 0|
|    0 1    0|0 0.866025    0.5|    0.5 0.866025 0|
|    _0.5 0 0.866025|0    _0.5 0.866025|    0    0 1|
-----+-----+-----+
```

To rotate the points in space around an arbitrary axis defined by the mirror requires 5 steps and is accomplished by `arbRot Y{ptRot3 =: r3points yangle, mangles:`

1. Translate the mirror to the origin, where the eye is (T).
- 2a. Find the composite rotation matrix defining the X and Z mirror adjustments (`ptRot3`). Extract the cosines of the direction angles from the composition rotation matrix which correspond to the Y axis, because Y is the arbitrary axis used in the rotation (`Y{ptRot3`).
- 2b. Construct the matrices required to adjust the X and Z axes around so they line up with the mirror's X and Z axes (R1 and R2).
3. Rotate around the Y axis 180 degrees (unless the Y axis is askew, in which case the 180 is adjusted) (`Rotate180y`), and reflect all three axes (`ReverseXYZ`).
4. Invert step 2 to return to the original orientation (`%R2` and `%R1`).
5. Invert step 1 to translate back to the mirror's original location (`%T`).

The transformation matrices constructed in these 5 steps are applied as a matrix multiplication (`x =: +/ . *`) sequence.

```
Rotmain =. T x R1 x R2
Rot =. Rotmain x Rotate180y x reverseXYZ x (%Rotmain)
```

Steps 2a and 2b will be discussed further. Step 2a is computed as follows. First calculate the direction cosines of the angles of the separate X and Z mirror adjustments—that is rotations. These direction cosines are derived from the raw angle adjustments of the mirror; the separate axis rotation transformations are composed into an overall rotation matrix containing the direction cosines. The verb `r3points` is used to build the 3 by 3 composed transformation matrix.

```

r3points =: (2 0 _ ry 0&{)x(1 2 _ rx 1&{)x(0 1 _ rz 2&{)
(r3points_mirrors_@];{.}) yangle__left,mangles__left =: _6 _5
+-----+-----+
| 0.996195 0.0871557      0|0|_6 _5|
|_0.0866783 0.990737 0.104528| | |
|0.00911026 _0.104131 0.994522| | |
+-----+-----+

```

Notice that monadic `r3points` takes angles in the improbable order Y, X, Z, so that it can be applied as follows, because the argument 'mangles' contains the X and Z mirror rotation angles. `r3points q,p,r` is q-rotate from axis 2 to 0 on p-rotate from 1 to 2 on r-rotate from 0 to 1.

The intermediate result is saved as `ptRot3` for later use. Row Y of `ptRot3` (Y { `ptRot3`) is a unit vector of direction cosines of the rotated mirror expressed as a vector in the world view space; as such, its three components define a vector in the world view space which is to be rotated around the Y axis of the space defined by the adjusted mirror. That is, we compute inputs to step 2b using the matrix multiplication (in `r3points`) to construct the composite transformation for the combined Y, X, and Z adjustments.

```
Y{ptRot3 =: r3points yangle , mangles
```

Step 2b employs `arbRot` with inputs `a`, `b`, and `c`, the direction cosines between the original Y axis and each of the new origin's X, Y, and Z axes, resulting from the composite rotation from the simple X, Z and possibly Y adjustments of the mirror. Take only the Y row from the composite matrix, because the (arbitrary) rotation is about Y.

```

arbRot =: monad define
a b c =. y.
aa bb cc =. -a,b,c
v =. hyp a,b
T =. Translate -m NB. m is the mirror coordinates
R1in =. v%~(b,a),:aa,b
R1 =. R1in (butbx Z)}I4
R2in =. (v,c),:cc,v
R2 =. R2in (butbx X)}I4
Rotmain =. T x R1 x R2
Rotmain x Rotate180y x ReverseXYZ x (%.Rotmain)
)

angles =. yangle__left,mangles__left =: _6 _5
arbRot__left Y_mirrors_ { r3points_mirrors_ angles
0.984974 0.171751 _0.0181207 2.98226e_15
0.171751 _0.963121 0.207121 6.09644e_15
_0.0181207 0.207121 0.978148 1.43872e_16
_5.2786 60.3347 _6.36565 1

```

Matrix R1 rotates the projection of the input unit vector onto the X-Y plane (with (X,Y)=(a,b)) around the Z axis to the Y axis. The rotation angle required by R1 (call it alpha), has cosine alpha = b/hypotenuse and sine alpha = -a/hypotenuse . We call the hypotenuse v. R1 defines the rotation around the Z axis.

The resulting rotated projection vector has a component v along the Y axis. The original unit vector has a component of c along the Z axis. Together these two components, v and c, define a projection of the original unit vector in the Y-Z plane (with (Y,Z)=(v,c)). We require the rotation of this projection in the Y-Z plane around the X axis to the Y axis. The cosine of the angle of this rotation (call it beta) is v, and the sine of beta is -c, producing transformation matrix R2.

Finally, if a Y angle adjustment is allowed in a mirror, special care must be taken with the algorithm employed herein for rotation around an arbitrary coordinate axes. The algorithm named arbRot itself uses the Y axis as the arbitrary axis of rotation, so any implicit Y rotation is ignored; only explicit Y rotation, is permitted and is accomplished using the verb Rotate180y. Notice that while Rotate180y amends a 4 by 4 shaped identity matrix, it is constructed from a 3 by 3 rotation, ry.

```

Rotate180y =: (0 2 ry 180-yangle)(<XYZ;XYZ)}I4
NB. yangle contains the Y mirror rotation angle

```

In step 2b above of the rotation around the mirror's Y axis, R1 and R2 are constructed in terms of the angles between the mirror's Y axis and the original Y axis. While in theory the original X or original Z axis could have been substituted

for the Y axis (with more straightforward subsequent reflections), in practice these alternatives are precluded by the driver's control over precisely the X and Z mirror movement. Either of those theoretical alternatives would mask one of the driver's mirror adjustments.

The final composite rotation transformation is captured by the matrix Rot4.

```
Rot4 =. arbRot Y{ptRot3 =: r3points yangle , mangles
```

Rot4 is used to locate the virtual eye by rotating and reflecting the origin (0 0 0) around the mirror, producing the (X Y Z) position of virt_eye.

```
virt_eye =: x&Rot4 &. (pfv 1) 0 0 0
```

Putting this all together and executing manually from the base locale requires the following J commands. In reality the execution is done in the mirror's own locale, and is more concise as shown in parallel.

```
Rot4      =. arbRot      Y      { r3points      angles
Rot4__left =. arbRot__left Y__mirrors_ { r3points__mirrors_ angles

virt_eye   =: x      &Rot4      &. (pfv      1) 0 0 0
lvirt_eye__left =: x__mirrors_&Rot4__left &. (pfv__mirrors_ 1) 0 0 0
_5.2786 60.3347 _6.36565
```

Perspective projection

To see the passing car which is a 3D object, in the mirror which is a 2D object, suggests a perspective projection. The 5 main steps in this projection correspond to the matrices T1 to T5; a final scaling transformation is performed by matrix N.

T1 x T2 x T3 x T4 x T5 x N

- T1. Translate the mirror to the virtual eye to make the viewpoint (the virtual eye) into the new origin relative to the mirror.
- T2. Rotate the coordinate system around the X axis by -90 degrees to get the Z axis pointing more in the direction of the origin (the mirror). A -90 degree axis rotation is identical to a +90 degree point rotation, which is actually embodied in T2.
- T3. Rotate the resulting points around the Y axis so that values on the Z axis are in the plane defined by the Z axis of the mirror and the viewing point (the eye).
- T4. Rotate the resulting points around the X axis so that values on the Z axis are on the line defined by the centre of the mirror and the viewing point (the eye).

T5. At this point the new axes are oriented as a right-handed system. Conversion to a left-handed system can be accomplished by reversing the Z axis.

N. Finally a scaling transformation is performed to make the viewport (the mirror) consistent with its size and distance from the eye in world view measurement units.

```
T2 =: xrotaxes 0 1
T5 =: ReverseZ

perspective =: monad define
T1 =. Translate -y.
T3 =. yrotaxes }:-y.
T4 =. xrotaxes (hyp@):,(:) -y.
N =. (DZS)((<XX),<YY)}I4
T1 x T2 x T3 x T4 x T5 x N
)
```

The actual result of the verb `perspective` is a set of 3D points for which the Z ordinate reflects the relative distance of each point from the mirror. To get the desired perspective projection, each new value of the X and Y coordinates must be rescaled by the value of the Z coordinate by dividing by the value of the Z coordinate.

The perspective transformation is retained by the matrix VN in the mirror's own locale. The argument of the verb `perspective` is the difference from the virtual eye to the mirror (symbolized m).

```
VN      =: perspective      m      - 1 virt_eye
]VN__left =: perspective__left m__left - 1 virt_eye__left
9.25475  _0.466636  0.436495  0
_4.49974  _0.959744  0.897753  0
0         17.977  0.0592588  0
6.44554e_15 _1.43155e_14  36.0174  1
```

First the positions of the centre of the passing car relative to the driver's (real) eye must be computed (`pcfe`). The centre's position is based on the car's centre relative to the eye (`eyeincar`).

```
phd =: -: pcardim_cars_ NB. half-dimension models the car centre
pfe =: _1 i _1 NB. measurement direction from eye
pafo =: 0 _1 0 * pcardim_cars_ NB. direction from origin adjustment
pcfe =: phd+pafo+pfe*eyeincar_base_ NB. car position from eye position
```

Second the position of the passing car's centre (`pcarcentre`) is computed from its passing path. W and D are the X and Y coordinates in the expressions below.

```
pcarcentre =. (W+X col pcfe),(D+Y col pcfe),Z col pcfe
```

Third the car's perspective image is constructed by first matrix post-multiplying the corners of the car relative to the mirror (symbolized m), by the transformation matrix VN.

```
badlines =. lines&e.@((0:<{:)"1#i.@#) NB. see below
pcar =. x&VN&.(pfv 1) ( pcarcentre + 1 pcarcorners_cars_) - 1 m
u =. badlines pcar
```

Finally, the X and Y coordinate values of the transformed points are divided by the Z coordinate. Notice that the X coordinate values must first be reversed (negation, instead of addition) because viewing from behind the mirror is not like viewing from in front of a mirror: the X axis is backwards. (Xs,Ys) pairs are the desired perspective projections.

```
Xs =. (X{V_c)+(X{V_s)*%/ 1 (X,Z)&{ 1 pcar x reverseX
Ys =. (Y{V_c)+(Y{V_s)*%/ 1 (Y,Z)&{ 1 pcar
```

There is a nasty little problem with this perspective projection algorithm which occurs when the passing car front end gets in front of a mirror. In such cases the Z coordinate changes sign and produces invalid projections. The verbs `badlines` and `cleanlines` select and edit those cases.

```
cleanlines =. [ { 0 1 [: ] [: 2 0&|: 0 0 _ ,~ lines _ { ]
t =. Xs,.Ys
u cleanlines t
```

Parallel Projection

Two output screens require parallel projection: the views from above and from the side. Parallel projection itself is quite easy in this case because the projections are between parallel axes. To project from (X Y Z) to (X Y) or to (Y Z) involves just dropping the appropriate dimension (that is column) of a coordinate array.

The challenge is to find the equations of the line segments that define the viewing range of each mirror. Each line segment is coincident with a line defined by the virtual eye behind the mirror and a mirror edge. First we define an array `nsew` with rows corresponding to the edges according to geographic directions: north, south, east, and west.

```

n s e w =: i. 4
expand =: #^:_1
xz =: 1 0 1&expand
nsew =: xz 1]0 1,0 _1,1 0,:_1 0
]nsew_mirrors_ =: xz_mirrors_ 1]0 1,0 _1,1 0,:_1 0
0 0 1
0 0 _1
1 0 0
_1 0 0

```

nsew is used with the mirror (half-) size vector S to locate the four edges of the unadjusted mirror.

```

S =. 3.5 2 NB. eg, left mirror (half-) width and height
rme =. nsew * 1 xz S NB. rectmirrorededges
]rme__left =. nsew_mirrors_ * 1 xz_mirrors_ 5__left =. 3.5 2
0 0 2
0 0 _2
3.5 0 0
_3.5 0 0

```

```

nsEW =: 0 1 3 2&{ NB. reverse e&w for virt_eye behind mirror
nsEW rme
nsEW_mirrors_ rme__left
0 0 2
0 0 _2
_3.5 0 0
3.5 0 0

```

The rotated mirror edges are finally identified by employing the rotation matrix ptRot3 defined earlier and translating each mirror edge to the mirror's centre by adding mirror to each edge.

```

]ptRot3__left =: r3points_mirrors_ yangle__left , mangles__left
0.996195 0.0871557 0
_0.0866783 0.990737 0.104528
0.00911026 _0.104131 0.994522

```

```

medges =. m + 1 nsEW x &ptRot3 rme
]medges__left =. m__left + 1 nsEW_mirrors_ x_mirrors_&ptRot3__left rme__left
_20.9818 27.7917 _6.51096
_21.0182 28.2083 _10.489
_24.4867 27.695 _8.5
_17.5133 28.305 _8.5

```

Next we create some utility verbs for extracting array information from medges and the like.


```
col =: &({ 1) NB. an operator or adverb, not a verb
ew =: (e,w)&{
Yew =: (Y col)@:ew
ns =: (n,s)&{
Zns =: (Z col)@:ns
```

As discussed in "The Penguin Dictionary of Mathematics" the "two-point form" of writing the equation of a line in three dimensions is

$$(x-x_1)/(x_2-x_1) = (y-y_1)/(y_2-y_1) = (z-z_1)/(z_2-z_1).$$

In both the screens projected here, we want all the line segments defining the range of sight to be visible to a known value of Y centrey, corresponding to the dimension for which the cars are progressing. Also, recall that all line segments are coincident with lines connecting the virtual eye and the mirror edges. Consequently the expression for the variable Y in "two-point form" $(y-y_1)/(y_2-y_1)$ becomes either

```
Yr =. ((centrey)-Yew medges)%((Y col virt_eye)- 1 Yew medges), or
Yr =. ((centrey)-Zns medges)%((Y col virt_eye)- 1 Zns medges)
```

depending on whether we are looking for east and west or north and south ranges. Once Yr is computed, because of the equal signs connecting the "two-point form" expressions, the unknown X and Z values can be calculated and then combined with the Yr value simply, via the appropriate sequence of instructions listed in two groups next.

```
xze =. (X,Z) col virt_eye
xzm =. (X,Z) col@:ew medges
xzp =. xzm+ 1 Yr * (xze - 1 xzm)
EW =. (centrey,.-{. 1 xzp),.((X,Y) col@:ew) medges
```

```
xze =. (X,Z) col virt_eye
xzm =. (X,Z) col@:ns medges
xzp =. xzm+ 1 Yr * (xze - 1 xzm)
NS =. (centrey,.-{: 1 xzp),.((Y,Z) col@:ns) medges
```

EW and NS are plotted in the parallel projection screens.

Findings

By comparing the new left-hand mirror positioning with the more conventional positioning, I have concluded that the blind spot can be almost totally eliminated. And having driven with the new positioning for a few months, I am quite comfortable with it. But I cannot quite break myself of turning my head a little to peak out the driver's window just before starting a passing manoeuvre.

The measurements of eye to mirror distances are a little conservative because I am not a cyclops, really. Both my eyes work, so the assumption of a point viewing position is conservative, and the viewing span in the mirrors is slightly wider than this model would suggest.

Possible sources of my initial disorientation with the new mirror positioning have suggested themselves to me as I have exercised this computer system. For one, I may not have used my left mirror very much and so this may be an unpracticed skill. Another possibility is that with the mirrors in the traditional position, I saw approximately the same thing in both (much like going to a TV sales display with multiple screens to view) and was just surprised by the different images in the new system.

The most likely possibility for disorientation occurs when a car starts behind the driver and then pulls out to pass from close behind. In this case the right to left movement of the car is combined with its back to front movement, and its trajectory in the left mirror can be somewhat U-shaped rather than linear. To picture the trajectory, think of the Nike whoosh, but reversed right to left.

References

John Daintith and R. D. Nelson (editors), "line", *"The Penguin Dictionary of Mathematics"*, Penguin Books, Limited 1989, p 199.

William M. Newman and Robert F. Sproull, *"Principles of Interactive Graphics, 2nd edition"*, McGraw-Hill Book Company, 1979.

Chris Burke, Roger K. W. Hui, Kenneth E. Iverson, Eugene E. McDonnell, and Donald B. McIntyre, *"J Phrases"*, Iverson Software Company, Toronto, Canada, 1998.

TECHNICAL SECTION

This section of VECTOR is aimed principally at those of our readers who already know APL. It will contain items to interest people with differing degrees of fluency in APL.

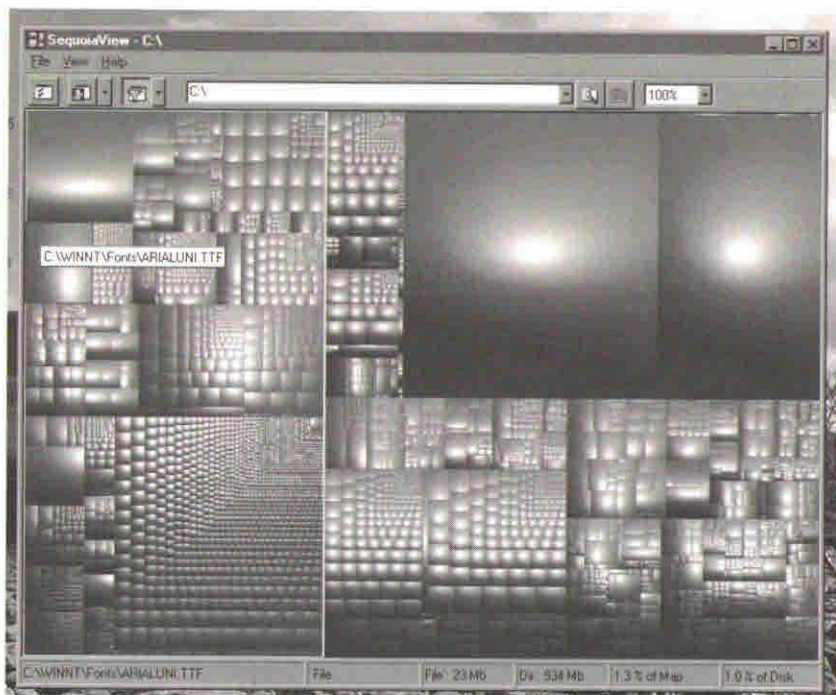
Contents

Hackers' Corner: Square Me Up	Adrian Smith	114
At Play with J: Greed	Gene McDonnell	115
Tools, Part 2. Basics too.	Dan Baronet	122
New Foundations	Graeme Robertson	132

Hacker's Corner: Square Me Up!

by Adrian Smith (adrian@causeway.co.uk)

Treemaps are a wonderful way of showing lots of numbers in a small space – if you want to know more, just search Google for Treemap. Here is a map of my C: drive produced by a marvellous (free) program called *SequoiaView* (get it from www.win.tue.nl/sequoiaview/) which will do wonders for your free disk space:



The two socking great files in the top right are the pagefile and hibernation file. The one with the tip showing is the Arial Unicode font. At each level, files are sorted by size, and drawn 'as square as possible' proportional to the file size.

So the challenge is deceptively simple: given a vector of (downsorted) positive numbers and a bounding rectangle, return a matrix of rectangles which exactly occupy the bounding rectangle, and are proportional in area to the given vector. May the squarrest man win! My solution will be in the next Vector.

CORRESPONDENCE

RE: R and APL

From: Mike Day

Did Sasha explain how he'd set up R as (presumably) a server process? I've got R, and it has associated folders for Tcl (for example) in its installation tree, but there's no obvious sign of

"StatConnectorSrv.StatConnector"

Maybe this is a feature to be paid for in S+ rather than the free R!

Reply: Alexander Skomorokhov wrote:

I use:

R-(D)COM Interface (for Windows) contains a COM server used to connect a client application (e.g., Microsoft Excel) with R. An Add-In for Microsoft Excel is provided as well as Active X controls to be included in your applications.

The url is as follows: <http://cran.r-project.org/contrib/extra/dcom/>

Dyalog 9 ws with 4 small interface functions is attached here (see [rapl.zip](#) on the resources page of the Vector website).

```

[vr" 'rexec'    'rget'    'rinit'    'rput'

▽ a←{r}rexec exp
[1]  A Execute R expression and possibly returns the result
[2]  AR: R expression ('char')
[3]  AL: if 1, then return the result
[4]  Ae: mlexec 'surf(peaks)'
[5]  ⚡(0=⊂NC'r')/'r+0'
[6]  a←θ
[7]  :If r
[8]    a←#.R.Evaluate(exp)
[9]  :Else
[10]   #.R.EvaluateNoReturn(exp)
[11] :EndIf
▽
```

```

▽ r←rget name;rho;Re;Im;ML
[1] A version for MLSE
[2] A Get R variable (matrix or vector) to APL
[3] AR-name of variable to import ('char')
[4] ML←3
[5] r←#.R.GetSymbol(name)
▽

▽ rinit
[1] A version for MLSE
[2] '#.R'\WC'OleClient' 'StatConnectorSrv.StatConnector'
[3] #.R.Init('R')
▽

▽ {m}rput a;x1
[1] A Version for MLSE
[2] A Put APL array into R
[3] AR-name of variable to export from APL
[4] AL-name of variable to create in Matlab
[5] ⍺(0=NC'm')/'m+'x'' A Default
[6] m←,cm
[7] a←,ca
[8] #.R.SetSymbol(m,a) A put var xu into R with name z
▽

```

Mike Day replies:

Thanks/spasibo to you both

I had looked at the cran.r-project site after mailing Adrian, and thought it MIGHT be R-(D)COM Interface that was needed; however, the associated web pages didn't mention StatServer but did seem to suggest that one would need to set up a class for each R command or function or group thereof, so I was inclined to think I must need something else instead.

It's not bundled into my R download, but as that (version 1021) is a couple of years old, it may be that the later/latest versions do include it.

I'll download R-(D)COM Interface when I find some time and let you know if I have any problems, for possible addition/amendment to the Vector article.

Mike

At Play with J: Greed

by Eugene McDonnell

My first experience of the British monetary system was in early 1953, in London. As was the case with many other visiting Americans, I felt daunted by the pound-shilling-pence currency. I had no idea what value the coins had. At a kiosk I picked up a newspaper, and then, expecting that a newspaper would be sold for just a few pennies, tendered, from the handful of coins I had, a smallish one. The vendor rapidly poured into my hand so many coins in change that I was completely unnerved. I took on faith that this was not a mistake, but I realized that I had better study the coins much more than I yet had. To this day I don't know what the coin was that produced such a flood of change. To solve the problem, I got in the habit, when I bought something, of just holding out a handful of coins, expecting that sturdy British honesty would ensure that vendors would take only the coins that would satisfy the transaction. This seemed to work quite well.

This paper discusses the problem of making change. When change is made in a store, we are used to seeing the clerk reach into the till and then take coins from its separate compartments. Unless there is a shortage of one or more coins, this is done by taking as many of the largest coin as are needed, followed by as many of the next largest, and so on, down to the penny. This almost instinctive method is called the *greedy algorithm* by computer scientists. The design of many, if not all, currency systems is such that the greedy algorithm also minimizes the number of coins that make up the amount of change.

I'll represent a set of coins by a list of the values in descending order. For example, the set of coins in the USⁱ is 25 10 5 1; the coins are called, in order, quarter, dime, nickel and cent, or penny. The list for European coinage is 200 100 50 20 10 5 2 1. In order to be able to give just 1 pence in change it is necessary that every coinage set have as its least coin one that is worth just one penny. The change itself can be represented by a corresponding list with the values showing the number of coins of each denomination used. For example, 99 pence change in the US would be represented by 3 2 0 4; in Europe by 0 0 1 2 0 1 2 0.

ⁱ There is a 50-cent piece in the US, but it is rarely used; the half-dollar is as rare in circulation as the two-dollar bill – which is very rare.

Although the greedy algorithm will give the fewest coins possible for any amount of change, it is not necessarily the case that existing coinage systems give the fewest number of coins possible. Jeffrey Shallitⁱⁱ recently suggested (with tongue firmly planted in cheek) that in order to make it possible in the United States to make change using the fewest coins, and still using just four values, the coinage should be replaced by one having coins worth either 25 18 5 1 or 29 18 5 1 cents. He showed that with the existing coins, assuming all values from 0 to 99 to occur equally often, which he admits may be far from the truth, the average number of coins needed to give change is 4.7; with either of his suggested sets only 3.89 coins would be needed. He recommended the 25 18 5 1 set, since only the 10-cent piece would need to be changed. To show the benefit of the proposal, note that to give 36¢ change, the minimum number of coins with the current system is three: 1 1 0 1; with the alternative, only two are needed: 0 2 0 0.

Shallit points out that there is a problem with his suggested change, and it has to do with the failure of the greedy algorithm. For example, to give 36¢ change using the set 25 18 5 1, the greedy algorithm gives the four-coin solution 1 0 2 1, but the optimal solution, as shown above, needs only two coins: 0 2 0 0. If a coinage system was such that the greedy algorithm was not always optimal, it would require such expertise in all those who make change to do so optimally that, if for no other reason, it would simply be too impractical. In the rest of this paper I'll concentrate on a set of J functions for the greedy algorithm.

A Greedy J Algorithm

In looking for a solution to a programming problem, I frequently try to picture the steps required. For the greedy algorithm, the picture that eventually stabilized was of two linked lists: one, a list A, beginning with a list of the number of coins needed for each coin considered so far, initially empty, followed by the total amount of change needed; the second list, C, was of the coins not yet considered, initially the complete coinage set in descending order. The two lists were linked to form the list AC. Assuming these two lists were available, the processing required to obtain the next result was to replace the last item of A by the quotient and remainder of this last item divided by the leading coin of C, and C was then modified by removing its leading item. For example, assuming that a function CS was available, that performed one step of the change process, the steps in obtaining 99 pence change in the Euro system would be:

ⁱⁱ Shallit, Jeffrey, "What This Country Needs is an 18¢ Piece.", *Math. Intelligencer* 25 (2) (2003) 20-23. Also available at Shallit's website, <http://www.math.uwaterloo.ca/~shallit/papers.html>. This gives pointers to the paper's text in two forms: ps (PostScript) and pdf.


```

EU =: 200 100 50 20 10 5 2 1
A =: (i. 0) , 99
C =: EU
] AC =: A ; C
+-----+
|99|200 100 50 20 10 5 2 1|
+-----+
] AC =: CS AC
+-----+
|0 99|100 50 20 10 5 2 1|
+-----+
] AC =: CS AC
+-----+
|0 0 99|50 20 10 5 2 1|
+-----+
] AC =: CS AC
+-----+
|0 0 1 49|20 10 5 2 1|
+-----+
] AC =: CS AC
+-----+
|0 0 1 2 9|10 5 2 1|
+-----+
] AC =: CS AC
+-----+
|0 0 1 2 0 9|5 2 1|
+-----+
] AC =: CS AC
+-----+
|0 0 1 2 0 1 4|2 1|
+-----+
] AC =: CS AC
+-----+
|0 0 1 2 0 1 2 0|1|
+-----+
] AC =: CS AC
+-----+
|0 0 1 2 0 1 2 0 0||
+-----+

```

The process stops when there are no more coins to be used. Since the last divisor is 1, only the quotient is germane, and after razing AC, the spurious remainder is discarded, forming the result R. A vector product shows that the number of coins provided does indeed give the needed amount of change:

```

] R =: } : ; AC
0 0 1 2 0 1 2 0
EU +/ . * R
99

```

We'll start by building the nuts and bolts that go into making CS. This involves developing a new A and a new C, and linking them.

$$CS =: NA ; NC$$

The new A is formed by curtailing A and appending the result of the quotient-remainder function QR.

$$NA =: CA , QR$$

CA is straightforward:

$$CA =: } : @ > @ \{ .$$

QR uses the quotient-remainder primitive of J: the quotient and remainder of X divided by Y is given by $(0, Y) \# : X$. For example, the quotient and remainder of 99 divided by 50 is 1 49. The two-part divisor is formed by appending the head of C, which is the largest remaining coin, to 0. The head of C is trivial:

$$HC =: \{ . @ > @ \} .$$

The divisor is:

$$DR =: 0 : , HC$$

The dividend is the current tail of A:

$$TA =: \{ : @ > @ \{ .$$

And QR is now easily formed:

$$QR =: DR \# : TA$$

The new A is the curtailed current A appended with QR.

$$NA =: CA , QR$$

The new C is just the behead of the old C:

$$NC =: } . @ > @ \{ :$$

CS can now be used to obtain the successive results, as shown above. We need to find the number of times CS should be used, which is the number of coins in the coinage system, given by NS:

```
NS =: # @ > @ }.
```

A function which executes the change step function CS the correct number of times is ES:

```
ES =: CS ^: NS
```

After CS has been executed the proper number of times, the result is still two linked lists, the first list having a spurious item at the end, and the second list empty. To get the final result, the result of ES is razed and curtailed. The outermost function which encapsulates all that has preceded is MC:

```
MC =: }: @ ; @ ES
```

You'll notice, I'm sure, that this is a great many defined functions. Perhaps it says something about my attention span. I prefer to think that by making every function as simple as possible, with as few steps as are meaningful, it is easier for me to test for errors as I go along. Since J has the *fix* adverb (f.), it is easy to obtain a single longish function, which on my computer executes eleven times faster, although when displayed it probably appears quite daunting. It's not necessary to look at the result of *fix*, any more than one would look at the result of compiling a program written in a compiler environment. For the diehard masochist, I offer:

```
] q =: 5 !: 5 < 'Mcf' NB. in character form
):@;@(((}:@>@{. , (0: , {. @>@{.) #: {:@>@{.) ; }. @>@{.)^: (# @>@{.)
```

Notice that of the 45 tokens, ten are parentheses. The line above may make more sense if I use words for most of the tokens:

```
curtail@raze@(((curtail@A , (0 , head@C) #: tail@A) ; behead@C) ^: (tally@C))
```

For convenience, here are the functions that have been defined, in top-down order:

MC =: }: @ ; @ ES	NB. make change: curtail razE IS
ES =: CS ^: NS	NB. iterate change step NI times
NS =: # @ > @ }.	NB. how many iterations: count C
CS =: NA ; NC	NB. change step: new A link new C
NC =: }. @ > @ {:	NB. behead C
NA =: CA , QR	NB. new head: curtail A, append QR
CA =: }: @ > @ {.	NB. curtail A
QR =: DR #: TA	NB. divisor antibase tail A
TA =: {:@ > @ {.	NB. tail A
DR =: 0: , HC	NB. divisor: 0, head C
HC =: {.@ > @ }.	NB. head C

Tools, Part 2. Basics Too

by Dan Baronet (dbaronet@milinta.com)

In the following article I use terms specific to our trade. You won't find them in the dictionary but I assume the reader is familiar with words such as 'monad', 'global' (as a noun) and 'default' (verb). I use the terms 'nested' (as in 'nested array'), 'enclosed' or 'boxed' interchangeably. I also use quotes and angle brackets to help determine the type of the object I am referring to. 'Quotes' denote a variable or workspace and <angle brackets> refer to a function/operator or file. Often, the context is sufficient to remove ambiguities. Italicized words have a special meaning and often precede their definition. Optional items are shown within [brackets].

Introduction

When I started programming I confronted problems that needed deep investigation. Very often I was the given code and little information. I needed to search code for clues. The tools provided were crude and limited. This is when I started thinking about having my own function to search other functions for a given string.

This is the second of a series of articles on tools in APL.

A Function to Search Code

This function is one of the first of my tools. It was first written for APL/PC but was quickly ported to other platforms. It allows looking at code and reporting where specific strings are found. I wrote it when there was no such thing in the public domain or what was available was of limited use. It was written when nested arrays did not exist but was eventually adapted for them and the newer types (packages, namespaces, overlays...).

SEARCH utility

Keywords: STRING LOOKUP, STRING SEARCH, ARGUMENT, DELIMITER

This function will show where a series of strings occur in APL objects found in a workspace or in a file.

The evolution of software

This is another good example of code that evolved from simple to complex utility, something we should all have seen by now (think M\$).

An interesting thing, here, is the way the function works, on top of what it does. Let's see what it does first. I'll describe it just like it was built. I'll show how the prototype became the program.

The idea is simple: you take the visual representation of a function and do a search a la `fss` on it. For each hit you perform an action like showing the name of the function where the string is found. For example, assuming that functions `<fn1>` and `<plus>` exist in the workspace, doing

```
fss 'A+B'
fn1 plus
```

Would do it. Function `<fss>` uses `VR` to look into each function in the workspace and reports where the string given as right argument is found.

This is not enough. An improvement is to show the line where the string is found:

```
fss 'A+B'
fn1[3]  X+A+B=X
fn1[7]  R+A+B+C
plus[2]  Z+A+B
```

Better yet, use the left argument to specify functions to be searched and show the exact location of the string:

```
'fn1' fss 'A+B'
  ▽ fn1 (2 occurrences)
[3] X+A+B=X
    ^
[7] R+A+B+C
    ^
```

So far nothing special. `<fss>` is about 10 statements including comments (☺) A small improvement is to allow multiple strings to be searched. With nested strings we could do `∇/strings` code but I originally didn't have that luxury. A delimited string does the trick (the first character is the delimiter):

```
'fn1 show' fss '/A+B/X'
```

```

    ▽ fn1 (4 occurrences)
[3] X+A+B=X
    ^ ^ ^
[7] R+A+B+C
    ^
    ▽ show (6 occurrences)

[2] 'X=', *XX+A+B A XX is the answer
    ^   ^^ ^   ^^

```

So far so good. There were a few problems like searching for '[1]' which would report a hit for every function. This meant I had to recognize the decorators and mask them out. I found it simpler to use the ravel of `CR` of the function instead. Later, when `u` appeared it proved correct.

A new problem was that I was now looking at larger and larger workspaces. There were some specific cases that I needed covered:

- the ability to mask out comments
- the ability to mask out text
- the ability to mask out code
- the ability to perform syntactic searches

The first three cases are all related: on a line of code the comments follow the first lamp character (`A`) not in text. We must know where the text is first. Text is everything between 2 quotes, quotes included, not in comments. Code is everything else³. So, the expressions

```

T←Tv≠\T←line=''' A strings and quotes
Comments←v\T<line='A'

```

tell us where the comments are.

```

Text←T>Comments A this where the text really is
Body←Text←Comments A this is where the code (the body)
                    of the function is.

```

This code also works on matrices like `CR`.

³ Some text or comments may be viewed as code but this is the basic definition

The syntactic search is a bit more complicated but not terribly. It requires knowing where the characters forming a name are. More on that later.

With this in hand we can implement screening of the code. We need to specify what we want. We need a third argument.

So many arguments, so little room

Supplying a third argument to specify the behaviour of the function was impossible in the days when the first version was written. Sophisticated argument parsing was ruled out. The simplest way options could be specified was using single letters combinations. It was easier to incorporate them to the right argument, which was already a string. All that was needed to do was tack it on at either end. I opted for the beginning. The syntax was going to be:

```
[namelist] OSS '[options]/str1/.../strn'
```

The first character of the right argument would still be used as a delimiter and the *options* would be a series of characters, between the first two delimiters, one per option.

I changed the name to OSS for Object String Search when the function accepted character variables to be searched as well. At that point only matrices were searched but I extended the search to character scalars and vectors as well. To properly identify the lines where the hits were found in variables containing line delimiters (CRs) I decided to first transform them into matrices. I needed a function to transform such variables. I used a "classic" VTOM function⁴.

The options were now like this: *c* would exclude comments, *t* would exclude text and *b* would exclude the code (body). Thus, to look for string 'xyz' only in the code of function <ABC> I would enter:

```
'ABC' OSS '/tc/xyz' A mask out text and comments
```

By default only all the functions in the workspace were to be analysed. If everything had to be looked into, it had to be specified using (□n 1 2 3) as left argument.

⁴ This function can also be executed on all APLs, all others being too specific.

More requirements

As time went by further enhancements were made. The first one was the ability to consider executable strings as code. Strings are considered executable if they follow `⍺` or a trap assignment. This wasn't fool proof but worked well most of the time⁵. The code that modifies the text mask for a given line is

```
text←text>(1,1↓code∧line='∘') POS code∧(line='⍺')∨'␣ELX'⍋line
```

<POS> is a "Partitioned Or Scan" function.

The code for selecting this option was set to 'a' (APL mode).

Case insensitive

This one was easy. All that was needed was a function to turn all alphabets into a single one before doing the search. For APLs like APL+Win and SHARP APL an <UPPER> function would do but later, when Dyalog APL was included, the function would also have to include any character used in identifier formation like those of the third (CAPS underlined) alphabet.

The code for selecting this option was set to 'i'.

Search numeric

To include numeric variables in the search I added the 'n' option. Since the search string is itself character <OSS> does a format (`⌘`) of the variable before the search. This is not perfect⁶ but works well most of the time.

Syntactic searches

One of the interesting things to do is to be able to look for identifiers in code. If we need to know where the single letter variable 'a' is being used in the code, looking for the single letter 'a' won't help much, even if we screen out comments and text. Syntactic search is really what we want here. Pushed to the extreme, regular expression would be ideal but its implementation would require much more code. A simple way to provide next-to syntactic search is to eliminate all hits preceded or followed by a character used in making up an identifier. This assumes that we are looking for a specific name. Looking for '+ab' or '2x' is meaningless

⁵ For example, doing `⍺"See?"` would pick up 'See' as code when it is still text!

⁶ Floating point numbers can be a problem

(syntactically) and would not work. Again, knowing how the program works is useful. Looking for 'a+b' is acceptable.

If regular expressions are too much trouble, a small improvement consists in allowing searches to "start with" or "end with" a specific string. This is what I call left syntactic search and right syntactic search. This is easy to implement. To look for all names starting with 'ABC' we can use

```
OSS '/1/ABC'
```

To find all names ending in 'xyz' we can do

```
OSS '/r/xyz'
```

To find an exact match of 'a', use

```
OSS '/1r/a'
```

The code for selecting these options was set to 'l' for left syntactic search and 'r' for right syntactic search.

Local names, global names

After large workspaces started to appear programmers couldn't wait to fill them up with 1000s of objects. In one case I needed to clean an application and find out where some global variables were being used to determine if they needed to be localized. I found myself rummaging thru code looking for the elusive variable 'x'. To better find it I needed to exclude the functions where 'x' was localized. I came up with the *'exclude if a hit is found in the first line'* option. This is not real "localization" exclusion but it's close enough. To complete the exercise I added the ability to screen out code NOT found in the first line.

Obviously, this is mostly useful with functions or function representations.

I picked ';' as the code for "locals" screening option and 'G' for the "globals" screening option.

Looking into files

It appeared fairly quickly that scanning code in the workspace was insufficient. Many applications store code on file, each component of an APL file containing code or data. For APL+Win users, the code for functions comes in the form of either matrices (⌈CR) or CR delimited vectors (⌈VR). For other APLs such as SHARP APL, other data types are also available that PACK objects together. The initial version of OSS would skip components containing these type of objects, just

as it would skip them in the workspace, and wasn't therefore very useful. The later versions, however, allowed scanning inside packages (or `⊠OR` of namespaces for Dyalog APL, or overlays for APLX).

To scan files I *overloaded* the function. If the left argument was a list of numbers I determined they would mean FILE TIE NO, START CPT, END CPT, STEP with appropriate defaults for short arguments.

The code for selecting the option to look into PACKED objects recursively was set to '↓'.

To look for '⊠SIZE' in file tied to 9 from component 11 until the end, into the object if packed, I used

```
9 11 OSS ' /↓/⊠SIZE '
```

The result

OSS originally only displayed the hits. It didn't return anything. There came a time when I was only interested if a match was found, or rather, the number of matches found in each object scanned. I made <OSS> do just that. The result would be an N (number of objects searched) by S (number of strings looked for). If 4 functions were searched for 7 different strings I would return a 4x7 table of integers.

The option for returning the number of hits was set to '=' (or '+').

Patterns

Regular expressions are not recognized but there is a way to tell OSS to use an external function to do so. Should some code be available it should be assembled under the cover function <OSSPATMATCH>, which should take a pattern as left argument and the string to look for as right argument. It should return a boolean indicating where the matches are found. OSS will make a call to that function instead of ε if the option 'p' is supplied.

This option was introduced for the Y2K projects I was involved with. It proved invaluable when coupled with functions that could recognize regular expressions.

How the function works

With those specs in hand let's have a look at its implementation.

A stand-alone function

To be useful this function has to be brought in alone. No one wants to have to remember to also copy a slew of utility functions with a utility every time it's needed and have to erase them all when finished with it. The need to encapsulate the whole thing into a single function implies that utility functions like <POS> and <UPPER> be defined locally. The same for the <VTOM> function used to make character vectors into matrices. Furthermore, <VTOM> itself can be used in defining other functions. For example, to define the POS function I used

```
⊞FX MTOV ' * r+p POS s;t * s+s/p ⊙ r+≠\s\t≠~1⊥0,t+s/p'
```

The fact that ⊞FX is a standard system function means any APL can accept this statement.

Name conflicts

With all the names in use in the function it is now obvious I was going to run into a name to scan that I was already using. And I did. I resorted to using 'A+number' names for variables and a mixture of upper and lowercase letters for functions. It wouldn't guarantee I'd never run into problems but it would minimize the odds. For example, function <POS> became <pOs>.

- determine what kind of search is required
- build 5 functions that will be needed:

MTOV, POS and UPPER, which are always the same. MASK, which in its simplest form returns the scalar 1; in its complex form it returns a boolean of the same shape as its argument showing where hits are valid. For example, in the case where the comments and text are to be excluded from the search the function is

```

      ∇m←MASK m;c;t;b;a
[1]  b←c←t←t>c←v\ (mε'A')>t←t≠\t+mε'''' ⊙ m←tvc
      ∇

```

Finally, SEARCH is the function that loops on each string (remember there's no nested strings here) to look for and builds a boolean using various *globals*⁷. It takes

⁷ Semi-globals

into account the "appear in first line" case. It calls MASK to screen out unwanted portions.

... for each object to scan:

- get the matrix representation of the object
- look for matches using SEARCH; if any, display the hits

Restrictions

Quotes. You have to account for the APL syntax. For example, to look for the string "I can't" means you have to write something like

```
OSS '//I can''t' A 4 quotes for 2
```

GUI objects are not searched.

Size does matter. Larger objects may make OSS fail.

OSS with all these options remains a fairly simple piece of code. It spends some time determining the options, building the local functions before looping tightly on each name to scan. To limit the number of symbols (name conflicts) I reused variables and removed labels. The final version takes less than 6K⁸.

Wait, there's more

Replacing strings

This entire search is fine but what if you want to replace a string by another? I ran into that too. And created another function to deal with it.

One of the reasons for creating a separate function is the lookup method.

In OSS, the search is simple: we use `⌵`. In OSR (for Object Search and Replace), OSS's counterpart, the search is orderly, non-overlapping. This makes the searching code quite different. Furthermore, OSS looks into matrices, making reporting of lines easier. In OSR, there is no string shown and a different algorithm, using CR delimited strings, is used to find, mask and replace.

Some of the options don't apply in OSR. For example, the "return a result" option is not used.

⁸ remember the original function ran under APL*PLUS/PC in very small workspaces

Why do this again?

Again, when I started this project there was nothing like this on the market. Even today, APL+Win offers a user command `JLOC` or `JWSLOC` and Dyalog APL has library functions that can be put in `⎕SE` but none offers all these features at once. Furthermore, because I go from one APL to the other I like my tools to be consistent and made them so. So whether I use SHARP APL, Dyalog APL, APL+Win or APLX, I know how to use and expect the same result.

I have been using these functions for quite some time now. They've saved the day a few times.

You can find this code for all the above APLs at www.milinta.com.

Appendix

These are the codes accepted by OSS and OSR and their meaning

<u>Codes</u>	<u>Meaning</u>
C	exclude comments
T	exclude text
B	exclude code (body)
A	APL mode: treat strings after <code>⋄</code> as code
L	perform a left syntactic search
R	perform a right syntactic search
;	exclude if a hit is found in the first line
⎕	exclude if a hit is NOT found in the first line
I	insensitive search
N	search numeric
P	use pattern matching function <code>OSSPATMATCH</code> instead of <code>⎕</code> (OSS only)
↓	search recursively into packed objects (OSS only)
←	return a numeric result (OSS only)

New Foundations

by Graeme D. Robertson (email: GraemeDR@aol.com)

The foundations of APL should be extended to incorporate features of computer algebra systems, and extended beyond that to encompass arrays of functions and operators, in true APL style. This would push back the boundaries of VALUE ERRORS and SYNTAX ERRORS and open up a whole new world of mathematical analysis to APL programmers.

Functions Returning Functions

You might have noticed that in Dyalog APL it is possible to write a function that returns another function.

```

      ∇ FN←FOO
[1]   FN←{ω*2}
      ∇

```

This says that FOO returns a function called FN, which can then be applied to an array argument to give an array result. (A scalar is a rank zero array.)

```

      FOO
      ∇FN
      (FOO)3
9

```

This technique can be employed recursively to produce some very deep syntax. Here are three simple functions which combine to produce some unusual syntax, never imagined in [1].

```

      ∇ f←top
[1]   f←middle
      ∇

      ∇ f←a middle w
[1]   f←bottom
      ∇

      ∇ f←bottom w
[1]   f←{ω*2}
      ∇

```

To call the function **top** you need to employ syntax like this:

```
((2(top)3)3)4
16
```

How do we distinguish between a function (with array arguments) which returns a *function*, and an operator (with array operands) which returns a *function*? For example, how can we be sure that the primitive function Replicate (/) is not interpreted as an operator with an array left operand? Well, simply inserting a pair of parentheses in a way which is legal for operators but not for functions is one sure way to decide.

```
      3/5
5 5 5  (3/)5
SYNTAX ERROR
      (3/)5
      ^
```

This proves that Replicate is interpreted as a function. Whereas

```
(+)/5
5
```

proves that Reduce (/) is interpreted as an operator. So there is a clear difference.

Let us introduce some words in order to describe the syntax of **top** more precisely.

Definition: The valency of a function is **niladic**, **monadic** or **dyadic** depending upon whether it has zero, one (right) or two (one right and one left) arguments.

Axiom: A non-niladic function may be **ambivalent** - either monadic or dyadic depending contingently upon the presence or absence of a left argument.

Definition: The potency of an operator is **nihilistic**, **monistic** or **dualistic** depending upon whether it takes zero, one (left) or two (one left and one right) operands.

Axiom: An operator is **unipotent** - exclusively nihilistic, monistic or dualistic depending upon whether it is defined to take zero, one or two operands.

Now we can say that Replicate is a dyadic function and Reduce is a monistic operator which returns a monadic derived function. In the case of **top** above we can describe it as a niladic function which returns a dyadic function which returns a monadic function which returns another monadic function.

Operators Returning Operators

By their action on functions, operators usually *derive* new functions (i.e. return derived functions) which then usually take array arguments and *return* array results. Operators could conceivably take array operands and still return derived functions. The Composition operator ($\circ$) Form II of Dyalog APL takes an array left operand (and a function right operand). It might seem hard to imagine an operator which only takes array operands and still returns a useful function, however, the dyadic Circle function ($\circ$) could have been defined as a monistic operator, but it was not. It was defined as a dyadic function.

```
(1∘).1
SYNTAX ERROR
(1∘)0.1
^
1∘.1
0.09983341665
```

If operators were allowed to derive arrays, or if functions were allowed to return operators, would APL grammar, as constructed in [1], disappear completely into mud and mist? Do we want a few simple tight rules, or should we encourage every conceivable expression? Well, we need *some* rules in any language before any informative statements can be constructed. Then, given the rules, not every expression will be possible. So what is possible?

First, what are the rules? The two basic rules of first generation APL parsing are:

Rule 1: The **right argument of a function** is the evaluated result of the entire expression to its **right** (taking into account any special ordering dictated by parentheses).

Rule 2: A function **left operand of an operator** is the function derived from the longest possible operator sequence to its **left** (taking into account any special ordering dictated by parentheses).

Rule 1 leads to the slogan "APL works from right to left". However, operators mirror functions in that they capture their left argument with priority. These rules and definitions are sufficient to nail down a powerful grammar while still leaving a lot of flexibility for constructions using the different parts of speech - arrays, functions and operators.

Now consider the *dot syntax* in Dyalog APL. This is a particularly convenient notation because it mimics Visual Basic for Applications - almost exactly. The trouble with the notation, from an APL point of view, is that the Dot ($\cdot$) of *dot*

syntax has been given no proper place in APL grammar. Is it a function or an operator or what?

A dot is used for the decimal point. It may be possible to argue that in this construct - integer part on the left and fractional part on the right - the dot is being used as a dyadic function, but clearly there are counter-arguments to suggest that no such thing has ever been intended. We shall not pursue that hypothesis. Dot (.) is also used for Inner and Outer Products. For Inner Product, at least, it is treated as a dualistic operator which returns a dyadic function which returns an array result.

Considered as a dualistic operator, Dot (.) can be regarded as changing its definition depending upon the class of operands it is given. We are used to great flexibility and (perhaps too much) tolerance in the design of APL. Consider the way a token changes meaning depending upon context - the way a function changes with the presence of a left argument, or the way the Index Generator (ι) or Factorial function (!) change the underlying algorithm depending upon the rank or type of the argument, or consider the way that the symbol ≡ changes its meaning depending upon ≡ML, or whether a symbol (e.g. \) was legally overstruck with another symbol (e.g. o) ... Many of these tolerances stem from an era when symbols were precious. Fully Unicode systems should make symbols less rare, less precious, and just 'there for the taking'. The APL dialect 'J' might have been too quick to jettison extended character sets because Windows applications are catching up now.

The basic use of *dot syntax* is to form a full hereditary name of a namespace, exactly as is done, for example, in Excel or Word VBA (and in Dyalog .NET):

```
Application.WorkBooks.Workbook.Styles.Style
```

or

```
Application.Documents.Document.Paragraphs.Paragraph
```

These are the full ancestral names of objects which, naturally, inherit stuff from their parents.

A syntactically, almost imperceptibly different, new use of dot syntax is to form a namespace-qualified variable name, exactly as, for example, in Outlook VBA:

```
Application.MailItem.Recipients.User.Type
```

If Type in namespace Application.MailItem.Recipients.User is set to 3 (Enum olBCC) then the message will be sent to the recipient as a blank carbon

copy. Type is the name of a *property* of an object, in VBA parlance. In APL it is syntactically synonymous with the name of a *variable* in a namespace.

Another example of a property (variable) of an object (namespace) is Count in a collection object:

```
Application.WorkBooks.Workbook.Worksheets.Count
```

The value of Count is read like a variable but it can only be changed by adding or deleting worksheets in some other way. Count is a sort of read-only variable. Special types of variables have long been a part of APL, at least since shared variables were released in APL SV by IBM in 1973.

But the syntax of VBA is more arbitrary than that of Dyalog APL. The Item *property* of a collection object takes an index argument as if it were a *method* (and returns an item object from the collection). The Item property cannot be set and can only be used with an argument. Furthermore VBA has the concept of a default method or property which is associated with a given object. For example, the Worksheets collection assumes that you mean

```
Application.WorkBooks.Workbook.Worksheets.Item(3)
```

when you write

```
Application.WorkBooks.Workbook.Worksheets(3)
```

because the default property of the Worksheets collection is Item. APL is more rigorous. Implicit defaults have to be made explicit when translating from VBA to APL.

Methods associated with objects in VBA are essentially *functions* in namespaces for APL. Consider the method

```
Application.Documents.Document.PrintOut
```

This method looks like a niladic function and causes the document to be printed, whereas

```
Application.Documents.Document.PrintOut(Range:=wdPrintCurrentPage)
```

makes the method appear as a monadic function. The basic contradiction in APL of having a name act contingently as a monadic function or a niladic function (a contradiction nevertheless exhibited by $\rightarrow$) is avoided in APL from VBA by insisting that the niladic case of a method is given the dummy argument of zilde (θ). Such dummy arguments appear in many places in Dyalog APL code because so many VBA methods may be called with zero or more right arguments

(employing *constructors* with *overloading* to determine the precise method to use). If a pair of empty APL parentheses were defined to be zilde inside the interpreter then the APL code might appear more legible and more compatible with VBA.

Definition: $() \equiv \emptyset$

We take the Dot of *dot syntax* to be a dualistic operator that can assume various forms. Let us remind ourselves of the various flavours of Dot.

We recognise the Dot of Inner Product to be a dualistic operator which takes two dyadic function operands and returns a derived dyadic function. In the case where the functions are $+$ and $\times$ and the arguments are matrices, the derived function $+\times$ follows the formula for matrix multiplication.

If the Dot has a namespace to the left and a namespace to the right, then Dot returns the child namespace as an object. If the Dot has a namespace to the left and an array to the right, then it returns the array as it would be found in the left namespace. These two cases are syntactically equivalent because namespaces and arrays are both essentially class 2 objects. If the Dot has a namespace to the left and a function (monadic or dyadic) to the right, then Dot returns that function as it would be found in the left namespace.

The reason why Dot in this context works so much like an operator is because of Rule 2 above which ensures that the namespace to the left is constructed starting from the leftmost, and therefore most distant, ancestor. In other words, the parser ensures that namespace

$N1.N2.N3.N4.N5.N6.N7$

is interpreted according to explicit ordering

$(((((N1.N2).N3).N4).N5).N6).N7$

The main point of difference to notice at this stage between this Dot operator and all previous examples of operators in APL is the fact that the operator has two essentially class 2 operands and returns a derived class 2 object. In terms of the definition in [1] of parsing character, this flavour of Dot has character 0 0 0, whereas the Dot of Inner Product has character 2 2 2. (Parsing character was defined as the derived function valency catenated to the left operand valency (or zilde) catenated to the right operand valency (or zilde).) In the case of $N.f$ Dot has character 1 0 1 or 2 0 2. These different characters do not seem to conflict in the interpreter. The parser seems to cope.

However, operators are also susceptible to dot syntax. It is possible to have a namespace on the left and an operator on the right, in which case an operator is returned.

This totally confounds parsing character in [1]. It is, on reflection, not unknown in mathematics that an operator should act on an operator to produce another operator. For example, the second derivative operator is formed from the action of the first derivative operator on itself:

$$\frac{\partial}{\partial x}(\frac{\partial}{\partial x}) = \frac{\partial^2}{\partial x^2}$$

So now, apparently, dot syntax allows operators to take operators as operands and Dot derives new operators. For example, given namespace #.a.b we can define and use a defined operator in relation to operator Dot.

```
#.a.b.op={αα ω}
*#.a.b.op 3
2.718281828 7.389056099 20.08553692
```

In this example, the construction #.a.b.op returns an operator. Dot can have a namespace to the left and an operator to the right, and return an operator.

The pictograms in [1] describing primitive syntax become arbitrarily tall at the function level because functions can return functions. They also become arbitrarily tall at the operator level because operators can return operators. Exhaustive encapsulation in pictograms of all possible primitive syntax becomes theoretically impossible.

3. Promotion of primitives $s++ -- xx \div \quad !! [] \boxtimes ** \otimes \otimes \phi \phi , , pp \div x x \circ q x x$

While on the one hand this new found freedom is frustrating to those who favour tight rules, on the other hand it reveals a panorama of possibilities to those who are happy to write unintelligible code.

Use the difficulty!

If something that was once clearly an operator ($f.g$) can now also be parsed consistently as something which looks like a classic function ($N1.N2$), then can the reverse be done? Can something which was once only known as a function, also be consistently used as an operator? Can we promote some primitive functions to useful operators?

One of the modern desires of *A Programming Language* is the possibility of arrays of functions. Such arrays are used all over the place in mathematics; tensors are arrays of functions of arbitrary rank whose form is invariant under a change of basis. (It is vital for all laws of physics that their form does not change with a change of coordinate system, therefore all laws are formulated as tensor equations.)

The first question is how to make a vector of functions. If we can promote Catenate (,) to an operator when it is applied to functions then we can use it to create vectors of functions in the same way as it is used to produce vectors of arrays.

$$VF \leftarrow \{1\}, \{\omega\}, \{\omega * 2\}, \{\omega * 3\}$$

We can promote Reshape (ρ) to operator level if the right argument is a function, or function array.

$$MF \leftarrow 3 \rho VF$$

We can promote indexing so that

$$25 \quad (MF[1;3])5$$

We can form the inner product of two such matrices by promoting $+ \cdot \times$ to operator level in the case that at least one of the derived function arguments is a function array.

$$MF3 \leftarrow MF1 + \cdot \times MF2$$

Note that Plus and Times in this situation are both promoted to operators. The promotion of Plus is so prevalent in mathematics that it is easy to overlook the fact that its definition has subtly changed. We move from adding numbers as in $3 + 4$, to adding functions as in $f(x) + g(x)$, to adding operators as in

$$\frac{\partial}{\partial x} + i \frac{\partial}{\partial y}$$

with hardly a thought as to the evolving definition of Plus in each case. This could be accommodated in APL if Plus can be promoted depending upon the class of the adjacent tokens. The following example would add operators together by promoting $+$ to an operator which acts on other operators to produce a new operator (left to right):

$$Op \leftarrow \{\alpha \alpha \omega\} + \{\alpha \alpha \alpha \omega\} + \{\alpha \alpha \alpha \alpha \omega\}$$

which is equivalent to

$$\text{Op} \leftrightarrow \{ \{ \alpha \alpha \ \omega \}, \{ \alpha \alpha \ \alpha \alpha \ \omega \}, \{ \alpha \alpha \ \alpha \alpha \ \alpha \alpha \ \omega \} \}$$

So we can now see that a number of primitives may be applied in many different ways to both functions and operators.

Vectors of operators are introduced in vector analysis. For example, the Gradient operator is defined as

$$\nabla = \left(\frac{\partial}{\partial x}, \frac{\partial}{\partial y}, \frac{\partial}{\partial z} \right)$$

This monistic operator takes a scalar monadic function of a 3-vector (x, y, z) as its operand and returns a 3-vector of scalar gradient functions. The Divergence operator is written as a scalar product $\nabla \cdot V$ which is mathematical shorthand for $\nabla \cdot V$ in which Times is more like Composition. Divergence takes a 3-vector function and returns a scalar function:

$$V(x, y, z) = (f(x, y, z), g(x, y, z), h(x, y, z))$$

$$\nabla \cdot V(x, y, z) = \left(\frac{\partial}{\partial x}, \frac{\partial}{\partial y}, \frac{\partial}{\partial z} \right) \cdot V(x, y, z)$$

$$= \frac{\partial}{\partial x} f(x, y, z) + \frac{\partial}{\partial y} g(x, y, z) + \frac{\partial}{\partial z} h(x, y, z)$$

The Curl operator takes a vector function operand and derives a vector function result. The Curl operator can be defined as the vector cross product of the vector Gradient operator with the vector function operand. The vector cross product is a very special type of anti-symmetric vector multiplication which only exists in 3 and 7 dimensions (because these are the remnant vector parts of the quaternion and octonion fields respectively). The inner product of the Gradient operator with itself is also a very significant operator.

Notice the use of a dot and its analogy with inner product. For Divergence, an inner product is applied between a vector operator ∇ and a vector function V . The functions $f(x, y, z)$, $g(x, y, z)$ and $h(x, y, z)$ are assumed to be scalar monadic functions which assign real numbers to each point of a region in the 3D space with coordinates x , y and z . So the function $V(x, y, z)$ is a monadic vector function of a vector which assigns a 3-vector of real numbers to each point in a 3D space (eg the velocity of a fluid at a point). Vector functions occur frequently in mathematical applications. A simple example is

$$V(x,y,z) = (xz, -xy^2, yz^2)$$

which might be programmed as

$$V \leftarrow \{\omega[1] \times \omega[3]\}, \{-\omega[1] \times \omega[2]^2\}, \{\omega[2] \times \omega[3]^2\}$$

and used as a rank 1 1 function:

$$\begin{matrix} & V & (1 & 2 & 3) \\ 3 & -4 & 18 \end{matrix}$$

An example of a *matrix operator* is the Dirac derivative which is a 4 by 4 matrix of partial derivatives with complex coefficients.

Ken Iverson has introduced in considerable detail [2] a typically brilliant attempt to implement vector analysis into APL. However, he does not have the luxury of native vector functions and operators nor the concept of promoted primitives to ease the transition from arithmetic to analysis. Nevertheless, it might be time to revisit this important work.

In order to implement arrays of functions and operators, it is probably necessary to considerably modify any existing APL interpreter. It will therefore be of interest to note that there are two GNU C++ libraries which would greatly assist with this non-trivial project to convert APL into a great algebraic (as well as a great arithmetic) language. The first is CLN - Class Library for Numbers [3]. This suite of programs would allow APL to incorporate easily many new types of numbers, including unlimited precision integers, rational numbers, unlimited precision floats and complex numbers, as well as many elementary functions that act on such numbers.

The second potentially very useful library is GiNaC - which stands for GiNaC Is Not A CAS [4,5]; where CAS means Computer Algebra System. (Not Germanium Sodium Carbide? - the recursive name itself, GiNaC, is worthy of an APL application!) GiNaC is a set of building blocks, based on CLN, for performing algebraic operations on functions and variables, such as forming derivatives or simplifying or expanding algebraic expressions. GiNaC was developed with the intention of replacing a Maple based system with a flexible modular system designed round a set of C++ programs with algebraic capabilities that could be incorporated easily into larger C++ programs. This 'larger program' could and should be an APL interpreter.

References

- [1] G.D.Robertson, *APL Linguistics*, Vector 2.2 (1985) 118-126
 - [2] K.E.Iverson, *The Derivative Operator*, Proceedings of APL79 (1979) 347-354
 - [3] <http://www.ginac.de/CLN>
 - [4] <http://www.ginac.de>
 - [5] C.Bauer et al, *Introduction to the GiNaC Framework for Symbolic Computation within the C++ Programming Language*, J.Symbolic Computation 33 (2002) 1-12
-

Index to Advertisers

Dyadic Systems Ltd	2
MicroAPL Ltd	4
Strand Software	6
Vector Back Numbers	5

All queries regarding advertising in VECTOR should be made to Gill Smith, at 01439-788385, Email: apl385@compuserve.com.

Submitting Material to Vector

The Vector working group meets towards the end of the month in which Vector appears; we review material for issue n+1 and discuss themes for issues n+2 onwards. Please send the text of submitted articles (hardcopy with diskette as appropriate) to the Vector Working Group via:

Vector Administration, c/o Gill Smith
Brook House
Gilling East
YORK, YO62 4JJ
Tel: +44 (0) 1439-788385
Email: apl385@compuserve.com

Authors wishing to use Word for Windows should contact Vector Production for a copy of the APL2741 TrueType font, and a suitable Winword template. These may also be downloaded from the Vector web site at www.vector.org.uk

Camera-ready artwork (e.g. advertisements) and diskettes of 'standard' material (e.g. sustaining members' news) should be sent to Vector Production, Brook House, Gilling East, YORK YO62 4JJ. Please also copy us with all electronically submitted material so that we have early warning of possible problems.

Subscribing to Vector

Your Vector subscription includes membership of the British APL Association, which is open to anyone interested in APL or related languages. The membership year runs from 1st May to 30th April. The British APL Association is a special interest group of the British Computer Society, Reg. Charity No. 292,786

Name: _____

Address: _____

Postcode / Country: _____

Telephone Number: _____

Email Address: _____

UK private membership	£20	☐
Overseas private membership	£22	☐
Airmail supplement (not needed for Europe)	£4	☐
UK Corporate membership	£100	☐
Corporate membership overseas	£135	☐
Sustaining membership	£500	☐
Non-voting UK member (student/OAP/unemployed)	£10	☐

PAYMENT – in Sterling or by Visa/Mastercard/JCB only

Payment should be enclosed with membership applications in the form of a UK Sterling cheque to "The British APL Association", or you may quote your Mastercard, Visa or JCB number.

I authorise you to debit my Visa/Mastercard/JCB account

Number: Expiry date: |

for the membership category indicated above,

- ☐ annually, at the prevailing rate, until further notice
☐ one year's subscription only

Signature: _____

Data Protection Act:
The information supplied may be stored on computer and processed in accordance with the registration of the British Computer Society.

Send the completed form to:

BAA, c/o Rowena Small, 12 Cambridge Road, Waterbeach, CAMBRIDGE CB5 9NJ, UK

Fax: +44 (0) 1653 697719

The British APL Association

The British APL Association is a Specialist Group of the British Computer Society. It is administered by a Committee of officers who are elected by a postal ballot of Association members prior to the Annual General Meeting. Working groups are also established in areas such as activity planning and journal production. Offers of assistance and involvement with any Association matters are welcomed and should be addressed in the first instance to the Secretary.

2003/2004 Committee

Chairman	Adrian Smith 01439-788385 apl385@compuserve.com	Brook House Gilling East YORK YO62 4JJ
Secretary	Anthony Camacho 0117-973-0036 acam@blueyonder.co.uk	11 Auburn Road Redland BRISTOL, BS6 6LS
Treasurer	Nicholas Small 01223-570850 treas.apl@bcs.org.uk	12 Cambridge Road, Waterbeach, Cambridge CB5 9NJ
Journal Editor	Stefano Lanzavecchia stf@apl.it	c/o APL Italiana Corso Vercelli 54 20145 - Milano Italy
Projects and Publicity	Dr Alan Mayer 01792-205678x4274 a.d.mayer@swansea.ac.uk	European Business Management School, Swansea University, Singleton Park, SWANSEA SA2 8PP
Webmaster	Bob Hoekstra 01483-771028 bob.hoekstra@hoekstrasystems.ltd.uk	Dominique, Salisbury Road WOKING, Surrey GU33 7UR
Activities	Jon Sandles 01904-612882 jon_sandles@csi.com	138 Burton Stone Lane, YORK YO30 6DF
School Project	Stephen Taylor sjt@lambenttechnology.com	
Administration	Rowena Small 01223-570850 treas.apl@bcs.org.uk	12 Cambridge Road, Waterbeach, Cambridge CB5 9NJ

Journal Working Group

Editor:	Stefano Lanzavecchia	see above
Production:	Adrian & Gill Smith	Brook House, Gilling East, YORK (01439-788385)
Advertising:	Gill Smith	Brook House, Gilling East, YORK (01439-788385)
Support Team:	Jonathan Barman (01488-648575), Anthony & Sylvia Camacho (0117-973-0036), Ray Cannon (01252-874697), Stephen Taylor (077-1340-0852) Bob Hoekstra (01483-771028), Jon Sandles (01904-612882), Marc Griffiths	

VECTOR

VECTOR is the quarterly Journal of the British APL Association and is distributed to Association members in the UK and overseas. The British APL Association is a Specialist Group of the British Computer Society. APL stands for 'A Programming Language' - an interactive computer language noted for its elegance, conciseness and fast development speed. It is supported on most mainframes, workstations and personal computers.

SUSTAINING MEMBERS

The Committee of the British APL Association wish to acknowledge the generous financial support of the following Association Sustaining Members. In many cases these organisations also provide manpower and administrative assistance to the Association at their own cost.

Causeway Graphics Systems Ltd
The Maltings, DeWegins
MILTON, North York YO17 7DP
Tel: 01483-667400
Fax: 01483-697778
Email: sales@causeway.co.uk
Web: www.causeway.co.uk

Dyking Ltd
Grave-House, Luffwell Chase
Chesham Court, BASINGSTOKE
Hants RG24 9AG
Tel: 01256-318401
Fax: 01256-316559
Email: sales@dyking.com
Web: www.dyking.com

Insight Systems ApS
Næstved Strychnæ 1100
DK-3150 Hulslev
Denmark
Tel: +45 70 26 13 20
Fax: +45 70 26 13 25
Email: info@insight.dk
Web: www.insight.dk

MemoAPL Ltd
The Rover Mill, Mill Lane
Uckfield
E Sussex TN22 5AA
Tel: 01825-780090 Fax: 01825-740472
Email: MemoAPL@memoapl.demon.co.uk
Web: www.memoapl.co.uk/mpl

Compage Ltd
Company House
88 Priestley Road
GUILDFORD, Surrey GU2 5YU
Tel: 01483-514590

APL2000 Inc
One Research Court
Suite 140
Rockville MD 20850
USA
Tel: +1 (301) 208 7180
Email: sales@apl2000.com
Web: www.apl2000.com

HMW Computing
Katherine House
1 Temple Avenue
LONDON EC4Y 0HA
Tel: 0870-1916-488
Email: HMW@ukbra.com

Selwyn Associates Ltd
Groot Blaauwberg 5.2
1062 AC Amsterdam
Netherlands
Tel: +31 20 646 4475
Fax: +31 20 644 1228
Email: sales@selwyn.com